



Universidad
Rey Juan Carlos

Escuela Técnica Superior de Ingeniería Informática

Ingeniería de datos meteorológicos del territorio español con el framework de big data Spark y Scala

Memoria del Trabajo Fin de Grado
en Ingeniería Informática

Autor:

Alberto Pérez Pérez

Tutor: Juan Manuel Serrano Hidalgo

Marzo 2026

Agradecimientos

Después de un largo camino, desde mis comienzos en la universidad en septiembre de 2018, me enorgullece haber sido capaz de terminar estos estudios que tanto ansiaba desde que tengo uso de razón y memoria. Todo este esfuerzo, sin embargo, no podría ser posible sin la gran ayuda y labor de las personas mas cercanas a mi que siempre formaron parte de mi red de apoyo.

Agradezco enormemente la implicación de mis padres y mi hermano, que día a día me han dado ánimos para seguir luchando por mis sueños, a pesar de las adversidades, desviéndose todo lo necesario para que pueda conseguirlo. También a mi abuela, que desde algún lugar se que eternamente estará mirándome orgullosa viendo el gran profesional en el que poco a poco voy convirtiéndome.

Agradezco también el apoyo incondicional de mi pareja, así como el de mis amigos, dándome siempre el impulso para avanzar, aunque haya habido ocasiones en las que parecía que las ganas me lo iban a impedir.

Otro agradecimiento al equipo docente de la universidad, que me han enseñado todo lo necesario para poder formarme, no solo como ingeniero, si no también como trabajador y ciudadano del mundo, así como especialmente a mi tutor, Juanma, que desde el primer momento ha estado atento a brindar su ayuda y conocimiento para el desempeño de este trabajo.

Gracias a todas las personas de mi entorno, a día de hoy, puedo decir que he podido realizarme como persona y como ingeniero informático. Muchas gracias, de todo corazón.

Resumen

Este trabajo fin de grado se centra en la extracción masiva de datos meteorológicos del territorio español y su posterior tratamiento con el fin de conseguir resultados que sean capaces de ser interpretados mediante un proceso completamente informatizado. Este tipo de estudios, en un mundo donde la información es uno de los bienes más valiosos del ser humano, son fundamentales para el conocimiento del entorno, especialmente, el clima, un elemento capital en la vida de todas las personas, cuyo entendimiento condiciona el día a día.

Con todo ello, se ha decidido confeccionar un sistema de extracción, consulta y análisis usando el lenguaje Scala y haciendo uso del *framework* Spark, creando una aplicación de big data que extrae información de los repositorios de datos abiertos tanto de la Agencia Estatal de Meteorología (AEMET) y del Instituto Andaluz de Investigación y Formación Agraria, Pesquera, Alimentaria y de la Producción Ecológica (IFAPA). Posteriormente se procesa, se da formato a la información y se extraen conclusiones mediante diferentes consultas con las que finalmente se generan gráficos, valiéndose de un servicio web desarrollado en lenguaje Python, a modo de resultado.

Estos resultados podrán ser accedidos desde una página web habilitada a cualquier usuario que desee consultar la información. Con ello, se pueden apreciar resultados sobre diferentes estudios como la distribución de estaciones meteorológicas por altitud, provincia...etc. Análisis de los diferentes climas de España. Estudios sobre variables climáticas individuales como la temperatura, presión o radiación solar, mostrando su evolución temporal, máximos y mínimos históricos, distribución a lo largo del territorio español...etc. Finalmente, estudios interesantes como la correlación entre la precipitación y la presión o lugares más propensos a sufrir eventos meteorológicos adversos o idóneos para el uso de elementos naturales como el viento o la luz solar.

Palabras clave

Análisis meteorológico del territorio español.

AEMET e IFAPA.

Estudio de big data con Scala y Spark.

Generación de gráficos con Python y Plotly.

Despliegue con Docker y Amazon Web Services.

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Estado del arte	2
1.2.1	Aspectos meteorológicos	2
1.2.2	Aspectos sobre el tratamiento y estudio de los datos	3
1.2.3	Situación actual de las herramientas y tecnologías usadas	4
1.3	Objetivos	8
1.4	Estructura de la memoria	8
2	Análisis	9
2.1	Descripción del problema	9
2.1.1	Fase de extracción de datos	9
2.1.2	Fase de estudios sobre los datos	12
2.1.3	Fase de representación de resultados	15
2.1.4	Aspectos comunes en todas las fases	17
2.1.5	Aspectos complementarios	18
2.2	Requisitos	19
2.3	Metodología	19
2.3.1	Metodología de trabajo	19
2.3.2	Plan de acción para desarrollar la solución	20
2.3.3	Herramientas utilizadas	21
3	Diseño	25
3.1	Arquitectura e implementación del <i>software</i>	25
3.1.1	Introducción al sistema	25
3.1.2	Sistema principal	26
3.1.3	Sistema Auto-Plot para generación de gráficos	42
3.1.4	Sitio web para visualización de resultados	50
3.2	Despliegue	51
3.2.1	Compilación del proyecto principal	51
3.2.2	Despliegue en el ecosistema local	52
3.2.3	Despliegue de la fase de estudios con Spark en AWS EMR	53
4	Resultados y métricas	55
4.1	Resultados de la ejecución	55
4.1.1	Estudios sobre estaciones meteorológicas	55
4.2	Estudios sobre distribución climática	56
4.3	Estudios sobre variables climáticas individuales	57

4.4	Estudios climáticos de interés	59
4.5	Métricas del desarrollo	61
4.5.1	Tiempos de desarrollo y otras métricas	61
4.5.2	Tiempos de ejecución en local	61
4.5.3	Métricas específicas de la fase de estudios con Spark	62
5	Conclusiones	65
5.1	Objetivos cumplidos	65
5.2	Trabajos futuros	66
A	Información complementaria	67
A.1	Tablas de requisitos	68
A.1.1	Requisitos funcionales de la fase de extracción de datos	68
A.1.2	Requisitos funcionales de la fase de estudios con los datos	69
A.1.3	Requisitos funcionales de la fase de representación de resultados	70
A.1.4	Requisitos funcionales sobre aspectos comunes en todas las fases	71
A.1.5	Requisitos funcionales sobre aspectos complementarios	72
A.1.6	Requisitos no funcionales	72
A.2	Jerarquía de ficheros y directorios del sistema principal	74
A.3	Visión global del sistema principal	78
A.4	Detalles sobre consultas en Spark	79
A.5	Detalles sobre el sistema de almacenamiento	85
A.6	Jerarquía de ficheros y directorios de Auto-Plot	88
A.7	Visión global de Auto-Plot	90
A.8	Algoritmo para crear mapas de calor	91
B	Diagramas del proyecto	93
	Bibliografía	113

Índice de tablas

1	Descripción de los campos de información de estaciones de AEMET	10
2	Descripción de los campos de datos meteorológicos diarios de AEMET	11
3	Descripción de los campos de información de provincia de IFAPA	11
4	Descripción de los campos de información de estaciones de IFAPA	12
5	Descripción de los campos de datos meteorológicos diarios de IFAPA	12
6	Clasificación climática de Köppen-Geiger y su localización en España	13
7	Tiempos que ha demorado el desarrollo para cada fase del proyecto	61
8	Tiempos de ejecución por proceso	62
10	Comparación de métricas generales de ejecución entre el entorno local y EMR en AWS	62
11	Comparación de métricas de los Jobs en el entorno local y EMR en AWS	64
12	Listado de requisitos funcionales del sistema de extracción de datos de AEMET .	68
13	Listado de requisitos funcionales del sistema de extracción de datos de IFAPA .	68
14	Listado de requisitos funcionales del sistema de conversión de los datos de IFAPA al esquema definido por AEMET	68
15	Listado de requisitos funcionales del sistema de transformación del formato de los datos extraídos	69
16	Listado de requisitos funcionales de los estudios sobre estaciones meteorológicas .	69
17	Listado de requisitos funcionales de los estudios de distribución climática . .	69
18	Listado de requisitos funcionales de los estudios sobre variables climáticas individuales	70
19	Listado de requisitos funcionales de los estudios meteorológicos interesantes .	70
20	Listado de requisitos funcionales generales para la representación de resultados .	70
21	Listado de requisitos funcionales de los resultados de los estudios sobre estaciones meteorológicas	70
22	Listado de requisitos funcionales de los estudios de distribución climática . .	71
23	Listado de requisitos funcionales de los estudios sobre variables climáticas individuales	71
24	Listado de requisitos funcionales de los estudios meteorológicos interesantes .	71
25	Listado de requisitos funcionales sobre valores constantes	71
26	Listado de requisitos funcionales de almacenamiento	72
27	Listado de requisitos funcionales sobre funciones auxiliares del sistema	72
28	Listado de requisitos funcionales sobre aspectos complementarios	72
29	Listado de requisitos no funcionales de arquitectura	72
30	Listado de requisitos no funcionales de fiabilidad	73
31	Listado de requisitos no funcionales de despliegue	73
32	Listado de requisitos no funcionales de documentación	73

33	Ejemplo del resultado de la consulta <code>getStationInfoById()</code>	79
34	Ejemplo del resultado de la consulta <code>getStationCountByColumnInLapse()</code> . .	80
35	Ejemplo del resultado de la consulta <code>getStationsCountByParamIntervalsInALapse()</code>	80
36	Ejemplo del resultado de la consulta <code>getStationMonthlyAvgTempAndSumPrecInAYear()</code>	81
37	Ejemplo del resultado de la consulta <code>getClimateParamInALapseById()</code> . . .	81
38	Ejemplo del resultado de la consulta <code>getTopNClimateParamInALapse()</code> . . .	82
39	Ejemplo del resultado de la consulta <code>getTopNClimateConditionsInALapse()</code> .	82
40	Ejemplo del resultado de la consulta <code>getClimateYearlyGroupById()</code>	83
41	Ejemplo del resultado de la consulta <code>getAllStationsByStatesAvgClimateParamInALapse()</code>	83
42	Ejemplo del resultado de la consulta <code>getStationClimateParamRegressionModelInALapse()</code>	84
43	Ejemplo del resultado de la consulta <code>getTopNClimateParamIncrementInAYearLapse()</code>	84

Índice de figuras

1	Tablero de Trello con sus columnas de gestión correspondientes y tarjetas en diferentes fases de desarrollo	20
2	Diagrama de componentes simplificado del sistema principal	26
3	Código de la función <code>getAemetAPIResource()</code>	27
4	Comparación de los JSON extraídos de AEMET sobre estaciones y registros meteorológicos	28
5	Comparación de los JSON extraídos de IFAPA sobre estación y registros meteorológicos	30
6	Código para la escritura del JSON de IFAPA con formato AEMET	30
7	Código de la función <code>createParquetSchemaFromMetadata()</code>	32
8	Código para formatear los datos del dataframe de estaciones	34
9	Código de la función <code>simpleFetchAndSave()</code>	36
10	Código de la ejecución de la sección de un estudio	38
11	Código de la función <code>barDTOTop5IncFormatter()</code>	39
12	Captura de pantalla de la consola durante el proceso de extracción de datos de AEMET	41
13	Diagrama de componentes simplificado de Auto-Plot	43
14	Código con un extracto de la definición de un DTO	44
15	Código con la definición de una clase del modelo de datos	46
16	Código de un controlador	47
17	Código con un ejemplo de construcción de gráfico con Plotly	48
18	Captura de pantalla de la web de visualización de resultados	51
19	Gráfico de sectores circulares con la distribución de estaciones por intervalos de altitud	56
20	Gráfico tipo climograma de Bilbao, Bizkaia, durante el año 2024	56
21	Gráfico de barras con los 10 lugares con mayor humedad durante 2024	57
22	Gráfico de barras con los 5 lugares con mayores decrementos en sus precipitaciones durante 2024	58
23	Gráfico lineal con recta de regresión que muestra la evolución de las temperaturas desde 1973 hasta 2024 en Sevilla	58
24	Gráfico con un mapa de calor de la distribución de la radiación solar a lo largo del territorio continental español durante 2024	59
25	Gráfico lineal doble con la evolución de las precipitaciones y la presión atmosférica durante el año 2024 en Getafe, Madrid	60
26	Gráfico en formato tabla con las 10 provincias más propensas a padecer lluvias torrenciales entre 2014 y 2024	60

27	Estructura de directorios en la raíz del sistema principal	74
28	Estructura de directorios del módulo modules	74
29	Estructura de directorios y archivos general para todas las fases	75
30	Estructura de directorios y archivos del código fuente principal de la fase de extracción de datos	76
31	Estructura de directorios y archivos del código fuente principal de la fase de estudios sobre los datos	76
32	Estructura de directorios y archivos del código fuente principal de la fase de representación de resultados	77
33	Estructura de directorios y archivos del módulo utils (dividida a partir de Storage)	77
34	Estructura de directorios en la raíz del sistema Auto-Plot	88
35	Estructura de directorios de Api en Auto-Plot	89
36	Estructura de directorios de Plotters en Auto-Plot	89
37	Estructura de directorios de Utils en Auto-Plot	89
38	Diagrama de componentes del sistema principal	94
39	Diagrama de ejecución del proceso de extracción de datos de la API de AEMET	95
40	Diagrama de ejecución del proceso de extracción de datos de la API de IFAPA	96
41	Diagrama de ejecución del proceso de conversión del esquema de datos de IFAPA al de AEMET	97
42	Diagrama de ejecución del proceso de transformación del formato de los datos	98
43	Diagrama de ejecución del proceso de arranque y finalización de la sesión de Spark	99
44	Diagrama de actividad de la consulta getStationInfoById()	100
45	Diagrama de actividad de la consulta getStationCountByColumnInLapse()	100
46	Diagrama de actividad de la consulta getStationsCountByParamIntervalsInALapse()	101
47	Diagrama de actividad de la consulta getStationMonthlyAvgTempAndSumPrecInAYear()	102
48	Diagrama de actividad de la consulta getClimateParamInALapseById()	102
49	Diagrama de actividad de la consulta getTopNClimateParamInALapse()	103
50	Diagrama de actividad de la consulta getTopNClimateConditionsInALapse()	104
51	Diagrama de actividad de la consulta getClimateYearlyGroupById()	105
52	Diagrama de actividad de la consulta getAllStationsByStatesAvgClimateParamInALapse()	106
53	Diagrama de actividad de la consulta getStationClimateParamRegressionModelInALapse()	107
54	Diagrama de actividad de la consulta getTopNClimateParamIncrementInAYearLapse()	108
55	Diagrama de ejecución del sistema de almacenamiento para archivos JSON	109
58	Captura de pantalla de la SparkUI con el DAG de la consulta getAllStationsByStatesAvgClimateParamInALapse()	110
56	Diagrama de componentes del sistema Auto-Plot	111
57	JSON presente en el cuerpo de una petición para la generación de un gráfico lineal en Auto-Plot	112

Nomenclatura

AEMET	Agencia Estatal de Meteorología
API	Application Programming Interface
AWS	Amazon Web Services
DAG	Directed Acyclic Graph
DF	Dataframe
DOM	Document Object Model
DTO	Data Transfer Object
IA	Inteligencia artificial
IDE	Integrated Development Environment
IFAPA	Instituto Andaluz de Investigación y Formación Agraria, Pesquera, Alimentaria y de la Producción Ecológica
IoT	Internet of Things
JVM	Java Virtual Machine
REST	Representational State Transfer
RIA	Red de Información Agroclimática de Andalucía
TFG	Trabajo Fin de Grado
UML	Unified Modeling Language
WSGI	Web Server Gateway Interface

Capítulo 1

Introducción

Durante este capítulo, se describirá el proyecto que se ha creado y ejecutado, mostrando la motivación que llevó al mismo; las diferentes tecnológicas y artefactos utilizados para su confección, así como su estado, uso y situación en la actualidad; los diferentes objetivos que se han llevado a cabo y finalmente la descripción de la estructura que seguirá esta memoria.

1.1. Motivación

El fin de escoger un tema principal para un Trabajo Fin de Grado (TFG) como es el estudio meteorológico usando procesos de big data radica esencialmente en su importancia para la vida diaria de la era actual, también llamada era de la información.

El mundo actual necesita de un flujo de datos constante e informatizado que brinde información útil para cualquier situación, dando especial atención en la informatización, permitiendo que pueda ser extraída, procesada y consultada en cuestión de segundos. Este aspecto, aplicado a la meteorología es crucial tanto para el conocimiento del entorno, pasado, presente y futuro, ya sea para desarrollar la vida cotidiana, como para saber donde poder desarrollar un proyecto energético usando los elementos del clima o como poder evitar una catástrofe anticipándose mediante el conocimiento de patrones de diferentes variables meteorológicas.

Un estudio de esta temática, usando métodos de big data, ejemplifica claramente como se podría diseñar un sistema real que pudiese ser puesto en producción al uso de cualquier tercero que necesitase de esta información. Este desarrollo resulta muy gratificante a título personal, considerando que muestra varias destrezas que son apreciadas en el ingeniero informático, como son, la creación de una arquitectura compleja y distribuida que sea capaz de extraer, procesar, consultar y generar finalmente resultados a través de datos en crudo; la programación del sistema en base a la arquitectura aplicando diferentes estrategias, como el uso de la programación funcional; o el despliegue de una aplicación haciendo uso de tecnologías de vanguardia aplicadas a servicios en la nube como Amazon Web Services (AWS) ¹ o basadas en contenedores como Docker ².

Todo este desempeño, será explicado a lo largo de toda la memoria, mostrando cada fase y proceso con su correspondiente planificación, ejecución y resultado.

¹ [Amazon Web Services](#) ² [Docker](#)

1.2. Estado del arte

En esta sección se comentarán los diferentes aspectos que han sido partícipes en la realización de este TFG y su situación actual, destacando varias secciones, entre ellas, aspectos centrados en la meteorología, aspectos centrados en el tratamiento y estudio de los datos o aspectos tecnológicos específicos presentes en el marco de la programación y el desarrollo de *software*.

1.2.1. Aspectos meteorológicos

Meteorología

La meteorología como la ciencia que estudia los fenómenos atmosféricos. [81] Mediante ella se comprenden los procesos físicos y químicos que ocurren en la atmósfera terrestre y permite diseñar modelos que predigan patrones climáticos.

Desde la antigüedad se han estudiado los fenómenos atmosféricos, encontrando escritos desde la era clásica como "*Meteorologica*" de Aristóteles alrededor del 340 a.C. A partir de la revolución científica en los siglos XVII y XVIII se introdujeron nuevos instrumentos como el barómetro o el termómetro que permitieron un estudio más riguroso del clima. Tras la llegada de la revolución espacial, se dio el siguiente paso al estudio de la atmósfera con los primeros satélites y globos sonda. Finalmente, con la llegada de la computación moderna, comenzaron a crearse los primeros modelos informatizados del clima, que permitieron la difusión del pronóstico del tiempo en los medios de comunicación. Actualmente, todos los sistemas meteorológicos hacen uso de una red de información global basada en modelos computacionales que permiten predecir el clima, con grandes mejorías en los últimos años con el auge de la inteligencia artificial (IA).

Múltiples empresas privadas u organismos nacionales e internacionales basan su trabajo en el estudio del clima, contando en España, por ejemplo, con la Agencia Estatal de Meteorología (AEMET) ³. [66, 5]

Agencia Estatal de Meteorología (AEMET)

La AEMET es el servicio nacional de meteorología de España. Su origen se marca con la creación del Servicio Meteorológico Español en 1887. En la actualidad, queda regulada bajo el artículo 1.3 del Real Decreto 186/2008, de 8 de febrero, integrando todas las competencias meteorológicas estatales.

Entre sus actividades destacan la observación y registro de datos meteorológicos, predicción del tiempo y emisión de alertas o apoyo técnico y científico a diferentes sectores de la sociedad española. Para el desempeño de dichas actividades la agencia cuenta con una gran dotación de instrumentos, así como gestiona datos satelitales y modelos computacionales de predicción y análisis climático. Respecto a los recursos humanos, la agencia cuenta con más de 1200 efectivos. Finalmente en cuanto al aspecto económico, el presupuesto anual de la agencia para el año 2023 fue de 133.758.140 euros.

Cabe destacar su labor con la creación de un repositorio de datos abiertos, donde, entre otros recursos, puede encontrarse una API de datos meteorológicos extensa (AEMET OpenData) ⁴ con la que se ha podido realizar este proyecto. [5, 3, 4]

³ [AEMET](#) ⁴ [AEMET OpenData](#)

Instituto Andaluz de Investigación y Formación Agraria, Pesquera, Alimentaria y de la Producción Ecológica (IFAPA)

El IFAPA ⁵ es un instrumento del gobierno de Andalucía que pretende impulsar la investigación e innovación de varios sectores, entre ellos el agrario, pesquero, acuícola y alimentario. Actualmente cuenta con 81 proyectos de investigación abiertos en los que trabajan 851 profesionales.

Dentro de este organismo destaca para este proyecto la Red de Información Agroclimática de Andalucía (RIA)⁶, que brinda información meteorológica al sector agrícola andaluz y mantiene un repositorio de datos abiertos llamado IFAPA OpenData ⁷, donde se puede acceder a datos de varias estaciones meteorológicas del territorio andaluz. Para este proyecto ha sido una fuente más de datos que ha podido aportar información meteorológica del desierto de Tabernas (Almería)⁸, el único desierto del continente europeo. [54, 53, 55]

1.2.2. Aspectos sobre el tratamiento y estudio de los datos

Era de la información

Es el periodo histórico contemporáneo, iniciado a finales del siglo XX, en el que la información y su procesamiento organizan la economía, sociedad y cultura marcando una transición entre la era industrial a la era de la digitalización con avances importantes en materia de microelectrónica, redes y comunicaciones. [99]

Ciencia de datos (Data Science)

La ciencia de datos, del inglés *data science*, se define como un campo de estudio interdisciplinar integrado por la estadística, informática y el dominio de estudio que extrae conocimiento a través de un volumen de datos. Su objetivo principal es transformar datos crudos en información interpretable. A este enfoque se suman herramientas de ayuda como el aprendizaje automático, minería de datos, técnicas de visualización...etc.

Históricamente la ciencia de datos tiene comienzo en la evolución de los estudios estadísticos y modelados predictivos previos a la revolución de la computación actual. A partir del surgimiento de las técnicas actuales de computación todos estos procesos pasaron a un plano digital caracterizado por una, cada vez menor, influencia del ser humano en el proceso. Actualmente esta disciplina es fundamental en el marco económico-social. Las investigaciones recientes introducen nuevos aspectos como el aprendizaje automático y la IA. [94, 51]

Datos masivos (Big Data)

El termino de datos masivos, del inglés *big data*, se refiere a conjuntos de datos caracterizados por su gran volumen, velocidad de generación y variedad de formatos. A estas tres características se suman otras dos, veracidad y valor de negocio. Por la naturaleza de los datos se excede la capacidad de herramientas tradicionales para su tratamiento, por lo que se debe dar un tratamiento especial para poder procesar e interpretar mediante herramientas informatizadas.

⁵ IFAPA ⁶ RIA ⁷ IFAPA OpenData ⁸ Desierto de Tabernas

Históricamente, este concepto tiene origen a la par que la ciencia de datos. A pesar de que el análisis de datos masivos no es nuevo, la diferencia actual radica en su gran escala y la preferencia por la automatización y procesamiento en tiempo real. Destacan algunas tecnologías como Apache Hadoop⁹, o sistemas de computación en la nube que permiten un dominio de manera distribuida. En los últimos años, la introducción de técnicas centradas en la IA han mejorado el campo de *big data*. [92, 68]

Datos abiertos (Open Data)

El concepto de datos abiertos, en ingles conocido como *open data*, se ha consolidado como una práctica común entre múltiples entidades que pretenden poner a disposición sus datos a terceros sin restricciones técnicas ni legales, generalmente bajo licencias abiertas.

El origen de esta práctica se vincula al crecimiento de internet y la expansión de iniciativas públicas de datos a principios de los 2000 con portales gubernamentales que hacían públicos sus datos para fines científicos. Después se sumaron otros portales sobre transparencia institucional, catálogos de productos y servicios...etc. Este concepto es fundamental en el marco actual, donde la información de libre y rápido acceso permite un mayor crecimiento económico y social. [96]

1.2.3. Situación actual de las herramientas y tecnologías usadas

Lenguajes de programación

Se han usado varios lenguajes de programación y paradigmas, en su mayoría, el funcional, imperativo y orientado a objetos. En cuanto a lenguajes destaca Scala para el desarrollo del proceso de extracción y procesamiento de datos, Python para la generación de gráficos y Javascript para la creación de la web de visualización de resultados.

Scala¹⁰: es un lenguaje de propósito general, creado por Martin Odersky en 2003, que combina los paradigmas de la programación funcional y la programación orientada a objetos. Se ejecuta en la maquina virtual de Java¹¹ (JVM) lo que le permite compatibilidad con bibliotecas y *frameworks* nativos de Java. Scala destaca por su sintaxis concisa, tipado estático y capacidad de manejo de colecciones y concurrencia de manera funcional. Destaca su uso en el procesamiento de grandes datos, utilizado por plataformas como Apache Spark¹² o Akka¹³ gracias a su compatibilidad con la JVM, su eficiencia en entornos distribuidos y expresividad para crear algoritmos complejos. [64]

Python¹⁴: es un lenguaje interpretado de propósito general, creado por Guido van Rossum en 1991, que combina los paradigmas de la programación imperativa, orientada a objetos y funcional. Su facilidad a la hora de crear código limpio, sencillo y potente ha hecho que sea uno de los referentes en el ámbito académico e industrial. Destaca por una sintaxis clara y un amplio catálogo de librerías. Se usa en múltiples ámbitos del desarrollo de *software*, como la IA, aprendizaje automático, ciencia de datos y programación web. [80]

Javascript¹⁵: es un lenguaje interpretado orientado al desarrollo de aplicaciones distribuidas, creado por Brendan Eich en 1995 para DotCom Netscape, que combina los paradigmas de la programación orientada a objetos y funcional. Destaca por una sintaxis

⁹ Apache Hadoop ¹⁰ Scala ¹¹ Java ¹² Apache Spark ¹³ Akka ¹⁴ Python ¹⁵ Javascript

flexible y su integración con HTML y CSS. Actualmente se estandariza bajo ECMAScript¹⁶. Originalmente fue creado para dar interactividad a las páginas web, pero ahora está presente en múltiples ámbitos del desarrollo distribuido, como la programación de servidores mediante Node.js¹⁷, desarrollo de aplicaciones móviles o plataformas *Internet of Things* (IoT). [65]

Apache Spark como herramienta de *big data*

Apache Spark, diseñado inicialmente en 2009 por la Universidad de California, es un *framework* de procesamiento de datos distribuidos de código abierto que permite manejar grandes volúmenes de datos. El *framework* ejecuta tareas de procesamiento en memoria, lo que reduce significativamente los tiempos frente a sistemas basados en disco. Su arquitectura soporta *streaming* de datos, *machine learning* y consultas SQL sobre datos.

Desde su creación, Spark se ha convertido en un referente en *big data*, ampliamente usado por entornos académicos y la industria. Gracias a su capacidad de integración con Apache Hadoop (un *framework* de código abierto usado para el almacenamiento de grandes volúmenes de datos de forma distribuida), diversas fuentes de datos, programación distribuida, APIs como la de Scala, Python o Java, lo hacen una herramienta muy versátil y accesible para la ciencia de datos. [16, 9, 91]

Herramientas para generación de resultados

Se ha creado un servicio externo en Python con el *framework* Flask para generar gráficos con la librería Plotly.

Flask¹⁸: creado por Armin Ronacher en 2010, es un *microframework* de Python centrado en el desarrollo de aplicaciones web. Destaca por su minimalismo, modularidad y extensibilidad permitiendo agregar nuevos componentes sin una estructura rígida. Surgió como una alternativa a *frameworks* web clásicos de Python como Django¹⁹, buscando una mayor simpleza y libertad. Su diseño está basado en *Web Server Gateway Interface* (WSGI). Permite el manejo de solicitudes HTTP y rutas de forma rápida y clara haciéndolo óptimo para el desarrollo de *APIs REST* sencillas. Actualmente es una herramienta muy usada en la creación de micro-servicios y aplicaciones *RESTful*, destacando librerías como Flask-RESTX²⁰. [100, 70]

Plotly²¹: es una librería interactiva que genera gráficos. Su diseño se centra en la facilidad para crear visualizaciones a partir de datos complejos. Incluye, entre otros, gráficos en dos y tres dimensiones, mapas o *dashboards*. Surgió como un intento de superar las limitaciones de librerías similares como Matplotlib²², integrándose en múltiples *frameworks* como Pandas²³ o Dash²⁴, facilitando la creación de *dashboards* web. Esta característica ha sido explotada por múltiples sectores académicos y profesionales. [108, 78]

Herramientas para visualización de resultados

Se ha diseñado una web para visualizar los gráficos generados con Next.js, un *framework* que hace uso de la librería React.

¹⁶ ECMAScript ¹⁷ Node.js ¹⁸ Flask ¹⁹ Django ²⁰ Flask-RESTX ²¹ Plotly ²² Matplotlib
²³ Pandas ²⁴ Dash

React²⁵: es una librería de Javascript de código abierto, creada por Jordan Walke en Facebook (actual Meta²⁶) que permite desarrollar interfaces de usuario. Uno de sus elementos clave es el diseño de componentes y el uso virtual del *Document Object Model* (DOM), optimizando las actualizaciones y el rendimiento. Estas características permiten a construir páginas donde solo se actualicen los componentes que son necesarios en cada momento. Es una de las librerías para desarrollo de *front-end* más usadas en entornos profesionales, permitiendo crear webs modernas, modulares y escalables. [110, 60]

Next.js²⁷: es un *framework* de código abierto basado en React desarrollado por Vercel²⁸ que permite diseñar páginas web modernas y escalables. Se diferencia frente a React nativo en características como el renderizado por parte del servidor, enrutamiento automático o carga dinámica de componentes, lo que permite mejorar el rendimiento, la indexación por motores de búsqueda y la experiencia de usuario. Además, permite una gran integración con herramientas como TypeScript²⁹, Tailwind CSS³⁰, o librerías de componentes como Shadcn³¹. Actualmente se encuentra entre una de las principales opciones para *front-end*, destacando además, la labor de la empresa desarrolladora, por brindar plataformas de despliegue sencillo y accesible. [105, 88]

Herramientas para despliegue de aplicaciones

Se desplegará el sistema usando las herramientas de Docker y AWS.

Docker: desarrollada en 2013 por Solomon Hykes, es una plataforma de código abierto que permite el desarrollo, empaquetado y despliegue mediante contenedores. A diferencia de las máquinas virtuales, los contenedores comparten el *kernel* del sistema operativo, lo que los hace más ligeros, rápidos y eficientes. Entre sus beneficios destacan la reproducibilidad y escalabilidad. Docker, a su vez, se ve reforzado con la herramienta de Docker Compose³², con la que pueden definirse y orquestarse aplicaciones multi-contenedor. Destacan otras aportaciones como Docker Desktop³³, para un manejo más sencillo gracias a una interfaz de usuario y Docker Hub³⁴, un sitio web donde poder almacenar imágenes en la nube. Esta herramienta se ha convertido en una de las más usadas en los procesos de despliegue, siendo una pieza clave en los flujos de integración continua. [98, 34]

AWS: es una plataforma de computación en la nube que ofrece servicios escalables de desarrollo, despliegue y operaciones. Su infraestructura permite crear servidores virtuales, almacenamiento y redes rápidamente eliminando la necesidad de *hardware* físico. Destacan múltiples servicios como EC2³⁵, S3³⁶, EMR³⁷ o Lambda³⁸. El despliegue en AWS destaca por la escalabilidad y disponibilidad, pudiendo dimensionar los recursos automáticamente en base a la necesidad, lo que garantiza resistencia ante fallos y un rendimiento óptimo. AWS se ha convertido en un estándar en la industria para el despliegue de aplicaciones, permitiendo integraciones con otras tecnologías como Docker. Esto le convierte en una gran opción a la hora de diseñar ciclos de desarrollo de integración continua. [90, 7]

Herramientas auxiliares

Git³⁹: creado por Linus Torvalds en 2005, es un sistema de control de versiones distribuido que permite mantener un historial de cambios de un código fuente, facilitando la

²⁵ React ²⁶ Meta ²⁷ Next.js ²⁸ Vercel ²⁹ TypeScript ³⁰ Tailwind CSS ³¹ Shadcn ³² Docker
Compose ³³ Docker Desktop ³⁴ Docker Hub ³⁵ EC2 ³⁶ S3 ³⁷ EMR ³⁸ Lambda ³⁹ Git

colaboración entre desarrolladores y la gestión de ramas de desarrollo de forma eficiente. Cada usuario mantiene una copia local del repositorio, proporcionando mayor flexibilidad y resistencia a fallos. Se integra con plataformas como GitHub⁴⁰, que permite alojar repositorios en la nube, gestionar equipos, buscar errores, o automatizar flujos de trabajo con GitHub Actions⁴¹. [101, 102, 45, 47]

Postman⁴²: es una herramienta de desarrollo de APIs que permite probar, documentar y automatizar el envío de peticiones HTTP a servicios web. Destaca por su facilidad de uso gracias a su interfaz sencilla e intuitiva. Una de sus mayores fortalezas es la automatización de pruebas y gestión de entornos. Permite crear colecciones y ejecutarlas de forma secuencial, pudiendo diseñar ciclos de pruebas en la construcción de una API. [109, 79]

Herramientas de JetBrains⁴³: integra varios entornos de desarrollo integrados (IDEs) y herramientas creadas por JetBrains. Destacan productos como IntelliJ IDEA⁴⁴, Pycharm⁴⁵ o WebStorm⁴⁶. Ofrecen funcionalidades de auto-completado, refactorización o depuración, mejorando la eficiencia del programador. Adicionalmente, integran, entre otros, sistemas de control de versiones, plataformas de colaboración o integración de aplicaciones externas. Estas herramientas son frecuentemente en entornos profesionales y académicos gracias a numerosos conciertos con instituciones educativas. Actualmente, se han implantado nuevas características a sus IDEs centradas en la IA, como Junie⁴⁷, que ayuda al programador a poder crear o mejorar código rápidamente. [103, 57]

ChatGPT⁴⁸ y **GitHub Copilot**⁴⁹: ChatGPT, lanzado en 2022 por OpenAI⁵⁰, es un asistente conversacional basado en modelos de lenguaje avanzados capaz de generar código, explicaciones, o asistir en tareas mediante lenguaje natural. GitHub Copilot, desarrollado en 2021 por GitHub y OpenAI, es un asistente integrado para IDEs que sugiere código mediante IA generativa. Ambas herramientas permiten acelerar los procesos de desarrollo aportando mayor productividad, así como facilitando el aprendizaje. [93, 46]

Trello⁵¹: lanzada en 2011 por Fog Creek Software (actual Glitch⁵²) y posteriormente adquirida por Atlassian⁵³, es una herramienta de gestión de proyectos basada en la metodología Kanban. Su funcionamiento se basa en la organización de tareas mediante listas y tarjetas. Su diseño intuitivo permite representar los flujos de trabajo de manera clara, facilitando a los integrantes del equipo una mejoría en cuanto a planificación y seguimiento del proyecto. Entre las características más notorias, destaca la colaboración en tiempo real de los usuarios. Además, permite integraciones con múltiples plataformas así como el uso de *plug-ins*. [112, 11]

PlantUML⁵⁴ y **Draw.io**⁵⁵: PlantUML es una herramienta de código abierto que permite la creación automática de diagramas a partir de descripciones textuales. Su principal objetivo es facilitar la creación y mantenimiento de diagramas UML. Permite crear diagramas de clase, secuencia, casos de uso, componentes, actividad...etc, que pueden ser exportados a múltiples formatos. Draw.io (actualmente Diagrams.net), es una herramienta web que permite crear diagramas. Se trata de una solución práctica y gratuita que brinda una mayor personalización frente a PlantUML. [107, 71, 97]

LaTeX⁵⁶ y **Overleaf**⁵⁷: LaTeX es un sistema de composición de textos orientado a publicaciones científicas, técnicas y académicas. A diferencia de los procesadores tradicionales de texto, se basa en un lenguaje de marcado que describe la estructura del documento

⁴⁰ GitHub ⁴¹ GitHub Actions ⁴² Postman ⁴³ JetBrains ⁴⁴ IntelliJ IDEA ⁴⁵ Pycharm ⁴⁶ WebStorm ⁴⁷ Junie ⁴⁸ ChatGPT ⁴⁹ GitHub Copilot ⁵⁰ OpenAI ⁵¹ Trello ⁵² Glitch ⁵³ Atlassian ⁵⁴ PlantUML ⁵⁵ Draw.io ⁵⁶ LaTeX ⁵⁷ Overleaf

que es compilado y exportado, lo que permite un control absoluto sobre la redacción. Se utiliza en varias interfaces de usuario donde destaca Overleaf, un editor colaborativo en la nube. Permite la creación de nuevos documentos, añadir referencias bibliográficas, editar sin usar código o la colaboración entre varios editores. [104, 62, 106, 69]

1.3. Objetivos

En este punto se mencionarán los objetivos del proyecto, centrados el estudio mediante técnicas de *big data* de datos meteorológicos recopilados en el territorio español entre 1973 y 2024 (rango de 50 años). Respecto a los objetivos individuales, destacan los siguientes:

1. Extraer datos meteorológicos usando las APIs públicas de AEMET e IFAPA.
2. Leer los datos en crudo mediante Spark y realizar consultas sobre dichos datos.
3. Generar gráficos mediante la librería Plotly para visualizar los resultados finales.
4. Desplegar el sistema usando Docker y la fase de Spark en un *cluster* EMR con AWS.
5. Crear una web interactiva que permita visualizar los gráficos.
6. Documentar el proceso con una memoria y su correspondiente presentación.

1.4. Estructura de la memoria

El resto de la memoria se divide en cuatro capítulos.

El siguiente capítulo, segundo siguiendo el índice y denominado análisis, pretende arrojar una descripción del problema que plantea el proyecto, así como una definición exhaustiva de todos los requisitos del sistema y las diferentes metodologías de trabajo usadas, el plan de acción a seguir para desarrollar la solución y las herramientas que se emplearán. El tercer capítulo, denominado diseño, mostrará la descripción tanto del diseño *software* de todas las partes que integran el sistema como su posterior proceso de despliegue. El cuarto capítulo, denominado resultados y métricas, donde se expondrán los resultados obtenidos tras ejecutar el proyecto mediante gráficos y diferentes métricas sobre la eficiencia y rendimiento del mismo. El quinto y último capítulo, mencionará las conclusiones a las que se ha llegado tras la realización del proyecto, mostrando los objetivos cumplidos y describiendo los posibles trabajos a futuro que podrían realizarse sobre el sistema final.

Como añadido se encuentra un apéndice, en primer lugar con explicaciones adicionales a nivel de diseño *software* muy interesantes para entender en profundidad el sistema. A continuación otro con diagramas y finalmente la bibliografía utilizada.

Capítulo 2

Análisis

En este capítulo se pretende dar una visión sobre los problemas que plantea el proyecto, así como los requisitos del sistema necesarios para su funcionamiento. Por otro lado, también se pretende hablar sobre la metodología de trabajo que se ha decidido usar, el plan de acción para desarrollar la solución al problema que se describe y las herramientas que formarán parte de la misma.

2.1. Descripción del problema

El sistema a crear se centra en el estudio de datos masivos de ámbito meteorológico. Este pretexto muestra tres fases: extracción, donde se adquieren los datos masivos en crudo de una fuente externa de información; consultas, donde, en base a esos datos se busca información concreta realizando diferentes cruzamientos; y representación, donde se crea un sistema que permita una rápida e intuitiva interpretación de los resultados. Adicionalmente, se integrará una sección para mencionar aspectos comunes a cada fase y finalmente otra para otros aspectos complementarios del proyecto.

2.1.1. Fase de extracción de datos

A la hora de conseguir los datos de una fuente externa se plantan unos requisitos fundamentales: fiabilidad, permitiendo tener la seguridad de que la información es veraz y no ha sido manipulada ni corrompida; consistencia, asegurando que los datos siguen un esquema claro y conciso; accesibilidad, garantizando la entidad que los pone a disposición que su obtención es rápida, sencilla, eficaz y gratuita.

Respecto a los datos meteorológicos que se buscan, se necesitan registros diarios desde 1973 hasta 2024 de varios instrumentos de medición como termómetros, pluviómetros, barómetros, anemómetros, fotómetros o higrómetros, así como los datos de localización de las estaciones meteorológicas.

En consecuencia, se debe buscar el portal que permita acceder y descargar dichos datos, que tras una búsqueda, el mejor de todos para trabajar sobre la meteorología del territorio español es el portal de datos abiertos de la AEMET, que garantiza el cumplimiento de los tres requisitos mencionados gracias a tener el respaldo de ser una institución muy conocida y de gran impacto sobre la meteorología española; mostrar al usuario un esquema previo

a la obtención de los datos que es seguido estrictamente por todo el conjunto; y garantizar la descarga de los datos mediante una API de uso sencillo y gratuito.

El servicio que brinda AEMET, resulta muy útil, contando con gran cantidad de datos extraídos de numerosas estaciones del territorio español, sin embargo, hay casos en los que la agencia no implanta sus sistemas en determinados lugares que, para la realización de ciertos estudios que posteriormente serán mencionados, se ha necesitado recurrir a otro portal de datos abiertos que permita cubrir estas carencias.

Así pues, siguiendo los mismos criterios descritos para la búsqueda de un portal adecuado, se llega a la conclusión que el mejor candidato es el portal de datos abiertos de IFAPA, con características casi idénticas al de AEMET, exceptuando el esquema de datos, que mantiene uno completamente diferente y deberá ser unificado. Es de este portal donde se extraen datos sobre, en concreto, el desierto de Tabernas en Almería, un punto donde AEMET no ha implantado sus sistemas, que ha sido necesario para uno de los estudios que se realizan en este proyecto.

Esta problemática que se ha presentado es un caso recurrente en los estudios de *big data*, donde la fuente principal de datos tiene algunas carencias que se deben cubrir para poder trabajar correctamente, teniendo que recurrir a fuentes secundarias. Posteriormente, dado que cada fuente suele mantener un estándar único para sus datos, estos deben ser unificados en un esquema final.

Particularidades de los datos abiertos de AEMET

Cabe destacar en este apartado que los datos abiertos de AEMET tienen algunas características remarcables [1, 2]:

- Es necesario el registro para obtener una clave de acceso (*API Key*) gratuita que permita el uso de los *endpoints*.
- El número de conexiones está limitado a cuarenta por minuto por usuario, por lo que se deben espaciar las peticiones para no obtener una expulsión.
- Para obtener datos sobre todas las estaciones, se puede hacer una única petición, pero para obtener datos diarios sobre registros meteorológicos en un intervalo de tiempo, se tiene que hacer en intervalos de quince días.
- Los datos son abiertos y gratuitos, pero debe referenciarse a la AEMET como la fuente oficial de los mismos en cualquier proyecto que los utilice.
- A continuación se mostraran unas tablas con el formato de los datos sobre estaciones y seguidamente, el formato de los datos meteorológicos:

Campo	Descripción	Tipo	Requerido
latitud	Latitud de la estación	string	Sí
provincia	Provincia donde reside la estación	string	Sí
indicativo	Indicativo climatológico	string	Sí
altitud	Altitud de la estación	string	Sí
nombre	Ubicación de la estación	string	Sí
indsinop	Indicativo sinóptico	string	Sí
longitud	Longitud de la estación	string	Sí

Tabla 1: Descripción de los campos de información de estaciones de AEMET

Campo	Descripción	Tipo	Unidad	Requerido
fecha	Fecha del día (AAAA-MM-DD)	string	–	Sí
indicativo	Indicativo climatológico	string	–	Sí
nombre	Nombre (ubicación) de la estación	string	–	Sí
provincia	Provincia de la estación	string	–	Sí
altitud	Altitud sobre el nivel del mar	float	m	Sí
tmed	Temperatura media diaria	float	°C	No
prec	Precipitación diaria 07-07	float	mm	No
tmin	Temperatura mínima	float	°C	No
horatmin	Hora de la temperatura mínima	string	UTC	No
tmax	Temperatura máxima	float	°C	No
horatmax	Hora de la temperatura máxima	string	UTC	No
dir	Dirección de racha máxima	float	decenas de grado	No
velmedia	Velocidad media del viento	float	m/s	No
racha	Racha máxima del viento	float	m/s	No
horaracha	Hora de la racha máxima	string	UTC	No
sol	Insolación	float	horas	No
presmax	Presión máxima	float	hPa	No
horapresmax	Hora de la presión máxima	string	UTC	No
presmin	Presión mínima	float	hPa	No
horapresmin	Hora de la presión mínima	string	UTC	No
hrmedia	Humedad relativa media	float	% (porcentaje)	No
hrmax	Humedad relativa máxima	float	% (porcentaje)	No
horaehrmax	Hora de humedad relativa máxima	string	UTC	No
hrmin	Humedad relativa mínima	float	% (porcentaje)	No
horaehrmin	Hora de humedad relativa mínima	string	UTC	No

Tabla 2: Descripción de los campos de datos meteorológicos diarios de AEMET

Particularidades de los datos abiertos de IFAPA

Al igual que los datos abiertos de AEMET, los de IFAPA también cuentan con una serie de características notorias [56]:

- No es necesario el registro para poder hacer uso de los *endpoints*.
- Se permiten obtener datos sobre estaciones y sobre registros meteorológicos en un lapso de tiempo en una única petición.
- Los datos son abiertos y gratuitos, pero debe referenciarse al IFAPA como la fuente oficial en cualquier proyecto que los utilice.
- A continuación se mostraran unas tablas con el formato de los datos sobre provincias, estaciones y finalmente, datos meteorológicos:

Campo	Tipo	Formato
id	integer	int64
nombre	string	--

Tabla 3: Descripción de los campos de información de provincia de IFAPA

Campo	Tipo	Formato
activa	boolean	--
altitud	integer	int64
bajoplastico	boolean	--
codigoestacion	string	--
huso	integer	int64
latitud	string	--
longitud	string	--
nombre	string	--
provincia	object	--
visible	boolean	--
xutm	number	float
yutm	number	float

Tabla 4: Descripción de los campos de información de estaciones de IFAPA

Campo	Tipo	Formato
bateria	number	double
dia	integer	int32
dirviento	number	double
dirvientovelmax	number	double
et0	number	double
fecha	string	date-time
fechautlmod	string	date-time
horminhummax	string	--
horminhummin	string	--
hormintempmax	string	--
hormintempmin	string	--

Campo	Tipo	Formato
horminvelmax	string	--
humedadmax	number	double
humedadmedia	number	double
humedadmin	number	double
precipitacion	number	double
radiacion	number	double
tempmax	number	double
tempmedia	number	double
tempmin	number	double
velviento	number	double
velvientomax	number	double

Tabla 5: Descripción de los campos de datos meteorológicos diarios de IFAPA

Cabe destacar que los archivos descargados tienen un formato JSON, basado en un sistema textual de claves y valores, lo que termina siendo incompatible con un procesamiento rápido cuando los datos crecen. En concreto, los archivos procedentes de la extracción de datos meteorológicos diarios, al hacerse de 15 en 15 días, desde el año 1973 hasta 2024, genera más de 1200 archivos que son leídos uno a uno, provocando latencias en los procesos de lectura y escritura secuencial del sistema informático, sumándole también el peso de los archivos. Para la solución de esta problemática es necesario idear un sistema que comprima los archivos para reducir su cantidad, así como plantear un cambio de formato que mejore el rendimiento.

2.1.2. Fase de estudios sobre los datos

Una vez extraídos los datos y almacenados en su formato correcto, se realizan estudios sobre los mismos, utilizando un sistema que permita idear y ejecutar consultas. Respecto a los estudios a realizar, destacan cuatro grupos: sobre estaciones meteorológicas, sobre la distribución climática del territorio español, sobre variables climáticas individuales, concretamente la temperatura, precipitación, velocidad del viento, presión atmosférica, radiación solar y humedad ambiental; y finalmente, estudios de interés climático.

Estudios sobre estaciones meteorológicas

Se basan en consultas sobre factores de distribución de las estaciones, destacando con mayor relevancia: la altitud, la localización por provincia y la evolución histórica en número.

En cuanto a la distribución por altitud, se quiere contabilizar por intervalos de 500 metros, desde 0 hasta 3092. Respecto a la distribución por provincia, se quiere contabilizar el número de estaciones por cada una de las 52 provincias del territorio español. Finalmente, la evolución histórica mostrará la cantidad de estaciones desde el comienzo de los registros.

Esta selección de consultas proporcionará una visión amplia sobre el estado de la red meteorológica española desde varios ámbitos, pudiendo ver principalmente donde hay mayor frecuencia de encontrar datos meteorológicos.

Estudios sobre la distribución climática

Estos estudios buscan obtener la suma de las precipitaciones y la media de las temperaturas mensuales durante todo el año 2024 en estaciones concretas. El objetivo es la creación de un gráfico climático denominado climograma que muestra las características de un clima en concreto, permitiendo distinguirlo de otro. [95]

En cuanto a las estaciones que serán escogidas, se ha seguido un criterio de elección de una estación por clima. En caso de que ese clima esté presente en el territorio peninsular, en las Islas Baleares o en las Islas Canarias, se escogerá una estación representativa de cada lugar.

Respecto a los climas presentes en el territorio español, se pueden encontrar los siguientes, siguiendo la clasificación de Köppen-Geiger [8, 6]:

Tipo de clima	Clima	Localización
Árido	BSh	Territorio peninsular Islas Baleares Islas Canarias
	BSk	Territorio peninsular Islas Baleares Islas Canarias
	BWh	Territorio peninsular Islas Canarias
	BWk	Territorio peninsular
Templado	Cfa	Territorio peninsular
	Cfb	Territorio peninsular
	Csa	Territorio peninsular Islas Baleares Islas Canarias
	Csb	Territorio peninsular Islas Baleares Islas Canarias
Frío	Dfb	Territorio peninsular
	Dfc	Territorio peninsular
	Dsb	Territorio peninsular

Tabla 6: Clasificación climática de Köppen-Geiger y su localización en España

Cabe destacar que el clima árido BWk es el que está presente en el desierto de Tabernas de Almería. Por el deseo de querer estudiar este clima se ha extraído ese dato de IFAPA.

Estudios sobre variables climáticas individuales

Por cada una de las variables climáticas mencionadas anteriormente en la introducción de la sección, se realizarán estudios de múltiples ámbitos, encontrando así los siguientes:

- **Estudio sobre valores extremos:** se pretende buscar un conjunto de diez estaciones que hayan registrado, por un lado, los valores más altos y por otro, los valores más bajos. Este estudio se enmarca en tres temporalidades, primero, durante todo el año 2024, después, durante toda la última década y finalmente, desde el inicio de los registros.
- **Estudio sobre mayores o menores incrementos:** se quiere encontrar un conjunto de cinco estaciones que hayan registrado, por un lado, los mayores incrementos en los valores climáticos y, por otro, los menores incrementos. Este estudio se enmarca en la temporalidad desde el inicio de los registros.
- **Estudio sobre evolución temporal:** este estudio busca un registro de la variabilidad de los valores durante dos marcos temporales, en concreto, durante todo el año 2024 y desde el comienzo de los registros. Particularmente, el marco temporal del año 2024 tendrá en cuenta todos los días del mismo, frente al del comienzo de los registros, que tendrá en cuenta solo un valor año tras año, donde además se desea generar un modelo de regresión que permita visualizar posteriormente la tendencia que han seguido los valores.
- **Estudio sobre el registro climático completo del territorio español durante 2024:** el objetivo de este estudio es conseguir un valor representativo por cada estación del territorio que permita la elaboración de un mapa de calor que posteriormente pueda ser visualizado. Concretamente, se elaborarán dos mapas, uno del territorio continental de España comprendiendo los territorios en la Península Ibérica, mar Mediterráneo y el continente africano; y otro del territorio de las Islas Canarias. La razón de esta decisión se fundamenta en que la distancia entre ambos territorios haría que un único mapa fuese ilegible.

Estudios meteorológicos interesantes

En cuanto a este ultimo tipo de estudios, se quiere plantear extraer resultados que puedan tener un interés especial.

Respecto al primero de los estudios, se basa en extraer la relación entre la precipitación y la presión atmosférica de un lugar en el marco temporal del año 2024 por un lado, y por otro, desde el inicio de los registros. La razón de la elección de estas dos variables radica en el conocimiento de qué cambios en el clima vienen asociados con aumentos o disminuciones de la presión atmosférica. En concreto, si un lugar presenta una presión atmosférica alta, fenómeno conocido como anticiclón, se asocia a un tiempo estable, seco y despejado. Por el contrario, si un lugar presenta una presión atmosférica baja, fenómeno conocido como borrasca, se asocia con un tiempo convulso, lluvioso y nuboso. [17, 18] La intención, por tanto, de este estudio será intentar verificar con los datos obtenidos esta afirmación.

En cuanto al segundo estudio, se quieren encontrar lugares con condiciones climáticas especiales. Por un lado, que tengan condiciones climáticas idóneas para poder aprovechar los recursos naturales del viento y el sol, para generar energía, y de la tierra para emprender explotaciones agrícolas. Este tipo de estudios son de vital importancia a la hora de emprender la construcción de nuevos parques energéticos o explotaciones del terreno con el fin de maximizar los beneficios. Por otro lado, se busca encontrar lugares que cumplan con unas condiciones climáticas que los hagan propensos a sufrir eventos meteorológicos adversos. Este tipo de estudios también resultan de gran importancia con el fin de poder prepararse antes de que estos eventos ocurran, mitigando su impacto. Concretamente, los eventos meteorológicos que se desean buscar son: lluvias torrenciales, tormentas, olas de calor, heladas, incendios y sequías.

2.1.3. Fase de representación de resultados

Finalmente, una vez obtenidos los datos procedentes de las consultas realizadas en los diferentes estudios, se debe representar la información para que pueda ser entendida de una manera rápida y sencilla. La intención de representar la información no es otra que facilitar al interesado una rápida interpretación mediante un formato amigable. El recurso más usado en estos casos, al tratarse principalmente de datos numéricos, es el uso de gráficas de diferente índole donde representar estos visualmente.

Esta fase se ha ideado para ser integrada por la composición de dos elementos del sistema: un primer elemento capaz de recibir datos sobre qué tipo de gráfico desea el usuario y que información debe contener, así siendo capaz de construirlo y entregarlo; y un elemento que sea capaz de pedir al primer sistema los gráficos pertinentes, actuando como cliente de dicho servicio. La razón de usar este diseño radica en el deseo de aislar ambos sistemas, con el fin de garantizar la encapsulación del proceso de construcción de los gráficos. Si en algún momento se decide cambiar el proceso de construcción usando otros procesos, no hará falta cambiar el cliente y podrá seguir comunicándose de la misma manera, separando así las responsabilidades de ambos componentes.

A continuación, se describen las decisiones a tomar en cuanto a la hora de escoger el tipo de gráfico, siendo necesario estudiar el tipo de dato a representar y de qué manera se desea transmitir la información. Así pues, se analizará para cada estudio el tipo de gráfico a elegir.

Resultados de los estudios sobre estaciones meteorológicas

Se obtienen datos basados en la distribución de las estaciones en cuanto a altitud, localización por provincia y evolución en número desde el comienzo de los registros.

La distribución por altitud necesita de un gráfico que permita representar intervalos numéricos de forma visual, siendo una opción acertada los gráficos basados en sectores. Respecto a la distribución por provincia, necesita relacionar una etiqueta, el nombre de la provincia, con un valor numérico, pudiendo escoger una amplia opción de gráficos, como los basados en barras, o tablas. En cuanto al último resultado, claramente tiene que estar basado en un sistema de dos ejes, tiempo frente a número de estaciones, pudiendo escoger gráficos de barras o lineales.

Resultados de los estudios sobre la distribución climática

El tipo de gráfico a usar con estos resultados ya tiene un formato concreto. Al querer representar la información mediante un climograma, su construcción marca la siguiente normativa:

Se utiliza un gráfico compuesto por la superposición de dos gráficos, basados en barras o líneas. En el eje horizontal se establecen los meses del año. A derecha e izquierda se establecen dos ejes verticales, dando igual el orden, uno con las temperaturas en grados Celsius y el otro con las precipitaciones acumuladas en milímetros. Es importante destacar que el valor numérico que se establezca en el eje de precipitaciones debe ser exactamente el doble al mismo nivel en el eje de temperaturas. [95]

Resultados de los estudios sobre variables climáticas individuales

Por cada una de las variables climáticas individuales mencionadas anteriormente, se obtienen resultados de los siguientes estudios, los cuales se desean representar:

- **Resultados del estudio sobre valores extremos:** en cuanto a este tipo de representación, se tiene una lista de valores extremos ordenados de mayor a menor según el criterio utilizado, por lo que se necesita un gráfico que permita representar esta característica. Usualmente se resuelve usando gráficos de barras con un estilo de *ranking* de posiciones.
- **Resultados del estudio sobre mayores o menores incrementos:** este gráfico cumple las mismas características que el anterior por lo que la estrategia a usar puede ser la misma.
- **Resultados del estudio sobre evolución temporal:** esta representación necesita disponer de un gráfico con dos ejes, tiempo frente al valor estudiado, una característica similar a las descritas en los anteriores puntos con un tipo de representación igual, por lo que, los gráficos lineales suelen ser opciones adecuadas.
- **Resultados del estudio sobre el registro climático completo del territorio español durante 2024:** respecto a este último gráfico, el fin último es representar un mapa de calor con la distribución del valor sobre la superficie. Como se mencionó anteriormente, se desea representar tanto el mapa continental del territorio español, así como el del territorio conformado por las Islas Canarias.

Resultados de los estudios meteorológicos interesantes

En cuanto a este último conjunto de resultados destacan dos elementos.

Por una parte, se necesita representar los resultados de evolución temporal bivariable de la relación entre precipitación y presión atmosférica, por lo que se necesita una solución similar a la descrita en casos anteriores, pero añadiendo ambos gráficos a la vez en la misma presentación para una comparación correcta.

En cuanto a los resultados sobre lugares con condiciones climáticas especiales, al tratarse de una representación basada en etiquetas, siendo estas el nombre de la provincia, ordenadas de mayor a menor según el criterio usado, es decir, la factibilidad de que se desarrolle esa condición estudiada en ese lugar, se requiere un gráfico que permita la representación de un *ranking*, como se ha descrito en casos previos.

2.1.4. Aspectos comunes en todas las fases

Una vez descritas individualmente cada una de las fases, es necesario mencionar que hay aspectos comunes que están presentes en todas ellas. A continuación se procede a su descripción:

Valores constantes

Para la ejecución de cada una de las fases, hay que tener en cuenta que existen valores constantes que definen gran parte del comportamiento del programa.

Es frecuente que en desarrollos similares se tenga en cuenta que estos pueden cambiar, por lo que sería adecuado disponer de herramientas que hiciesen que este cambio fuese lo menos costoso para el desarrollador. Por otro lado, mantener todos los valores constantes en un único lugar asegura un mayor mantenimiento a futuro, así como rapidez a la hora del desarrollo en caso de necesitar modificar alguno de estos. Finalmente, los valores constantes deberán ser cargados en cada una de las fases con el fin de poder ser usados en el momento necesario.

Almacenamiento

Todos los ficheros generados durante la ejecución deberán persistir en el sistema informático, por lo que será necesario idear sistemas que permitan fácilmente la opción de guardar los datos. Cabe destacar que, a la vez que se desea guardar datos, también se desea leerlos.

Respecto al sistema de almacenamiento, se desea que mantenga una organización mediante un sistema de carpetas y subcarpetas dependiendo de la fase en la que se encuentre y el ámbito del que trata la información que contienen, por ejemplo, en la fase de estudios sobre los datos, se podrían encontrar varias carpetas por cada estudio o en la fase de extracción de datos, varias subcarpetas por cada lugar de donde se haya extraído la información.

Por otro lado, se pretende dar soporte a sistemas de computación en la nube, como AWS, para poder desarrollar toda la ejecución en este ámbito, cuestión que también deberá ser tomada en cuenta por el sistema de almacenamiento, permitiendo así realizar operaciones de lectura y escritura usando una interfaz común tanto en un sistema local como remoto.

Funciones auxiliares del sistema

El sistema deberá integrar algunas funcionalidades que sirvan de soporte para el desarrollo de cada fase, pudiendo destacar las siguientes:

- Sistema de realización de peticiones HTTP que permita la comunicación con internet para obtener datos o para enviar información.
- Tratamiento de los JSONs obtenidos de las diferentes APIs de obtención de datos, de modo que se puedan realizar transformaciones a los datos representados según sea conveniente, como por ejemplo, normalizar todo el texto a minúscula o construir JSONs a partir de los esquemas de las APIs.

- Funciones para el tratamiento del tiempo, principalmente para la realización de peticiones a AEMET teniendo en cuenta la restricción de que se pueden obtener datos de registros meteorológicos de quince en quince días, o también para permitir la contabilización del tiempo transcurrido.
- Decoradores sobre los elementos impresos por consola durante la ejecución, permitiendo al desarrollador mayor facilidad en la lectura. Entre las características buscadas se destaca la impresión de texto con color o tabulación a distintos niveles.

2.1.5. Aspectos complementarios

Finalmente, en este punto, se mencionarán varios aspectos complementarios al desarrollo de las fases que se tendrán en cuenta para ultimar el proyecto.

Web para la visualización de resultados

Una vez generados todos los gráficos se desea, con el ánimo de facilitar su visualización, construir un sitio web interactivo que permita su organización y visionado de forma rápida y sencilla. La web integrará tantas secciones como tipos de estudios se hayan hecho. Cabe destacar, que el peso de los archivos gráficos generados será elevado, por lo que proporcionar estos mismos como archivos estáticos de la web no sería viable en tiempos de carga, por tanto, sería buena idea plantear su servicio desde un portal que provea archivos estáticos.

Despliegue de la aplicación

Por cada una de las fases, se pretende realizar un despliegue que permita una ejecución rápida y sencilla. Principalmente, el modelo a buscar se basa en contenedores. Este modelo proporciona sencillez en su construcción y ejecución, además de permitir aislar la aplicación y construir un entorno de ejecución propio, ganando resistencia a fallos de compatibilidad frente a una ejecución en el sistema local. Adicionalmente, un sistema de contenedores permite poner a funcionar la aplicación en cualquier entorno capaz de manejar esta característica informática.

Cabe destacar que, después del despliegue, se realizarán pruebas que permitan conocer los tiempos de ejecución de cada fase.

Respecto a la fase de ejecución de consultas, se pretende ejecutar también en un *cluster Elastic Map Reduce* (EMR) de AWS y estudiar su diferencia frente a una ejecución en local. Normalmente, los desarrollos basados en *big data* se idean para una ejecución en sistemas distribuidos tipo *cluster*.

Documentación

Finalmente, se desea crear una documentación del proyecto, basada en diagramas de ejecución y componentes que permitan dar una visión clara del funcionamiento del sistema y su composición. Adicionalmente, la redacción de esta memoria servirá como otro elemento de documentación que se verá complementado con los diagramas mencionados.

2.2. Requisitos

Como complemento a esta sección, se ha creado una definición de requisitos que presente organizar todas las ideas después de la descripción exhaustiva de todo el proyecto. Puede consultarse en el apéndice A, [Sección A.1](#).

Se diferencia entre requisitos funcionales, aquellos que describen que hace el sistema, y requisitos no funcionales, aquellos que especifican como opera. Podrá visualizarse por cada fase una tabla con un identificador y una descripción del requisito al que hace referencia. Los requisitos funcionales seguirán la nomenclatura RF-N siendo N el número de requisito. Lo mismo con los requisitos no funcionales, solamente cambiando el prefijo por RNF, quedando el esquema RNF-N.

2.3. Metodología

En este último apartado se describirán varios aspectos. Primero, la metodología de trabajo utilizada durante el proyecto que expondrá por qué y cómo se ha usado. A continuación, una explicación de plan de acción a tomar para desarrollar una solución al problema descrito. Finalmente, una descripción de las diferentes herramientas que se utilizarán.

2.3.1. Metodología de trabajo

En cuanto a la estructura que plantea el problema se diferencian varias fases, cada una dependiente de la anterior. Este aspecto es crucial para elegir una metodología, por esta razón, la forma de trabajo más adecuada es la metodología en cascada, caracterizada por dirigir proyectos secuencialmente dividiendo el desarrollo en fases muy marcadas: análisis de requisitos, diseño del sistema, implementación, pruebas y despliegue.

En cuanto a las ventajas y desventajas de usar esta metodología destaca, su facilidad de entendimiento y gestión, y su idealidad cuando los requisitos son claros y no cambiantes. En cuanto a sus debilidades, es poco flexible a cambios, los errores se detectan tarde, el cliente no ve un producto completo hasta el final del proceso y hay riesgos si los requisitos no están bien definidos. [\[82\]](#)

Cabe destacar que, sobre la metodología, se han planteado algunas modificaciones para hacerla más flexible, sobre todo en las fases de desarrollo y pruebas, haciendo crecer el *software* en cada fase de forma incremental. En cada iteración se realizan pruebas para comprobar que todo funcione correctamente. Al finalizar la fase se realizan las pruebas que demuestren el funcionamiento completo. Por otro lado, entre fases, también se ha dado la libertad de poder arreglar errores que afecten a la siguiente de manera inmediata si han sido detectados tarde.

En cuanto a la planificación de tareas y requisitos se ha decidido usar el sistema ágil *Kanban*, que se caracteriza por una gestión visual del trabajo, lo que permite mejorar el flujo del mismo y hacerlo más eficiente mediante limitaciones sobre las tareas en curso.

La característica más utilizada de *Kanban* es el uso de un tablero que permite mover la tarea en proceso entre diferentes estados: “por hacer”, “en progreso” y “hecho”. La columna de “en progreso” debe tener una restricción en cuanto al número de tareas que se realizan en el momento, con el fin de no saturar el proyecto. [\[59\]](#)

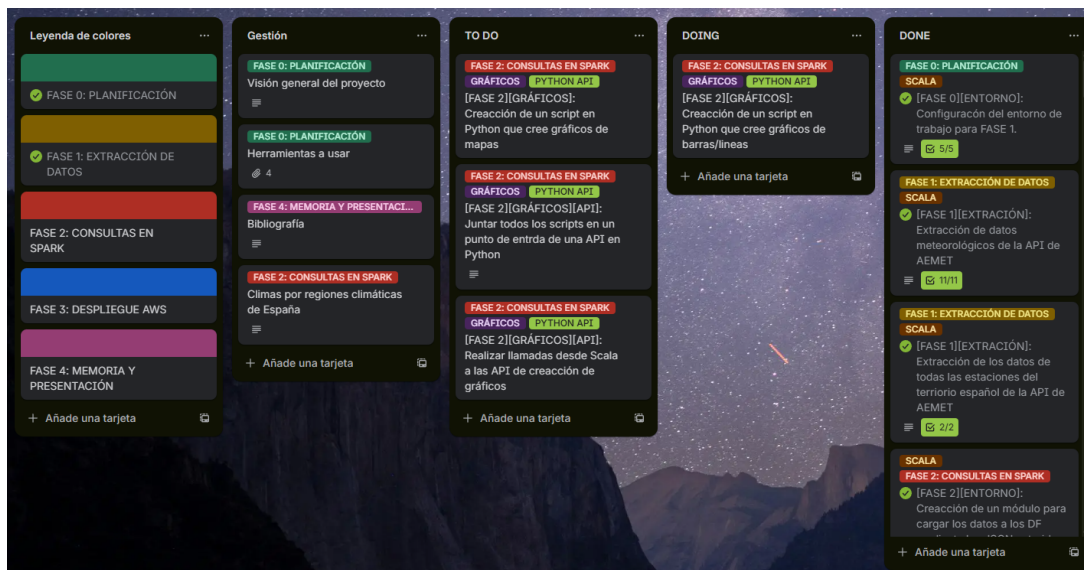


Figura 1: Tablero de Trello con sus columnas de gestión correspondientes y tarjetas en diferentes fases de desarrollo

Ha sido de ayuda la herramienta Trello, donde se ha creado el tablero para especificar las tareas a realizar, de modo que cuando se produzca una actualización, pueda ser desplazada de columna, mostrando una visión clara del proceso de creación del *software*. En la [Figura 1](#), se muestra una ilustración del tablero Trello con las columnas y tarjetas mencionadas.

2.3.2. Plan de acción para desarrollar la solución

Tras describir el problema que plantea el proyecto anteriormente, se debe crear un plan para poder desarrollar una solución al mismo, que consta de cuatro puntos principales:

1. **Planificación y diseño:** se crearán las diferentes tareas a realizar para cada una de las fases del proyecto: extracción, consultas y representación de resultados. También se diseñará la arquitectura general de cada uno de los sistemas.
2. **Desarrollo:** consiste en la parte más importante para conseguir la solución. Cuenta con varios aspectos.
 - a) **Definición de constantes:** se realizará una labor de investigación sobre los valores que deberán tomarse como constantes durante la ejecución, como códigos de estaciones representativas, así como se definirán esquemas de datos o rutas de almacenamiento.
 - b) **Desarrollo de herramientas comunes:** se crearán las diferentes herramientas que serán de utilidad para el desarrollo de las fases posteriores, como un sistema de almacenamiento, sistemas para realizar peticiones HTTP o para el tratamiento de JSONs.
 - c) **Desarrollo de la fase de extracción:** primero se desarrollará el sistema para la API de AEMET, después para la de IFAPA y finalmente se creará el sistema unificador de esquemas y de transformación del formato de archivos a Parquet.

- d) **Desarrollo de la fase de consultas:** se creará primero una base que permita, mediante el uso de Spark, realizar consultas fácilmente. Posteriormente, se realizarán los estudios en base a ese sistema sobre estaciones, distribución climática, variables individuales y estudios de interés climático.
 - e) **Desarrollo de la fase de representación de resultados:** en primer lugar, se diseñará el servidor capaz de proveer los gráficos, y a continuación, su cliente, generando resultados por cada consulta con Spark. Finalmente, se diseñará la web que permitirá visualizar los resultados de forma interactiva.
3. **Despliegue y ejecución:** una vez desarrollada la aplicación, se despliega en local mediante Docker, para cada una de las fases, y en un *cluster* EMR de AWS para la fase de consultas con Spark. También se deberán tomar métricas de eficiencia y rendimiento durante la ejecución.
 4. **Documentación:** para terminar el proyecto se crearán diferentes diagramas de componentes y secuencia, así como la memoria de todo el proyecto.

2.3.3. Herramientas utilizadas

Este punto sirve de complemento a las explicaciones dadas en la sección “[Situación actual de las herramientas y tecnologías usadas](#)”. Se describirá así la razón de uso de cada una de las tecnologías mencionadas, así como una breve descripción de tecnologías secundarias que no hayan sido relevantes en dicha sección.

Lenguajes de programación

En esta sección se mostrarán los diferentes lenguajes utilizados y las razones por las que han sido elegidos para el proyecto.

■ Uso de Scala y la programación funcional:

El uso de la programación funcional ha sido fundamental con el fin de garantizar la creación de un código mucho más idiomático y fácil de comprender frente al uso de otro paradigma. La programación funcional permite en pocas líneas plantear soluciones en el código muy potentes gracias a características como el uso de funciones de orden superior o la desestructuración mediante patrones. En cuanto al código del proyecto, se deseaba que pudiese cumplir, en mayor medida, los principios de la programación funcional, siendo otra de las razones por las que se eligió el paradigma. Entre estos principios destacan [19]:

- **Funciones puras:** para los mismos argumentos, se devuelven los mismos resultados. Sin efectos secundarios.
- **Inmutabilidad:** no se modifican los datos, se crean nuevos a partir de los anteriores.
- **Funciones como ciudadanos de primera clase:** se tratan las funciones como datos comunes pudiendo ser pasadas como argumentos a otras funciones, devolverse como resultado y guardarse.
- **Composición de funciones:** se construye el código combinando el uso de múltiples funciones.

Scala, por su parte, cumple con todos los estándares de la programación funcional, siendo uno sus referentes. Su compatibilidad con la JVM da la posibilidad de usar múltiples librerías y herramientas que han ayudado a la realización de este proyecto, así como su integración con Spark, el pilar fundamental del desarrollo. (Véase: [Scala](#))

■ Uso de Python:

En cuanto al uso de Python, destaca en la creación del sistema proveedor de gráficos para la fase de representación de resultados. La razón principal de su uso es la sencillez a la hora de implementar código y su integración con múltiples librerías. Esta característica ha sido fundamental para desarrollar un servidor que provea estos recursos. (Véase: [Python](#))

■ Uso de Javascript:

Javascript se ha usado en una sección bastante reducida, pero fundamental para crear un sitio web que muestre los gráficos obtenidos de forma interactiva. Este lenguaje se ha usado como base sobre el *framework* Next.js de React. La principal razón del uso de estas herramientas fue poder acceder a librerías de componentes y crear un entorno web con una estética moderna, a la vez que brinde rapidez tanto a la hora de diseñar como de ser ejecutado. (Véase: [Javascript](#), [React](#), [Next.js](#))

Herramientas principales

Se describen las tecnologías utilizadas más importantes para el sistema.

■ Spark como *framework* de *big data*:

Se ha escogido el ecosistema Spark como la mejor opción para el tratamiento de datos masivos. Su compatibilidad con Scala lo hace el entorno perfecto para desarrollar un proyecto de *big data* basado en la programación funcional, posibilitando la construcción de consultas mediante funciones de orden superior, lo que facilita la tarea del programador. Por otro lado, su integración con Apache Hadoop lo hace idóneo para leer o escribir archivos en un entorno local o remoto, como un *bucket* S3 de AWS. Finalmente, AWS EMR cuenta con soporte para Spark, por lo que es ideal si posteriormente se desean hacer pruebas en un *cluster*. (Véase: [Spark](#))

■ Flask para la creación del servidor web:

En los requisitos se pidió que la creación de gráficos se aislase del sistema principal, siendo necesario crear un servicio web para este cometido. Flask, por su parte, es capaz de brindar la suficiente solidez y facilidad para su construcción. Este entorno de trabajo ha permitido crear una arquitectura sin mucha complejidad ni restricciones, basada en modelos de datos, controladores y servicios interconectados. (Véase: [Flask](#))

■ Plotly para la creación de gráficos:

La elección de usar Plotly frente a cualquier otra librería de creación de gráficos ha sido su amplio catálogo de figuras a representar y las posibilidades de personalización de forma intuitiva y directa. Además, su integración con Javascript permite que los gráficos sean interactivos, dando la posibilidad de reescalar o ampliar, incluso ser exportados a formato HTML. Otra de las razones de la elección de la librería fue su integración con Scala y Python, sin embargo, esta versión no cuenta con la madurez de la versión integrada con este último lenguaje, tomando la decisión de usarla con este mismo. (Véase: [Plotly](#))

■ Despliegue mediante Docker y AWS:

En primer lugar, Docker ha sido elegido como el sistema principal para desplegar la aplicación, basado en un sistema de contenedores ligero y fácilmente configurable. Este sistema

permite ejecutar la aplicación en cualquier entorno, cambiando solo algunos parámetros del funcionamiento del contenedor. También permite estudiar las estadísticas de ejecución de forma sencilla. Por otro lado, la elección de AWS para desplegar la fase de estudios sobre los datos ha sido la posibilidad de uso de un *cluster* EMR compatible con Spark y un *bucket* S3, de manera gratuita, gracias a AWS Academy. (Véase: [Docker](#), [AWS](#))

Herramientas auxiliares

En este punto, se exponen las herramientas auxiliares más utilizadas en el proyecto.

■ Herramientas usadas junto con Scala:

- **STTP Client**⁴¹: se trata de una librería que permite crear fácilmente un cliente HTTP. Posibilita la comunicación con APIs usando peticiones GET y POST.
- **Upickle**²: es una librería ligera para trabajar con archivos JSON. Su elección se debe a la posibilidad de realizar acciones de lectura, escritura y modificación rápidamente con un enfoque funcional mediante su interfaz uJSON.
- **Fansi**³: otra librería del mismo autor que la anterior, usada para el tratamiento del color del texto impreso por consola. Se caracteriza por su minimalismo y sencillez, simplificando las operaciones de estilado.
- **PureConfig**⁴: esta librería permite cargar configuraciones basadas en archivos con formato HOCON⁵. Desde Scala, estos archivos pueden interpretarse inmediatamente gracias a la definición de una `case class`.
- **AWS SDK**⁶ (**S3**): es el kit de desarrollo que permite trabajar en Java con AWS. Cuenta con una interfaz de trabajo robusta y gran cantidad de documentación.
- **Apache Avro**⁷: es una librería de serialización de datos en Java. Destaca su uso sobre archivos en formato **Parquet** proporcionando una interfaz que simplifica los trabajos sobre este formato.
- **Apache Hadoop**⁸: Aporta un marco de trabajo para el control de datos distribuidos. Su uso es heredado de Avro y Spark.
- **SBT**⁹: Herramienta para la gestión de dependencias en Scala.

■ Herramientas usadas junto con Python:

- **Flask-RESTX**¹⁰: es una extensión para Flask que añade soporte para la construcción de APIs REST. Se caracteriza por su minimalismo y sencillez. También tiene soporte directo con Swagger¹¹.
- **Pandas**¹²: esta librería se centra en el análisis y manipulación de datos de forma rápida, flexible y sencilla. Su elección se debe a la capacidad de lectura de **Parquet**.
- **Boto3**¹³ (**S3**): se trata de un kit de desarrollo para AWS. Cuenta con las mismas características descritas para AWS SDK.
- **Librerías para la construcción de mapas**: respecto a gráficos tipo mapa de calor, se ha ideado un algoritmo especial que usa varias librerías:
 - **Geopandas**¹⁴: permite trabajar con datos georreferenciados fácilmente.
 - **Scipy**¹⁵: proporciona múltiples algoritmos enfocados en la computación científica y técnica, entre ellos, optimización, interpolación o estadística.

¹ STTP Client ² Upickle ³ Fansi ⁴ PureConfig ⁵ HOCON ⁶ AWS SDK ⁷ Apache Avro
⁸ Apache Hadoop ⁹ SBT ¹⁰ Flask-RESTX ¹¹ Swagger ¹² Pandas ¹³ Boto3 ¹⁴ Geopandas
¹⁵ Scipy

- **Numpy**¹⁶: ofrece la posibilidad de trabajar de forma optimizada con matrices de N dimensiones, además de otras características matemáticas.
- **Pykrige**¹⁷: ofrece algoritmos de kriging, una técnica geoestadística de interpolación espacial.
- **Rasterio**¹⁸: permite el tratamiento de datos geoespaciales y el uso de algoritmos para transformaciones geométricas.
- **OpenCV**¹⁹: proporciona herramientas de trabajo con imágenes y visión artificial.
- **Gunicorn**²⁰: es un servidor HTTP que permite trabajar sobre *frameworks* basados en WSGI como es el caso de Flask, posibilitando el despliegue con varios trabajadores.

■ Herramientas complementarias:

Se pueden destacar las siguientes herramientas auxiliares usadas en el desarrollo:

- **Postman** (véase: [Postman](#)). Permite realizar pruebas sobre *endpoints* de APIs.
- **Docker Desktop** (véase: [Docker](#)). Facilita Docker con una interfaz gráfica.
- **Git y GitHub** (véase: [Git](#)). Mantiene un correcto control de versiones y permite subir los cambios a la nube.
- **Localstack**²¹. Permite simular en local todos los servicios de AWS, pudiendo realizar pruebas sin gastar recursos reales.
- **Herramientas de desarrollo de JetBrains** (véase: [JetBrains](#)). IntelliJ IDEA Ultimate para Scala, PyCharm para Python y WebStorm para la web de resultados.

■ Herramientas de inteligencia artificial:

En la realización del proyecto han tenido relevancia las herramientas de IA, como ChatGPT o GitHub Copilot para los procesos de documentación y aprendizaje, ayudando en dominios complejos como la creación de consultas en Spark, elección de librerías, implementación de algoritmos complejos o arquitectura de *software*. También han resultado útiles para la realización de tareas repetitivas como generación de documentación o refactorización de código. Su uso ha permitido mejorar el rendimiento y velocidad del desarrollo en gran magnitud, así como aumentar la calidad del proyecto. Finalmente es importante mencionar que todo el contenido generado ha sido contrastado con fuentes de documentación oficial y fiable aportadas en la bibliografía, así como estudiado y comprendido.

¹⁶ [Numpy](#) ¹⁷ [Pykrige](#) ¹⁸ [Rasterio](#) ¹⁹ [OpenCV](#) ²⁰ [Gunicorn](#) ²¹ [Localstack](#)

Capítulo 3

Diseño

Durante este capítulo, se describirá la solución al problema expuesto en el capítulo de análisis para cumplir con los objetivos y requisitos del proyecto. Este se compone de tres secciones: herramientas utilizadas, donde se dará una descripción de por qué han sido usadas y con qué propósito; arquitectura e implementación del *software*, donde se describirá detalladamente para cada fase la estructura y funcionamiento del código fuente, así como se acompañará de descripciones sobre el sistema de archivos y diferentes diagramas que permitirán sintetizar la información de manera visual; y finalmente, despliegue del *software*, donde se describirá cómo se ha desplegado la aplicación en local usando un sistema de contenedores, y cómo se ha realizado el despliegue de la fase de estudios sobre los datos en un *cluster* EMR en AWS.

3.1. Arquitectura e implementación del *software*

En esta sección se describe la arquitectura de cada parte del sistema, así como las diferentes secciones de código que lo integran. Como introducción, se dará una descripción general de todo el sistema y, posteriormente, se explicarán de manera individual los aspectos arquitectónicos y de diseño del sistema principal y sus sistemas complementarios.

Como apoyo a las explicaciones, se puede acceder al código fuente en GitHub:

https://github.com/cfkr-dev/aemet_spark_study

3.1.1. Introducción al sistema

Como se ha mencionado anteriormente, se busca extraer datos meteorológicos de una fuente externa, estudiarlos realizando múltiples consultas y representarlos para visualizar los resultados. Destacan tres fases: extracción, estudios y representación.

Al tratarse de un sistema integrado por varias fases, es necesario idear un programa individual para cada una. Esta estrategia permite delegar la responsabilidad a secciones individuales, alejándose de un sistema monolítico que puede dificultar enormemente su desarrollo a medida que crezca el sistema, debido a las interdependencias entre fases. Cabe destacar que el desarrollo se desea bajo un enfoque funcional, por lo que cada uno de estos programas se desarrollará en Scala.

En cuanto al sistema de representación de resultados, se integra por dos elementos: el primero, capaz de proveer gráficos, y el segundo, capaz de solicitarlos. La parte solicitante, o cliente, se desarrollará en Scala, dado que no presenta una gran complejidad, frente al sistema generador, o servidor, que debe individualizarse dado que hace uso de la librería Plotly. Este cuenta con una versión mucho más madura en su versión Python frente a Scala, por lo que no puede integrarse dentro del ecosistema que usa este lenguaje. Finalmente, todos los gráficos podrán visualizarse en un sitio web creado con Next.js.

3.1.2. Sistema principal

El objetivo de la siguiente explicación es exponer la arquitectura y el funcionamiento del sistema principal de manera específica para cada uno de los elementos que lo integran. Cabe destacar que en el apéndice A se puede consultar la jerarquía de directorios y ficheros en la [Sección A.2](#) y una descripción global del sistema en base a un diagrama de componentes detallado en la [Sección A.3](#). La lectura de ambos contenidos es recomendada para comprender mejor el alcance de cada sistema. Puede visualizarse igualmente un diagrama de componentes simplificado en la [Figura 2](#)

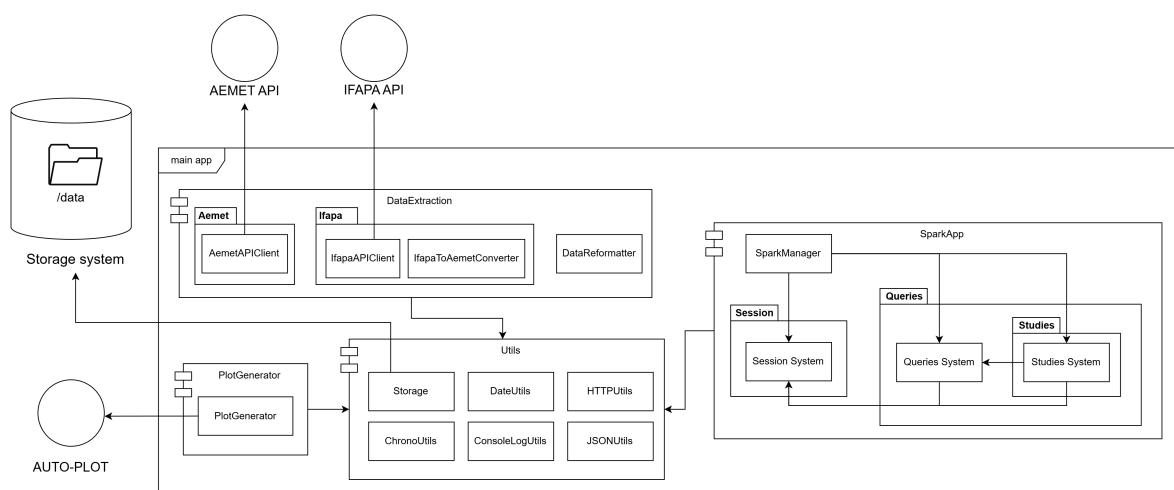


Figura 2: Diagrama de componentes simplificado del sistema principal

Descripción específica de la fase de extracción de datos

En este punto, se describirá desde la visión del *software*, cada uno de los elementos que integra el sistema de extracción de datos. Este sistema funciona de tal manera que es capaz de obtener los datos de las APIs externas de AEMET e IFAPA y almacenar la información. Posteriormente, los datos de IFAPA se convierten al formato definido por el esquema de AEMET y finalmente todos los datos son convertidos a un formato adecuado para la lectura del sistema que usa Spark.

■ Extracción de datos de AEMET:

En cuanto a la explicación del proceso de extracción de datos de la API de AEMET, se centra en el código del fichero `AemetAPIClient.scala`. Este fichero actúa como cliente capaz de realizar peticiones a la API de AEMET extrayendo la información referente a

datos y metadatos de las estaciones meteorológicas así como los mismos de los registros meteorológicos diarios desde 1973 hasta 2024.

Su ejecución comienza con la función `aemetDataExtraction()` cuya secuencia de acciones es la siguiente:

1. Extracción de los metadatos sobre estaciones.
2. Extracción de los metadatos sobre registros meteorológicos.
3. Extracción de los datos sobre estaciones.
4. Extracción de los datos sobre registros meteorológicos.

En cuanto a la organización del código, se ha separado la extracción de datos de estaciones y de registros meteorológicos en dos `object` u objetos de Scala, permitiendo definir ámbitos de programación separados. En cuanto a las estaciones, se encuentra el objeto `AllStationsData` y respecto a registros meteorológicos el objeto `AllStationsMeteorologicalDataBetweenDates`.

El proceso de extracción de datos y metadatos, ya sean sobre estaciones o registros meteorológicos, sigue un patrón constante a lo largo de todo el código. Primero se realiza una petición a la API de AEMET usando una dirección específica para cada recurso, obteniendo una respuesta, que si es correcta, contiene un JSON con dos direcciones nuevas, una para datos y otra para metadatos. Dependiendo de qué se desee extraer se realiza una nueva petición a la dirección correspondiente. Puede ocurrir el caso en el que al hacer alguna de las peticiones falle la API o el cliente, por lo que se integra todo este proceso en un bucle que ejecuta una y otra vez las peticiones hasta ser correctas. El funcionamiento de este sistema puede verse simplificado en la [Figura 39](#).

Detalladamente, cada objeto contiene una función `getAndSave()` capaz de formatear la dirección con los parámetros necesarios, enviar la petición a la API y discernir si ha sido correcto el proceso, almacenando los datos obtenidos, o en caso de fallo, comunicar al sistema que debe repetir esta petición mediante el paso de un booleano. Las peticiones las realiza mediante una función global llamada `getAemetAPIResource()` (véase: [Figura 3](#)). Esta función llama a `sendGetRequest()` del módulo de utilería `HTTPUtils.scala` que se encarga de realizar una petición `GET` a una dirección. Se realizan dos llamadas a esta función en base al funcionamiento ya explicado de la API, y si todo es correcto, la lee la información obtenida en formato JSON usando el módulo `uJSON` de la librería `uPickle`. En caso de fallo devuelve una excepción.

```

1 private def getAemetAPIResource(uri: Uri, isMetadata: Boolean = false): Either[
2     Exception, ujson.Value] = {
3     sendGetRequest(uri) match {
4         case Right(response) =>
5             val dataParsedToJSON = ujson.read(response.body)
6             dataParsedToJSON(ctsStateNumber).num.toInt match {
7                 case 200 =>
8                     val requestURIResource: String = if (isMetadata) ctsMetadata else ctsData
9                     sendGetRequest(uri"${dataParsedToJSON(requestURIResource).str}") match {
10                         case Right(response) => Right(ujson.read(response.body))
11                         case Left(exception) => Left(exception)
12                     }
13                 case _ => Left(new Exception(...))
14             }
15     case Left(exception) => Left(exception)
16 }

```

Figura 3: Código de la función `getAemetAPIResource()`

En cuanto a las direcciones a las que se realizan las peticiones de la API para conseguir, respectivamente, las estaciones y los registros meteorológicos, son las siguientes:

```
https://opendata.aemet.es/opendata/api/valores/climatologicos/inventarioestaciones/
todasestaciones/?api_key=[API_KEY]

https:
//opendata.aemet.es/opendata/api/valores/climatologicos/diarios/datos/fechaini/[FECHA_
INICIO_INTERVALO]/fechafin/[FECHA_FIN_INTERVALO]/todasestaciones/?api_key=[API_KEY]
```

Cada objeto tiene un método individual para realizar las peticiones, pero tienen similitudes en las llamadas a su función `doWhileWithGetAndSave()`. Esta llama a `getAndSave()` indicando el *endpoint* de la API al que hacer la petición, los parámetros necesarios para la misma, la distinción entre metadatos o datos y la ruta de almacenaje. Finalizada la ejecución se comprueba si ha sido correcta según el valor devuelto y en caso de fallo, repite la ejecución con un intervalo de inactividad.

En referencia al diseño en cada objeto para hacer las peticiones se distingue una estructura similar, comenzando con una función inicial que se usa de contenedor para otra que llama a `doWhileWithGetAndSave()` mediante una función auxiliar o *helper* en `ChronoUtils.scala` llamado `executeAndAwaitIfTimeNotExceedMinimum()` que permite ejecutar la función y dormir la ejecución un tiempo si esta se ha ejecutado más rápido de lo debido, permitiendo saltar las restricciones de límite de tiempo entre intervalos de llamadas a la API. Cabe destacar que en el caso de las peticiones de registros meteorológicos, estas funciones de alto nivel son capaces de realizar avances entre fechas, ya que los *endpoints* de la API para estos datos requieren los parámetros de fecha de inicio y fin de cada registro, teniendo en cuenta además que dicho intervalo ha de ser de una longitud máxima de quince días. Esto se consigue con las funciones de manejo de fechas de `DateUtils.scala`. Cabe destacar que por cada petición se escribe en un JSON personalizado la fecha de fin de intervalo si la petición fue correcta, para que en caso de cortarse la ejecución a la mitad, si se reanuda, se toma como fecha de inicio la fecha del archivo.

Para finalizar estas explicaciones, se muestra un fragmento de los JSON extraídos a modo de ejemplo. Puede apreciarse en la [Figura 4](#), a la izquierda, una porción de los datos sobre estaciones, y a la derecha, otra de los datos sobre registros meteorológicos:

<pre> 1 { 2 "latitud": "394924N", 3 "provincia": "ILLES BALEARS", 4 "altitud": "490", 5 "indicativo": "B013X", 6 "nombre": "ESCORCA, LLUC", 7 "indsinop": "08304", 8 "longitud": "025309E" 9 }</pre>	<pre> 1 { 2 "fecha": "1973-03-02", 3 "indicativo": "C249I", 4 "nombre": "FUERTE... AEROP...", 5 "provincia": "LAS PALMAS", 6 "altitud": "25", 7 "tmed": "18,0", 8 "prec": "0,0", 9 ... 10 }</pre>
--	---

Figura 4: Comparación de los JSON extraídos de AEMET sobre estaciones y registros meteorológicos

■ Extracción de datos de IFAPA:

El código de extracción de datos desde la API de IFAPA es muy similar al de AEMET. Al contener esta API menos restricciones que la anterior, se pueden evitar algunas de las

soluciones que se han descrito anteriormente. Todo el código se encuentra en el fichero `IfapaAPIClient.scala`.

Cabe destacar que solo se van a realizar las peticiones sobre la estación referente al desierto de Tabernas en Almería, necesaria para un estudio posterior, por lo que se han tenido que hacer peticiones previamente al desarrollo para conseguir el identificador de dicha estación y la provincia en la que se encuentra (en concreto, ambos tienen como identificador 4). Además, esta vez solo se realizará la extracción para el año 2024.

En líneas generales, las peticiones a la API funcionan de la misma forma que las de AEMET salvo que los metadatos han de ser obtenidos desde una dirección en concreto donde se extrae toda la información de los esquemas de la API proporcionados por su documentación en Swagger, por lo que hay que interpretar el JSON obtenido y quedarse solo con las partes que importan. Por otro lado, a la hora de realizar una petición para obtener datos, estos son servidos directamente, sin tener que pasar por una petición intermedia. Finalmente, al no haber restricciones temporales en cuanto al intervalo de fechas se puede hacer todo con una sola petición. El funcionamiento del sistema puede apreciarse resumido en el diagrama de la [Figura 40](#).

En cuanto a las particularidades del código, se encuentra la función `getAndSave()`, análoga al sistema anterior pero esta vez individual a cada función de alto nivel que extrae información. Esta individualización se debe a la diferencia entre la extracción de datos sobre metadatos descrita anteriormente. Esta función llama a `getIfapaAPIResource()` con un funcionamiento similar al del sistema anterior pero eliminando el paso intermedio de la primera petición que había que hacer en la API de AEMET.

Las direcciones a las que se realizan las peticiones para conseguir, respectivamente, los metadatos, datos de la estación y datos de los registros meteorológicos, son las siguientes:

```
https://www.juntadeandalucia.es/agriculturaypesca/ifapa/riaws/v2/api-docs
https://www.juntadeandalucia.es/agriculturaypesca/ifapa/riaws/estaciones/[ID_
PROVINCIA]/[ID_ESTACION]
https://www.juntadeandalucia.es/agriculturaypesca/ifapa/riaws/estaciones/[ID_
PROVINCIA]/[ID_ESTACION]/[FECHA_INICIO_INTERVALO]/[FECHA_FIN_INTERVALO]/[GET_ETO]
```

En cuanto al resto del código, continúa existiendo la función `doWhileWithGetAndSave()` por cada función de extracción de alto nivel que repite la ejecución en caso de fallo del mismo modo que se realiza en el sistema de AEMET. El resto de aspectos son de iguales características a este último sistema.

Para finalizar estas explicaciones, se muestra un fragmento de los JSON extraídos a modo de ejemplo. Puede apreciarse en la [Figura 5](#), a la izquierda, una porción de los datos de la estación, y a la derecha, otra de los datos sobre registros meteorológicos:

■ Conversión del esquema de datos de IFAPA al de AEMET:

Resulta necesario realizar una conversión del esquema de los datos aportados por IFAPA al de AEMET. Puede observarse la diferencia entre ambos esquemas en [Tabla 1](#), [Tabla 2](#), [Tabla 4](#), [Tabla 3](#) y [Tabla 5](#). Así pues, todo el código referente a esta labor puede encontrarse en el fichero `IfapaToAemetConverter.scala`.

La ejecución del sistema comienza con la función `ifapaToAemetConversion()`. Así pues, primero convierte el esquema de los datos sobre estaciones y después el de los registros meteorológicos. Puede verse un diagrama de ejecución resumido en la [Figura 41](#).

<pre> 1 { 2 "provincia": { 3 "id": 4, 4 "nombre": "Almería" 5 }, 6 "codigoestacion": "4", 7 "nombre": "Tabernas", 8 "bajoplastico": false, 9 "activa": true, 10 "visible": true, 11 "longitud": "021808000W", 12 ... 13 }</pre>	<pre> 1 { 2 "fecha": "2024-01-01", 3 "dia": 1, 4 "tempmedia": 9.53, 5 "tempmax": 17.93, 6 "hormintempmax": "14:06", 7 "tempmin": 2.963, 8 "hormintempmin": "23:50", 9 "humedadmedia": 70.3, 10 "humedadmax": 96.9, 11 "horminhummax": "23:58", 12 ... 13 }</pre>
--	---

Figura 5: Comparación de los JSON extraídos de IFAPA sobre estación y registros meteorológicos

Dando una explicación más específica, el código contiene dos objetos como en los sistemas anteriores: `SingleStationInfo` para la conversión de los datos sobre estaciones y `SingleStationMeteoInfo` para la conversión de los datos sobre registros meteorológicos. Cada uno contiene una función principal que se encarga de pasar del esquema de IFAPA al de AEMET y almacenar los nuevos datos.

La ejecución del proceso de conversión se basa en la lectura de los datos del JSON a convertir de IFAPA, así como también se realiza una lectura de los JSON de metadatos de AEMET con el esquema objetivo. Posteriormente, se genera un formateador de datos, valiéndose de una función que crea un mapa con claves, las nuevas claves para el JSON con el esquema de AEMET, y como valores, una función que dado un `uJSON.value` realiza una transformación, como puede ser convertir a texto, reemplazar contenido o cambiar el formato numérico.

```

1 val aemetAllStationInfoMetadata: Value = readJSON(srcPath) match {...}
2 val formatter: Map[String, ujson.Value => ujson.Value] =
3   genAemetStationInfoFormatters()
4 writeJSON(destPath, readJSON(jsonPath) match {
5   case Left(exception: Exception) => ...
6   case Right(json: Value) => transformJSONValues(buildJSONFromSchemaAndData(
7     genEmptyAemetJSONFromMetadata(aemetAllStationInfoMetadata),
8     genAemetAllStationInfoJSONKeysToIfapaJSONValues(json)
9   ), formatter)) match {
10   case Left(exception: Exception) => ...
11   case Right(_) => copyJSON(srcMetadataPath, destMetadataPath) match {...}
12 }
```

Figura 6: Código para la escritura del JSON de IFAPA con formato AEMET

A continuación, valiéndose de la función `buildJSONFromSchemaAndData()` del módulo `JSONUtils`, se crea un nuevo objeto JSON con el esquema de AEMET y los valores de su correspondiente JSON de IFAPA. Este proceso pasa por dos pasos intermedios: primero generando un objeto vacío solo con las claves de AEMET valiéndose de la función `genEmptyAemetJSONFromMetadata()` que recibe el esquema de los metadatos. En segundo lugar, se usa una función interna individual para cada proceso, que define un mapa con claves, las nuevas con en el esquema de AEMET, y como valores, los propios del valores del JSON de IFAPA, estableciendo en esta estructura de datos la traducción. El proceso de construcción, así pues, crea el nuevo objeto JSON iterando por todas las claves y obteniendo instantáneamente los nuevos valores correspondientes.

Finalmente, con el formateador que fue definido al inicio, se aplican las transformaciones pertinentes a cada clave del JSON, mediante la función `transformJSONValues()` de `JSONUtils`, usando una técnica similar a la que acaba de ser descrita con el proceso de construcción, transformando los valores al formato deseado.

Si todo el proceso es correcto, se escribe el JSON nuevo con los datos de IFAPA ahora con el esquema de AEMET. Además los metadatos de AEMET correspondientes, son copiados en el mismo directorio (véase: [Figura 6](#)).

■ Transformación del formato de los datos:

Todos los datos obtenidos tras los procesos de extracción están en formato JSON. Este mantiene la información en formato textual, de modo que no es adecuado para representar datos masivamente por su peso en memoria, así como, en caso de lectura esta se hace de manera secuencial y completa, lo cual provoca grandes periodos de latencia.

Los datos almacenados de AEMET e IFAPA (con el esquema convertido) tienen un peso en disco de 3,03 GB. A este aspecto se suma que como los datos de registros meteorológicos de AEMET deben ser extraídos en intervalos de quince en quince días, se cuenta, sumando todos los archivos, con un total de 1276 archivos JSON. La cantidad de llamadas al sistema para poder leer estos archivos resulta incompatible con los recursos que se poseen para realizar el proyecto, además de ser totalmente impráctico.

Bajo esta problemática se abordan dos soluciones: en primer lugar, se busca convertir todos los archivos a formato **Parquet**, permitiendo aprovechar su formato binario que reducirá el peso de los archivos, así como la posibilidad que aporta su formato columnar, que favorece lecturas de columnas de datos individuales. En cuanto a la segunda decisión, dado que los archivos JSON de registros meteorológicos son muy numerosos, se crearán cúmulos, agrupando la información bajo archivos JSON de aproximadamente 350 MB, siendo un peso razonable teniendo en cuenta el balance entre carga del sistema y cantidad de información por fichero. Posteriormente, estos ficheros también se convertirán a **Parquet**.

En cuanto a la estructura del código, cuenta también con dos objetos organizadores, `AemetDataReformatter` y `IfapaDataReformatter`, cada uno destinado a la conversión de los datos de AEMET e IFAPA respectivamente.

El punto de entrada a la ejecución es una función `reformatData()` de la que se extrae el siguiente funcionamiento. También puede observarse un diagrama de ejecución con el proceso simplificado en la [Figura 42](#).

1. Transformación del formato de los datos sobre estaciones de AEMET.
2. Transformación del formato de los datos sobre registros meteorológicos de AEMET.
3. Transformación del formato de los datos sobre estaciones de IFAPA.
4. Transformación del formato de los datos sobre registros meteorológicos de IFAPA.

En detalle, el sistema comienza con la lectura en disco de todos los archivos del proceso de extracción, incluyendo datos y metadatos. Los datos serán convertidos de formato y los metadatos se usarán para crear el esquema de los archivos **Parquet**. El proceso de conversión, así pues, comienza con la función `withAvroParquetWriter()`, del módulo de utilidad para el almacenamiento `AvroParquetStorageBackend`, aceptando como parámetros el nuevo directorio de escritura, el esquema de datos y una función de aplicación que contendrá el código de transformación.

En cuanto a la creación del esquema, se utiliza la función `createParquetSchemaFromMetadata()` que acepta como parámetro el objeto `JSON` con el esquema de datos a usar contenido en los metadatos. El objetivo será crear una lista con los nuevos elementos del esquema, uno por cada campo, realizando una transformación con la función `map()` sobre la estructura de datos del objeto `JSON`. Así, se crea cada elemento usando la función `Schema.Field()` que representa la definición del esquema de una de las columnas. Cada columna necesita un tipo de dato que es agregado mediante la función `Schema.create()` donde se inserta el tipo, siempre será una cadena de texto o `String`, aunque puede ocurrir que el campo pueda estar vacío o `Null`. Para representar este formato, a esta última función se añade como parámetro `Schema.Type.STRING` o `Schema.Type.NULL`. En caso de necesitar ambos (campo de texto que puede estar vacío), esta última función se coloca como parámetro a `Schema.createUnion()`. Una vez finalizan las iteraciones y se tienen todos los elementos por columna se integran creando el esquema mediante la función `Schema.createRecord()` (véase: [Figura 7](#))

```
1 private def createParquetSchemaFromMetadata(metadataJSON: ujson.Value): Schema = {
2     val fields = metadataJSON(ctsSchemaDef).arr.map { field =>
3         val fieldSchema =
4             if (field(ctsFieldRequired).bool) Schema.create(Schema.Type.STRING) else
5                 Schema.createUnion(Schema.create(Schema.Type.NULL), Schema.create(Schema.
6                     Type.STRING))
7         new Schema.Field(field(ctsFieldId).str, fieldSchema, null, null)
8     }
9     Schema.createRecord("schema", null, null, false).tap(_.setFields(fields.toList.
10         asJava))
11 }
```

Figura 7: Código de la función `createParquetSchemaFromMetadata()`

En cuanto a la función de aplicación que necesita `withAvroParquetWriter()` se basa en la iteración por cada clave del objeto `JSON`, accediendo a cada valor así insertando filas dentro del `Parquet`. Primero se crea una fila con `GenericData.Record()` que pide como parámetro el esquema creando anteriormente, y después, se aplica a esta fila la función `put()` que acepta como parámetros el nombre de la columna donde insertar y el valor en cuestión. [42, 15]

Finalmente, en el caso de los registros meteorológicos de AEMET, para reducir el número archivos, antes de cambiar el formato a `Parquet`, se acumula la información en archivos `JSON` más grandes mediante un proceso realizado en la función `buildChunks()` que acepta una lista de archivos a la que se le va a aplicar una función de transformación `foldLeft()`. El algoritmo comienza con una lista vacía con un acumulador de peso y por cada iteración se lee un fichero y se valora su peso. Si no existen bloques, se crea uno añadiendo el archivo y sumando su peso. Si es menor al valor del acumulador, puede incluirse al bloque actual. Si supera el tamaño límite del acumulador, se crea un nuevo bloque y se añade el archivo.

Una vez aplicadas las transformaciones a los archivos, se consiguen grandes mejoras, teniendo ahora un total de 12 archivos (reducción del 99,06 %) con un peso total de 122 MB (reducción del 96,07 %). Gracias a este proceso, la información ahora está en un formato más compatible para ser trabajada en la siguiente fase utilizando `Spark`, reduciendo el número de llamadas al sistema así como se optimiza la lectura gracias a las características de `Parquet`.

Descripción específica de la fase de estudios sobre los datos

En este punto se explicará desde el punto de vista del *software*, el sistema que realiza estudios sobre los datos ejecutando consultas con el uso de Spark.

Los estudios a efectuar son: sobre la distribución de las estaciones meteorológicas, sobre distribución climática del territorio español, sobre variables climáticas individuales y sobre aspectos climáticos de interés.

Por lo que respecta a la arquitectura del *software*, el sistema se compone de varios paquetes, cada uno administrado por un *manager* u orquestador que accede a funcionalidades dentro de su *core* o núcleo. Destacan dos subsistemas: gestión de la sesión y ejecución de consultas. El primero administra la carga de los datos al sistema o escritura de resultados, así como permite el acceso a las funciones internas de Spark. El segundo diseña las consultas y las ejecuta.

Se ha elegido esta arquitectura para compartimentar el sistema, creando módulos reducidos y especializados frente a un sistema monolítico. Entre sus ventajas destaca una potente escalabilidad, importante para este tipo de sistema compuesto por multitud de estudios. Con esta característica, añadir uno nuevo no provoca la necesidad de modificar código crítico. Esta arquitectura también aporta una mayor organización del código, lo que permite desarrollar mucho más rápido, así como favorecer un mantenimiento mucho más específico.

El control el sistema se gestiona en el fichero `SparkManager.scala` con su función `execute()` con el siguiente flujo de ejecución:

1. Creación de la sesión de Spark y arranque del sistema leyendo los datos extraídos.
2. Creación del núcleo para la ejecución de consultas.
3. Ejecución de consultas sobre estaciones meteorológicas.
4. Ejecución de consultas sobre distribución climática.
5. Ejecución de consultas sobre estudios meteorológicos de variables individuales.
6. Ejecución de consultas sobre estudios meteorológicos de interés.
7. Finalización de la sesión de Spark.

A continuación, se explicarán más en detalle cada uno de los puntos que integran la ejecución del sistema.

■ Subsistema de control de la sesión:

Se compone de los ficheros `SparkSessionCore.scala` y `SparkSessionManager.scala` y su objetivo es iniciar la sesión de Spark y permitir la carga de datos.

`SparkSessionCore.scala` define la clase `SparkSessionCore`, que es una envoltura al objeto `Spark` para añadirle nuevas funcionalidades, entre ellas, dar acceso rápido al sistema de ficheros y a la representación interna de los datos con un *dataframe* (DF). Contiene las funciones principales `startSparkSession()` y `endSparkSession()`, que arrancan o apagan la sesión de Spark. También están las funciones de almacenamiento de resultados `saveDataframeAsParquet()` para guardar en formato Parquet y `saveDataframeAsJSON()` para formato JSON.

`SparkSessionManager.scala` contiene el objeto `SparkSessionManager` que crea la sesión Spark y lee los datos. Su función principal es `createSparkSessionCore()`. Puede observarse en el diagrama de ejecución de la [Figura 43](#) un resumen de su cometido.

Su ejecución comienza con `createSparkSession()` que crea la sesión de Spark valiéndose de la función `SparkSession.builder()`. Mediante la aplicación de otras funciones a esta última se establecen las características de la sesión, destacando `master()` que define la dirección a la que el nodo maestro de Spark debe conectarse. A esta función han de pasarse unos parámetros u otros dependiendo del entorno en el que se desee realizar la ejecución, ya sea local o en *cluster*, gestionando esta disyunción con una variable de entorno. En cuanto a sus parámetros, en local se usa `"local[NÚMERO_NÚCLEOS | *]"`, que indica el número de núcleos del procesador a usar (si se usan todos se usa `*`). Para el caso del *cluster*, no se usa esta función. También destacan otras funciones secundarias de configuración como `appName()` para el nombre a la sesión o `config()` para cambiar configuraciones internas usando un par clave y valor. Finalmente, `getOrCreate()` crea la sesión. [44]

La ejecución prosigue con el acceso al sistema de archivos y creando los DFs, en concreto cuatro, dos para los datos de estaciones y registros meteorológicos de AEMET y otros dos para los de IFAPA. Estos son creados con `createDataframeFromParquet()` que ejecuta la función de Spark `read.parquet()` que lee los archivos *Parquet* de la fase anterior. [40] Tras ejecutar se obtiene un objeto *Dataframe* que mantiene una representación interna de los datos, sin embargo, para poder trabajar sobre los mismos deben aplicarse algunas transformaciones que serán de utilidad en las consultas en base a los siguientes objetivos:

- Adecuar el tipado de los datos, ya que todos están en formato *String*, por otros más representativos como *DoubleType* o *IntegerType*.
- Mantener la consistencia de valores, evitando dos posibles valores para una entidad (por ejemplo provincias escritas con abreviatura o no simultáneamente).
- Eliminar *Strings* en campos numéricos por *null* y redondear a dos decimales.
- Obtener los datos de latitud y longitud en formato decimal en base a los aportados en formato sexagesimal.

Las transformaciones se aplican en dos pasos, primero a la unión de los DFs de estaciones (usando la función de Spark `union()`) y segundo a la unión de los DFs de registros meteorológicos. Este proceso se logra con el uso de `foldLeft()` sobre un mapa de transformaciones con claves, el nombre de la columna del DF, y valores, una función de modificación del valor de la columna aplicada mediante `withColumn()` que permite a Spark realizar modificaciones sobre columnas individuales de un DF (véase: [Figura 8](#)). [67]

```

1 private def formatAllStationsDataframe(dataframe: DataFrame): DataFrame = {
2   val formatters: Map[String, String => Column] = Map(
3     ctsProvincia -> (column => udf((value: String) => Map(
4       ctsStaCruzDeTenerife -> ctsSantaCruzDeTenerife, ...
5     ).getOrElse(value, value)).apply(col(column))),
6     ctsAltitud -> (column => col(column).cast(IntegerType)),
7     ctsLatDec -> (_ => round(udf((dms: String) => {
8       val degs = dms.substring(0, 2).toInt val mins = dms.substring(2, 4).toInt
9       val secs = dms.substring(4, 6).toInt val dir = dms.last
10      val decimal = degs + (mins / 60.0) + (secs / 3600.0)
11      if (dir == ctsSouth || dir == ctsWest) -decimal else decimal
12    }).apply(col(ctsLatitud)), 6)),
13     ctsLongDec -> ...)
14   formatters.foldLeft(dataframe) {
15     case (accumulatedDf, (colName, transformationFunc)) =>
16       accumulatedDf.withColumn(colName, transformationFunc(colName))
17   }
18 }

```

Figura 8: Código para formatear los datos del dataframe de estaciones

Para terminar, los DFs, la sesión de Spark y el sistema de almacenamiento se pasan al constructor de `SparkSessionCore`, creando así la envoltura mencionada anteriormente.

■ Subsistema de ejecución de consultas:

Esta parte del sistema es capaz de realizar consultas sobre los datos. Destacan los ficheros `SparkQueriesManager.scala` y `SparkQueriesCore.scala`, y el directorio `Studies`, que será descrito posteriormente. El primer elemento define el objeto `SparkQueriesManager` que orquesta la creación del núcleo del subsistema con `createSparkQueriesCore()` inyectando `SparkSessionCore` como dependencia. El segundo elemento define la clase `SparkQueriesCore` con las funciones que definen el plan de acción de las consultas, realizadas por los estudios, mediante funciones de alto nivel aportadas por el módulo `SQL` de Spark. A continuación se mencionarán todas las funciones que realizan consultas y el resultado que se obtiene. Puede consultarse una ampliación de cada una a nivel de implementación en el apéndice A, [Sección A.4](#). [41, 39]

- **`getStationInfoById()`**: devuelve la información de una estación por identificador.
- **`getStationCountByColumnInLapse()`**: devuelve el número de estaciones meteorológicas filtradas por una columna durante un intervalo de tiempo determinado.
- **`getStationsCountByParamIntervalsInALapse()`**: devuelve el número de estaciones meteorológicas filtradas por una columna, distribuidas por intervalos, durante un tiempo determinado. Cabe destacar que los intervalos son dados por una lista de tuplas con la columna del criterio de búsqueda, límite inferior del intervalo y límite superior.
- **`getStationMonthlyAvgTempAndSumPrecInAYear()`**: devuelve de una estación dada la media de las temperaturas y la suma de las precipitaciones cada mes durante un año concreto.
- **`getClimateParamInALapseById()`**: devuelve el valor de los parámetros climáticos tomados desde una estación (dado su identificador) durante un periodo de tiempo determinado.
- **`getTopNClimateParamInALapse()`**: devuelve un conjunto de N valores ordenados, ya sea de menor a mayor o viceversa, de los valores extremos alcanzados por una variable climática para todas las estaciones, durante un tiempo determinado.
- **`getTopNClimateConditionsInALapse()`**: devuelve un conjunto de N valores ordenados, ya sea de menor a mayor o viceversa, de las estaciones, o bien, las provincias, que cumplan con una mayor o menor cantidad de días en los que se hayan registrado valores de una serie de parámetros climáticos entre un intervalo dado, durante una franja temporal.
- **`getClimateYearlyGroupById()`**: devuelve el valor de unos parámetros climáticos agrupados anualmente, tomados desde una estación dado su identificador y durante una franja temporal.
- **`getAllStationsByStatesAvgClimateParamInALapse()`**: devuelve un valor representativo referente a un parámetro climático por cada estación de un conjunto de provincias durante un periodo de tiempo.
- **`getStationClimateParamRegressionModelInALapse()`**: devuelve los valores característicos de un modelo de regresión lineal basado en los valores de un parámetro climático concreto durante un espacio temporal para una estación dada.

- **getTopNClimateParamIncrementInAYearLapse()**: devuelve una serie de N valores, ordenados de mayor a menor o viceversa de los mayores o menores incrementos del valor de un parámetro climático dado, durante un espacio de tiempo, tomado desde un conjunto de estaciones.

■ Subsistema de realización de estudios:

Esta sección del sistema queda integrada por los ficheros del directorio **Studies**. Su propósito es la realización de estudios en base a una serie de parámetros especificados, haciendo uso de las consultas anteriormente descritas.

Una de las partes principales del subsistema esta en el fichero **StudyQueriesCore.scala** que contiene la clase **StudyQueriesCore**. Define la función **simpleFetchAndSave()** que realiza la operativa del estudio. Se basa en la llegada de una lista de objetos agrupados en torno a una estructura de datos personalizada en la que llega información sobre la consulta a realizar, el directorio de guardado, diferentes mensajes para mostrar por consola o si debe guardarse la información en **JSON** o **Parquet**. Por cada uno de estos elementos dentro de la lista se itera dentro de la función principal, ejecutando la consulta, mostrando la información obtenida por pantalla si se desea y guardando los datos dentro del sistema de almacenamiento.

Otro fichero clave es **StudyQueriesManager.scala** que contiene el objeto **StudyQueriesManager** destinado a crear los núcleos de cada uno de los estudios a realizar. La creación se gestiona en la función **createStudyQueriesCore()** que estandariza el proceso de creación de cada uno de los núcleos obligándolos a heredar la clase **StudyQueriesCore** para así poder acceder a la función **simpleFetchAndSave()** (véase: [Figura 9](#)).

```
1  protected def simpleFetchAndSave(qryTle: String = "", qrs: Seq[FSInf], enclHLenSt:
    Int = 35): Seq[DataFrame] = {
2      // PRINT START QUERY TITLE
3      qrs.foreach(subQry => {
4          // PRINT START SUBQUERY TITLE
5          if (sparkSessionCore.showResults) subQry.dataframe.show()
6          // PRINT DEST PATH
7          if (!subQry.saveAsJSON)
8              sparkSessionCore.saveDataframeAsParquet(subQry.dataframe, subQry.
                pathToSave) match {...}
9          else
10             sparkSessionCore.saveDataframeAsJSON(subQry.dataframe, subQry.pathToSave)
                match {...}
11         // PRINT END SUBQUERY TITLE
12     })
13     // PRINT END QUERY TITLE
14     queries.map(query => query.dataframe)
15 }
```

Figura 9: Código de la función **simpleFetchAndSave()**

Finalmente se encuentran los distintos ficheros con el código de los núcleos para los estudios a realizar. Cada fichero se compone de una clase **[ESTUDIO]QueriesCore** con una única función **execute()** con la secuencia de llamadas a **simpleFetchAndSave()** por cada elemento a estudiar.

- **StationsQueriesCore.scala**: se compone de tres elementos respecto al estudio de la distribución de estaciones meteorológicas.

- Evolución del número de estaciones: haciendo uso de la consulta `getStationCountByColumnInLapse()`. Se estudia su evolución desde 1973 hasta 2024.
- Cantidad de estaciones por provincia: haciendo uso de la consulta `getStationCountByColumnInLapse()`. Se estudia su distribución en 2024 a lo largo de las 52 provincias españolas.
- Cantidad de estaciones por intervalos de altitud: haciendo uso de la consulta `getStationsCountByParamIntervalsInALapse()`. Se estudia su distribución en intervalos de altitud de 0 hasta el máximo en incrementos de 500 metros.
- **ClimographsQueriesCore.scala**: se realizarán dos consultas sobre cada uno de los climas descritos en la [Tabla 6](#). Cabe destacar que para la realización de este estudio se ha tenido que realizar un análisis previo para encontrar estaciones representativas por cada clima y localización, integrándolas en un fichero de constantes.
 - Obtención de datos de la estación representativa: haciendo uso de la consulta `getStationInfoById()`. Se guardará la información en formato JSON para su posterior uso de forma sencilla en el proceso de representación de resultados, aspecto que siempre se repetirá por cada una de las llamadas a esta consulta en el resto de estudios.
 - Obtención de los datos del climograma: haciendo uso de la consulta `getStationMonthlyAvgTempAndSumPrecInAYear()`. Se toman los valores para la estación representativa durante el año de observación 2024.
- **SingleParamStudiesQueriesCore.scala**: se realizan varias consultas sobre los parámetros climáticos individuales de temperatura, precipitación, velocidad del viento, presión atmosférica, radiación solar y humedad ambiental.
 - Estudio sobre valores extremos: haciendo uso de la consulta `getTopNClimateParamInALapse()` se estudian los diez mayores y menores valores extremos en el intervalo temporal del año 2024, la década comprendida entre 2014 y 2024, y desde 1973 hasta 2024.
 - Estudios sobre evolución temporal: se estudia sobre cada una de las 52 provincias españolas y se tienen en cuenta dos temporalidades, desde 1973 hasta 2024 y todo el año 2024. Primero se extraen los datos de la estación representativa para cada una de las temporalidades haciendo uso de la consulta `getStationInfoById()`. Después, para el año 2024 se extrae su evolución con la consulta `getClimateParamInALapseById()` y para el intervalo de 1973 a 2024 con `getClimateYearlyGroupById()`. Finalmente se toman los resultados del modelo de regresión inferido de los datos del intervalo de 1973 a 2024 con la consulta `getStationClimateParamRegressionModelInALapse()` que serán usados a continuación en el siguiente estudio.
 - Estudio sobre mayores o menores incrementos: se estudian los mayores incrementos y decrementos en la temporalidad desde 1973 hasta 2024 mediante el uso de la consulta `getTopNClimateParamIncrementInAYearLapse()`. Esta consulta necesita el modelo de regresión para calcular el incremento por lo que es proporcionado de la ejecución del anterior conjunto de consultas.
 - Estudio sobre registro climático completo del territorio español durante 2024: se toman todos los registros de todas las estaciones, por una parte del territorio continental y por otra, de las Islas Canarias, durante el año 2024, haciendo uso de la consulta `getAllStationsByStatesAvgClimateParamInALapse()`.

- **InterestingStudiesQueriesCore.scala**: compuesto por dos estudios.
 - Evolución temporal de la comparativa entre precipitación y presión atmosférica: se estudia sobre estaciones representativas tanto para el intervalo temporal de 1973 hasta 2024 como para el año 2024. En primer lugar se toma la información de las estaciones con la consulta `getStationInfoById()` y después para el año 2024 se toman los valores de evolución con la consulta `getClimateParamInALapseById()` y para el intervalo de 1973 hasta 2024 con la consulta `getClimateYearlyGroupById()`.
 - Lugares con condiciones climáticas especiales: se buscan las diez provincias donde es más probable que se de una condición climática especial (descritas en capítulos anteriores). Para ello, se ha creado una lista de intervalos de valores por cada parámetro climático influyente en cada condición especial, guardando esta información en los ficheros de constantes. Para la obtención se hace uso de la consulta `getTopNClimateConditionsInALapse()`.

Véase en la [Figura 10](#) un ejemplo de como se ejecuta cada sección de un estudio.

```
1 simpleFetchAndSave(  
2   ctsLogs.stationCountEvolFromStart,  
3   List(  
4     FetchAndSaveInfo(  
5       sparkQueriesCore.getStationCountByColumnInLapse(  
6         column = (year(col(ctsParam)), ctsParamSelectName),  
7         startDate = ctsExecution.countEvolFromStart.startDate,  
8         endDate = Some(ctsExecution.countEvolFromStart.endDate)) match {  
9           case Left(exception: Exception) => ...  
10          case Right(dataFrame: DataFrame) => dataFrame  
11        }, ctsStorage.countEvolFromStart.data  
12      )  
13    )  
14  )
```

Figura 10: Código de la ejecución de la sección de un estudio

Descripción específica de la fase de representación de resultados

En este punto se explica desde el punto de vista del *software* el sistema que representa los resultados obtenidos tras la ejecución de los estudios. Como ya se ha mencionado anteriormente a lo largo de la memoria, este sistema hace uso de otro externo que provee resultados representados en gráficos, denominado Auto-Plot, por lo que la parte a explicar en este punto es la referente al cliente de dicho sistema.

Este sistema se integra en el fichero `PlotGeneratior.scala` que define un objeto contenedor del resto de elementos llamado `PlotGenerator`. Dentro de este objeto se encuentran otros por cada uno de los estudios realizados, es decir: `Stations`, referente a los estudios sobre distribución de estaciones; `Climograph`, referente a los estudios de distribución climática; `SingleParamStudies`, referente a los estudios sobre variables climáticas individuales; y `InterestingStudies`, referente a los estudios de interese climático. Cada uno de estos estudios contiene una función `generate()` con el código principal donde se hacen las peticiones a Auto-Plot.

Por lo que respecta a el objeto principal, también cuenta con una función `generate()` con el siguiente comportamiento:

1. Llamada a `generate()` de `Stations`.
2. Llamada a `generate()` de `Climograph`.
3. Llamada a `generate()` de `SingleParamStudies`.
4. Llamada a `generate()` de `InterestingStudies`.

En cuanto al resto de funciones de este objeto principal, se encuentra `generatePlot()`, encargada de realizar la petición a Auto-Plot. Este sistema provee una serie de *endpoints* a los que se puede hacer una petición `POST` por cada tipo de gráfico a generar, pasándole la información necesaria para su construcción en el cuerpo de la petición. Cabe destacar que los gráficos que se permiten realizar son: gráficos de barras, climogramas, lineales dobles y simples (estos últimos con y sin recta de regresión), de sectores circulares, mapas de calor y tablas.

La información referente al cuerpo es modelada por el sistema mediante un *data transfer object* (DTO), definido, por cada uno de los *endpoints*, en los archivos de representación interna de constantes, modelando cada uno de los elementos que necesita el JSON de la petición. Así pues, en el archivo de configuración se establece una plantilla para cada petición a realizar que posteriormente será rellenada con información individual para cada tipo de estudio. Esta plantilla se rellena con datos importados desde las constantes en funciones de formato que modifican el DTO generando copias de los objetos internos a modificar con la función `copy()` (véase: [Figura 11](#)).

```

1 private def barDTOTop5IncFormatter(dto: BarDTO, formatInfo: Top5IncFormatInfo):
2     BarDTO = {
3         dto.copy(
4             src = dto.src.copy(
5                 path = dto.src.path.format(
6                     formatInfo.meteoParamInfo.meteoParamAbbrev,
7                     formatInfo.order
8                 )
9             ),
10            dest = dto.dest.copy(
11                path = dto.dest.path.format(...)
12            ),
13            style = dto.style.copy(
14                lettering = dto.style.lettering.copy(
15                    title = dto.style.lettering.title.format(...),
16                    yLabel = dto.style.lettering.yLabel.format(...)
17                )
18            )
19 }

```

Figura 11: Código de la función `barDTOTop5IncFormatter()`

Finalmente, mediante un método de *parsing* automático de la librería de `uPickle`, permite el uso del *trait* `ReadWriter` sobre cada clase que compone el DTO para convertirlo a JSON mediante la función `writeJs()` [50], consiguiendo un `String` con la información en crudo lista para enviar, y con ella, se usa la función `sendPostRequest()` de la librería de utilería `HTTPUtils` para realizar la petición al servidor que será el encargado de generar el gráfico y almacenarlo.

Para terminar, se describirá a continuación, por cada estudio, los gráficos que serán pedidos a generar:

- **Stations.**

- Evolución del número de estaciones: gráfico lineal, tiempo frente a número de estaciones.

- Cantidad de estaciones por provincia: gráfico en formato tabla con dos columnas, provincia y número de estaciones.
- Cantidad de estaciones por intervalos de altitud: gráfico de sectores circulares. Tantos sectores como intervalos de altitud y cada sector tiene una amplitud proporcional al número de estaciones por intervalo.
- **Climograph:** gráfico tipo climograma basado en el esquema descrito en capítulos anteriores. Los datos que se extrajeron respecto a la estación, serán usados para dar formato al cuerpo de la petición, usando Auto-Plot dicha información para rellenar diferentes títulos. Este proceso será común a todos los estudios que necesitaron extraer los datos sobre la estación representativa.
- **SingleParamStudies.**
 - Estudio sobre valores extremos: gráfico de barras, donde la altitud de la misma es el valor a representar. Cada barra esta ordenada en formato *ranking*.
 - Estudios sobre evolución temporal: gráfico lineal simple para la información del año 2024 y lineal con la recta de regresión para la información de 1973 hasta 2024.
 - Estudio sobre mayores o menores incrementos: gráfico de barras al igual que el referente al estudio sobre valores extremos.
 - Estudio sobre registro climático completo del territorio español durante 2024: gráfico tipo mapa de calor del territorio continental y de las Islas Canarias.
- **InterestingStudies.**
 - Evolución temporal de la comparativa entre precipitación y presión atmosférica: gráfico doble lineal, parámetro frente a tiempo.
 - Lugares con condiciones climáticas especiales: gráfico en formato tabla con dos columnas, provincia y posición en el *ranking*.

Descripción específica de la librería de utilería

Esta última sección del sistema principal se compone de varios módulos con funciones necesarias para el del código de las diferentes fases del proyecto.

■ Utilería para el manejo de aspectos temporales:

El manejo del tiempo es importante en el sistema, destacando el uso de cronómetros para medir los tiempos de ejecución, o las esperas que se hacen en las funciones de extracción de datos para no obtener una expulsión de las APIs. También destaca el uso de funciones para realizar avances entre fechas, necesario para el proceso de extracción de datos de AEMET. Estas funciones se encuentran en dos ficheros.

Por un lado, `ChronoUtils.scala` define un objeto con funciones para los tiempos de espera usando la función `Thread.sleep()` que permite pausar la ejecución un número de milisegundos. También está la clase `Chronometer` que define un cronómetro.

Por otro lado, `DateUtils.scala` define un objeto con funciones aplicadas a realizar operaciones sobre fechas. Permite transformar `Strings` en objetos que representan una fecha como `LocalDate` o `ZonedDateTime`, así como operar con horas, minutos y segundos para avanzar un lapso de tiempo sobre una fecha dada. [12]

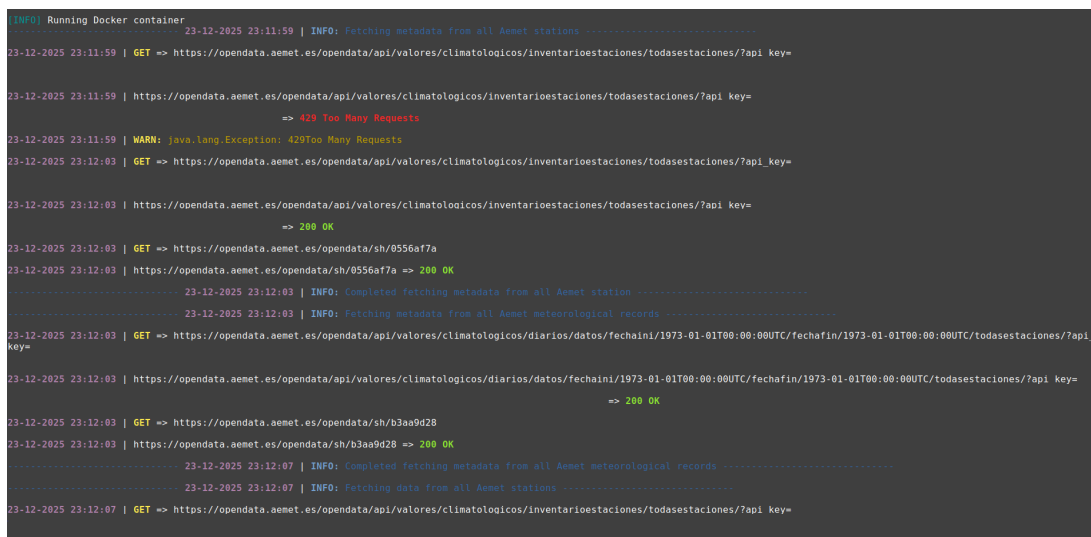
■ Utilería para impresión de mensajes por consola:

Este módulo se centra en el código del fichero `ConsoleLogUtils.scala` que define un sistema para imprimir mensajes por pantalla con un estilo personalizado centrado en el color del texto y su formato, incluyendo la hora y tabulaciones. Por lo que compete al uso de colores, la librería `Fansi` posibilita un uso más sencillo. [48] Respecto a la composición del código, cuenta con dos secciones:

La primera, centrada en funciones para la impresión de solicitudes HTTP con la función `printlnHTTPMethod()` que permite, con el uso de un enumerado, la elección del método HTTP para imprimir, cada uno con un color diferente. También se pueden imprimir las respuestas enviadas por el servidor con `printlnResponse()`. Cabe destacar que ambas funciones hacen uso de la librería `STTP Client 4` para el tratamiento de aspectos HTTP.

La segunda tiene funciones para imprimir mensajes genéricos. Se define un enumerado con diferentes tipos de mensajes: información, alerta y error, cada uno asociado a un color. Pueden representarse así mensajes con `printlnConsoleMessage()` o con `printlnConsoleEnclosedMessage()` si se desea usar un carácter para indentar el mensaje. Estas funciones aceptan como parámetro uno de los tipos de mensaje generando así un estilo propio.

A lo largo de todo el código pueden verse sentencias que hacen uso de estas funciones que permiten imprimir mensajes por consola durante la ejecución mostrando el progreso del programa. A continuación se muestra una captura de la ejecución durante el proceso de extracción de datos con esos mensajes personalizados (véase: [Figura 12](#)).



```
[INFO] Running Docker container
23-12-2025 23:11:59 | INFO: Fetching metadata from all Aemet stations .....
23-12-2025 23:11:59 | GET => https://opendata.aemet.es/opendata/api/valores/climatologicos/inventarioestaciones/todasestaciones/?api_key=
23-12-2025 23:11:59 | https://opendata.aemet.es/opendata/api/valores/climatologicos/inventarioestaciones/todasestaciones/?api_key=
=> 429 Too Many Requests
23-12-2025 23:11:59 | WARN: java.lang.Exception: 429Too Many Requests
23-12-2025 23:12:03 | GET => https://opendata.aemet.es/opendata/api/valores/climatologicos/inventarioestaciones/todasestaciones/?api_key=
23-12-2025 23:12:03 | https://opendata.aemet.es/opendata/api/valores/climatologicos/inventarioestaciones/todasestaciones/?api_key=
=> 200 OK
23-12-2025 23:12:03 | GET => https://opendata.aemet.es/opendata/sh/0556af7a
23-12-2025 23:12:03 | https://opendata.aemet.es/opendata/sh/0556af7a => 200 OK
23-12-2025 23:12:03 | INFO: Completed fetching metadata from all Aemet station .....
23-12-2025 23:12:03 | INFO: Fetching metadata from all Aemet meteorological records .....
23-12-2025 23:12:03 | GET => https://opendata.aemet.es/opendata/api/valores/climatologicos/diarios/datos/fechaini/1973-01-01T00:00:00UTC/fechafin/1973-01-01T00:00:00UTC/todasestaciones/?api_key=
23-12-2025 23:12:03 | https://opendata.aemet.es/opendata/api/valores/climatologicos/diarios/datos/fechaini/1973-01-01T00:00:00UTC/fechafin/1973-01-01T00:00:00UTC/todasestaciones/?api_key=
=> 200 OK
23-12-2025 23:12:03 | GET => https://opendata.aemet.es/opendata/sh/b3aa9d28
23-12-2025 23:12:03 | https://opendata.aemet.es/opendata/sh/b3aa9d28 => 200 OK
23-12-2025 23:12:07 | INFO: Completed fetching metadata from all Aemet meteorological records .....
23-12-2025 23:12:07 | INFO: Fetching data from all Aemet stations .....
23-12-2025 23:12:07 | GET => https://opendata.aemet.es/opendata/api/valores/climatologicos/inventarioestaciones/todasestaciones/?api_key=
```

Figura 12: Captura de pantalla de la consola durante el proceso de extracción de datos de AEMET

■ Utilería para peticiones HTTP:

Este módulo proporciona funciones de alto nivel que permiten realizar peticiones `GET` y `POST` a una dirección. El funcionamiento de este sistema se basa en el uso de la librería `STTP Client 4`. Entre las funcionalidades del sistema destacan varias:

En primer lugar, se posibilita el trabajo con URLs compuestas por segmentos y parámetros, de modo que se ha diseñado una función `buildUrl()` sobrecargada para poder

trabajar en varios casos donde cada uno de dichos elementos este presente en la dirección. Su funcionamiento se centra en el uso del objeto `Uri` de la librería.

Por otro lado, se permite la realización de peticiones con las funciones `sendGetRequest()` para `GET` y `sendPostRequest()` para `POST`. Las funciones tienen un comportamiento similar, haciendo uso de un *back-end* que procesa y envía la petición, escogiendo `HttpURLConnectionBackend()` para `GET` y `OkHttpSyncBackend` para `POST` (este último permite definir un cliente personalizado donde establecer mayores tiempos de *time-out*). A continuación, se definen unas cabeceras que son insertadas en un objeto `quickRequest` encargado de gestionar la petición. La recepción de la respuesta se procesa sobre el valor devuelto de su función `send()`, permitiendo discernir con el código de error si fue exitosa o no, y si lo fue, obtener la información recibida. [87]

■ Utilería para el trabajo sobre archivos JSON:

Este módulo se centra en el fichero `JSONUtils.scala` que aporta funciones para trabajar sobre JSON con el módulo `uJSON` de la librería de `uPickle`. Las funciones permiten realizar modificaciones a los objetos JSON basándose en una estrategia de recorrido recursivo sobre la estructura y aplicando las transformaciones pertinentes. Así pues, `uJSON` permite codificar un JSON en un objeto `Value` y acceder a sus diferentes elementos con el uso de la recursión y estrategias de desestructuración por pares clave y valor mediante la correspondencia por patrones o *pattern matching*. [49]

En el código se encuentran funciones como `lowercaseKeys()` que convierte todas las claves del JSON a minúscula, `removeNullKeys()` que elimina las claves que contengan valores nulos, `transformJSONValues()` que permite la transformación de los valores del JSON con la aplicación de funciones por cada clave, y `buildJSONFromSchemaAndData()` que permite la construcción de un nuevo JSON dado un esquema, muy importante en el proceso de transformación del formato de los datos de IFAPA al de AEMET.

■ Utilería para el trabajo sobre el sistema de almacenamiento:

Este módulo contiene el código del directorio `Storage`, cuya creación busca diseñar un sistema de almacenamiento de interfaz única que permita trabajar sobre un sistema local como remoto con AWS S3.

Así pues, este sistema basa su arquitectura en la implementación de un núcleo donde se definen las funciones necesarias tanto para el sistema local como remoto y sobre este, se crean implementaciones para cada tipo de archivo a tratar por el sistema. En concreto, se da soporte a archivos JSON, necesarios para la comunicación con APIs; archivos Parquet, el formato de archivo elegido para trabajar con Spark; y archivos de configuración usados por la librería `PureConfig` para gestionar las constantes del proyecto.

En el apéndice A, [Sección A.5](#), hay una explicación detallada y de lectura recomendada sobre la implementación de este sistema.

3.1.3. Sistema Auto-Plot para generación de gráficos

Como se mencionó en la descripción del proyecto, el proceso de representación de los resultados obtenidos usando gráficos estaría incluido por dos componentes: un servidor

capaz de recibir peticiones con el tipo de gráfico a generar y su información pertinente, y un sistema cliente que envíe dicha información. Así pues, el propósito de este punto es la explicación del servidor proveedor de gráficos, denominado Auto-Plot.

A continuación, se dará una descripción específica de cada aspecto del sistema. Cabe destacar que en el apéndice A se puede consultar la jerarquía de directorios y ficheros en la [Sección A.6](#) y una descripción global del sistema en base a un diagrama de componentes en la [Sección A.7](#). La lectura de ambos contenidos es recomendada para comprender mejor el alcance de cada sistema. Puede visualizarse igualmente un diagrama de componentes simplificado en la [Figura 13](#)

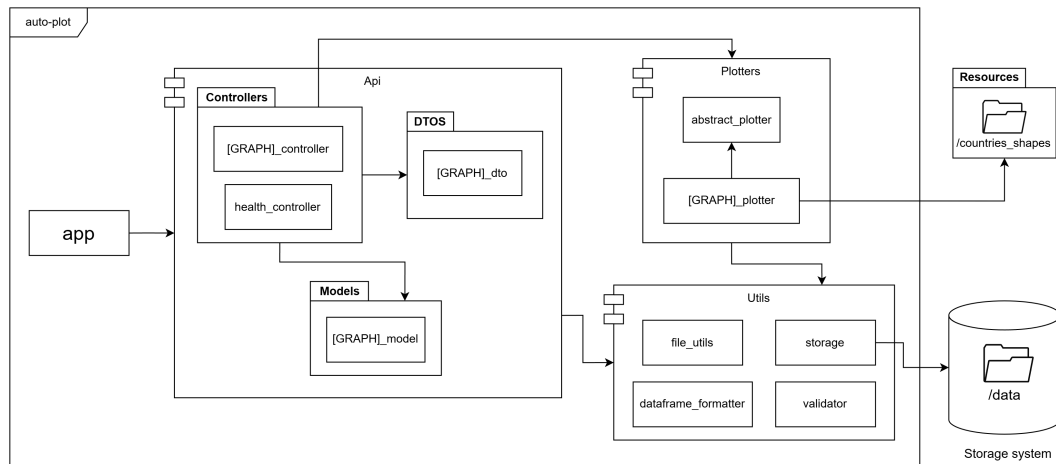


Figura 13: Diagrama de componentes simplificado de Auto-Plot

Descripción específica del punto de entrada a la aplicación

Se explica el código del fichero `app.py`, desde donde la aplicación inicia, recepcionando las peticiones entrantes y redirigiendo en base a la ruta de las mismas a su controlador correspondiente. Este funcionamiento se logra gracias a la definición de un objeto `Api`, propio del *plug-in* Flask-RESTX para Flask. Este objeto permite definir una aplicación basada en una API REST que da la posibilidad de abrir múltiples *endpoints* para acceder a recursos del sistema. Cada uno de estos queda definido bajo una dirección. [22]

Respecto a la definición de las direcciones, han de definirse tantos *namespaces* como recursos haya en el sistema. Estos elementos exponen un esquema de datos que el cuerpo de la petición debe cumplir, y una vez definidos, son importados al objeto `Api` con su función `add_namespace()`, donde se asocia el *namespace* con la ruta del recurso. [23] A continuación, se muestran las rutas creadas con el recurso al que hacen referencia:

- **/health**: comprobación de la salud del sistema.
- **/bar**: creación de un gráfico de barras.
- **/pie**: creación de un gráfico de sectores circulares.
- **/table**: creación de un gráfico formato tabla.
- **/linear**: creación de un gráfico lineal.
- **/linear-regression**: creación de un gráfico lineal con recta de regresión.
- **/double-linear**: creación de un gráfico lineal doble.
- **/climograph**: creación de un gráfico tipo climograma.
- **/heat-map**: creación de un gráfico tipo mapa de calor.

Finalmente, se definen algunos aspectos de configuración referentes al servidor Gunicorn que despliega la aplicación, y se da comienzo al sistema con la función de Flask, `run()`, que abre un puerto en el sistema desde donde se comienzan a escuchar peticiones.

Descripción específica del núcleo del sistema

Se describe a nivel de *software* todos los elementos del núcleo de Auto-Plot, contenidos en el directorio `Api`, integrado por los controladores, modelos y DTOs. Cabe destacar que en cada uno de estos ámbitos se han creado tantos ficheros como recursos del sistema existen, por lo que se dará una explicación general sobre el esquema que siguen, que en todos es similar, salvo si existe alguna excepción que diferencie a algún elemento.

■ Data transfer objects (DTOs):

Se describe el código de los ficheros del directorio `DTOS`, que contiene las definiciones del esquema que debe seguir el cuerpo de cada una de las peticiones recepcionadas por el sistema. Este esquema se interpreta como la definición del *namespace* para un recurso.

En cuanto a su estructura, se basa en la creación de una clase, donde el constructor define un atributo `post_input` usando una función auxiliar para exponer el esquema llamada `_create_input_post_dto()` que hace uso de varias funciones de Flask-RESTX. Su construcción se basa en la aplicación de `model()` que admite un diccionario constituido por claves, los nombres de los campos del JSON del cuerpo de la petición, y valores, los objetos que modelan su estructura interna, que a su vez recursivamente también admiten otros diccionarios. Cada objeto de estas estructuras de datos se construye con objetos `Fields` que permite acceso a funciones de definición que pueden ser referentes a tipos de datos atómicos, es decir, `String()` o `Float()`; o compuestos, como `List()` o `Nested()`. También las funciones necesitan definir si el elemento al que hacen referencia es requerido, así como una descripción (véase: [Figura 14](#)). [20]

```
1  'src': fields.Nested(ns.model('BarSrc', {
2      'path': fields.String(required=True, description="Relative route to data"),
3      'axis': fields.Nested(ns.model('BarSrcAxis', {
4          'x': fields.Nested(ns.model('BarSrcAxisX', {
5              'name': fields.String(required=True, description="X Column name")
6          }, required=True),
7          'y': fields.Nested(ns.model('BarSrcAxisY', {
8              'name': fields.String(required=True, description="Y Column name")
9          }, required=True)
10     }, required=True)
11 })),
```

Figura 14: Código con un extracto de la definición de un DTO

Esta estructura permite al sistema conocer un esquema del formato que deben seguir los datos recepcionados, lo que posibilita la aplicación de validaciones en cuanto a su forma, así como la posible construcción de una documentación de los recursos con Swagger.

En cuanto a la estructura del esquema diseñado para el cuerpo de la petición se han tenido en cuenta tres aspectos principales:

- **Datos de entrada:** esta sección del JSON se integra dentro de la clave `src` desde donde se define la información necesaria para el acceso a los recursos que construyen los gráficos, es decir, los resultados de los estudios de Spark. En su interior se encuentra normalmente la siguiente estructura:

- **path**: la ruta de acceso a los datos.
- **axis**: definiciones del nombre de las columnas a utilizar y su formato. Suele mostrarse un formato con tantas claves como ejes haya, normalmente **x** y **y**.
- **Datos de salida**: esta sección del JSON se integra dentro de la clave **dest** desde donde se definen directivas sobre los gráficos creados respecto a su almacenamiento. En su interior se encuentra normalmente la siguiente estructura:
 - **path**: la ruta de almacenamiento.
 - **filename**: el nombre del fichero.
 - **export_png**: booleano que indica si el gráfico se exportará también en formato PNG (siempre lo hace en HTML).
- **Especificación de estilos**: esta sección del JSON se integra dentro de la clave **style** desde donde se definen aspectos de estilo del gráfico a generar. En su interior se encuentra normalmente la siguiente estructura:
 - **lettering**: aspectos referentes a títulos y descripciones. Cabe destacar que integra un sistema para la definición de etiquetas con descripciones dinámicas.
 - **figure**: valores referentes a la construcción de la figura del gráfico: color, nombres, posiciones...etc.
 - **margin**: márgenes del gráfico para cada borde.
 - **legend**: valores para la leyenda. Posibilita su posicionamiento o presencia.

Una vez definida esta estructura, permite mantener un objeto interno donde el sistema realiza un proceso de *parsing* del JSON recibido al DTO desde donde se puede acceder a la información. Puede apreciarse en la [Figura 57](#) un ejemplo de JSON presente en el cuerpo de una petición para la generación de un gráfico lineal.

■ Modelo de datos:

Se describe el contenido del directorio **Models** con los ficheros que definen el modelo de datos. El uso de este modelo frente directamente al DTO radica en la necesidad de definir una estructura de datos más accesible, por medio de atributos de clases y no claves sobre un diccionario. Además, definir un modelo de datos permite realizar validaciones más profundas que las permitidas por el modelo anterior que se limita a validar sobre el esquema. De este modo, pueden aplicarse restricciones como, por ejemplo, que todas las rutas sean correctas respecto al sistema de almacenamiento, o que algunos parámetros concretos cumplan expresiones regulares, como los colores que tienen que estar en hexadecimal, o que algunos valores se encuentren entre un intervalo.

Así pues, en cuanto al código de cada uno de los modelos, se aprecia la creación de una clase, con el nombre del recurso al que hace referencia, compuesta por tres funciones principales:

La primera, el constructor, donde se definen los atributos de la clase que siempre son una referencia al objeto encargado de la gestión del sistema de almacenamiento llamado **storage**, y los atributos **src**, **dest** y **style** que modelan la estructura de cada una de las secciones del DTO ya descritas, mediante clases secundarias. Cabe destacar que estas clases cuentan con las mismas tres funciones que están siendo descritas.

La segunda función, denominada **setup()**, se encarga de llamar a las funciones **setup()** de cada una de las clases que modelan la estructura del DTO, pasándole como argumento la sección del diccionario con los datos a la que hace referencia. Este proceso se realiza así

recursivamente hasta los elementos atómicos donde se aplica la función para diccionarios `get()` sobre una clave para obtener el valor, o en su ausencia, uno preestablecido.

Finalmente, la tercera función, `validate()`, realiza los procesos de validación, usando una clase complementaria del módulo de utilería llamada `Validator`. Este último tiene la capacidad de ir acumulando errores y gestionar si el modelo es correcto o no. El funcionamiento del código de la función `validate()` es el mismo que en el caso anterior, aplicando la validación correspondiente en los elementos atómicos, de modo que, en caso de que se detecte un fallo, se llama a `set_invalid()` de `Validator`, que invalida automáticamente el modelo y acumula un mensaje de error mediante `add_error_msg()` (véase: [Figura 15](#)).

```
1 class Src:
2     def __init__(self):
3         self.path: Optional[Path] = None
4         self.axis = Axis()
5
6     def setup(self, data: dict):
7         self.path = get_src_path(data['path'])
8         self.axis.setup(data['axis'])
9
10    def validate(self, validator: Validator):
11        if not validator.storage.exists(self.path.as_posix()):
12            validator.set_invalid()
13            validator.add_error_msg('Source path must be a valid path')
14
15        self.axis.validate(validator)
```

Figura 15: Código con la definición de una clase del modelo de datos

■ Controladores:

Este punto se centra en el directorio `Controllers`, donde están los ficheros con los controladores de la API. Por cada uno de los recursos definidos se crea un controlador, siendo el punto al que se enrutan las peticiones, ejecutando así un código concreto.

En cuanto a la descripción del código, en primer lugar se define el objeto `Namespace` del recurso y, a continuación, se crea el DTO pasándole como argumento el `Namespace`. Después, se define la clase controlador que hereda de `Resource` de Flask-RESTX.

Las clases controlador en Flask-RESTX se definen creando funciones para cada uno de los verbos HTTP que admite. Así se define en la mayoría de los controladores una función `post()` (acepta POST) con el decorador `@[namespace].expected()`, que define el DTO sobre el que se realizará el *parsing* del cuerpo de la petición y la necesidad de validar o no el mismo. Cabe destacar que para la ruta `/health` se define un controlador más simple, solo con la función `get()`, ya que solo admite GET. [\[21\]](#)

Dentro de las funciones `post()` se crea un objeto `Storage` que da acceso al sistema de almacenamiento. A continuación, se crea el modelo de datos a utilizar y se le pasa la referencia al almacenamiento, y una vez creado, se llama a su función `setup()` pasándole el DTO con el atributo `payload` del `Namespace`. Prosigue con la llamada a `validate()` que devuelve el objeto `Validator`. Si el proceso de validación no ha sido exitoso, se llama a su función `build_error_message()` que integra todos los mensajes de error acumulados en uno solo y, de este modo, el controlador define una respuesta con código 400 que incluye los mensaje de error.

Si el proceso de validación sobre el modelo de datos ha sido superado, se crea el objeto encargado de la construcción de los gráficos pasándole el modelo. Sobre este objeto se

llama a la función `create_plot()`, que devuelve un objeto con toda la información del gráfico y finalmente se guarda el mismo con `save_plot()` (véase: [Figura 16](#)).

```

1 class BarController(Resource):
2     @ns.expect(bar_dto.post_input, validate=True)
3     def post(self):
4         storage = Storage(STORAGE_PREFIX, AWS_S3_ENDPOINT)
5         model = BarModel(storage)
6         model.setup(ns.payload)
7         validation = model.validate()
8
9         if not validation.is_valid():
10             error_message = validation.build_error_message()
11             current_app.logger.warning(error_message)
12             abort(400, message=error_message)
13
14         bar_plotter = BarPlotter(model)
15         figure = bar_plotter.create_plot()
16         dest_path = bar_plotter.save_plot(figure)
17         return {'dest_path': dest_path}

```

Figura 16: Código de un controlador

Para terminar, el controlador envía un JSON con solo una clave, `dest_path`, con el String de la ruta donde se guardó el gráfico. Cabe destacar que en el caso de la ruta `/health` se devuelve un JSON con la clave `status` y como valor `'ok'`, indicando que el sistema se encuentra funcionando correctamente.

Descripción específica de los servicios de creación de gráficos

Se describe el funcionamiento de los servicios encargados de la creación de gráficos que contienen su código en los ficheros del directorio `Plotters`, con tantos elementos como tipos de gráficos a representar.

Su estructura interna se basa en el esquema marcado por la clase abstracta `Plotter`, definida en `abstract_plotter.py`, heredando el resto de clases de esta. Por tanto, quedan obligadas a definir funciones que se aprecian a continuación, lo que genera una única interfaz de trabajo muy útil para trabajar desde los controladores.

- `load_dataframe()`: carga de datos para el gráfico.
- `create_plot()`: creación de la figura usando Plotly.
- `save_plot()`: guardado de la figura generada.

El flujo de trabajo para todas las clases es similar, de modo que primero se cargan los datos necesarios para la construcción del gráfico desde la dirección que fue especificada en la petición, después se genera el gráfico en base a estos datos y aplicando los estilos correspondientes que también fueron especificados, y finalmente se exporta el resultado al sistema de ficheros con la dirección deseada.

Por lo que respecta a una explicación más profunda sobre la carga de los datos, se produce en el constructor de la clase, donde se definen los atributos `storage`, que brinda acceso al sistema de almacenamiento; `model`, que brinda acceso al modelo de datos a usar; y `dataframe`, que brinda acceso a los datos que serán plasmados en el gráfico, siendo estos tomados de los resultados de los estudios realizados por Spark, en formato `Parquet`, cargados al sistema utilizando la librería `Pandas`.

La carga se realiza a través de la función `load_dataframe()` definida en la clase heredada, donde se llama a la función `read_parquet()` de `ParquetStorageBackend`. El resultado devuelve una representación del DF con la información para el gráfico que, en algunos casos, puede necesitar un proceso transformación de formato para algunas columnas, aplicando la función de utilidad `format_df()` a la que se le pasa el DF y un diccionario con claves, las columnas a transformar, y valores, un `String` con el formato escogido.

En lo referente a la construcción de la figura del gráfico, requiere de un proceso individualizado por cada tipo de gráfico a generar, sin embargo, todo su funcionamiento se basa en el uso de las funciones aportadas por la interfaz `plotly.graph_objects` de Plotly que permite la construcción de objetos `Figure` modelando la estructura interna del gráfico. En líneas generales, para su construcción, primero es necesario elegir una función para crear la figura base. Existen múltiples ejemplos, como `Bar()`, para gráficos de barras, o `Pie()`, para sectores circulares. Cada uno de ellos permite modificar múltiples parámetros para definir el gráfico, destacando `values`, donde se insertan los datos de la columna del DF a representar, y gracias a la compatibilidad entre Pandas y Plotly hace que el traspaso de valores sea instantáneo. Ocasionalmente se desea modificar parámetros específicos del gráfico una vez creada la base, así pues, se usa la función `update_layout()`, posibilitando dicho fin. Finalmente, otro aspecto que se ha tenido en cuenta en la creación de gráficos es la necesidad de que una misma figura contenga dos gráficos diferentes, así pues, la función `make_subplots()` permite dicho fin. Cabe mencionar, que para la creación de cualquier tipo de gráfico, la documentación de Plotly aporta ejemplos completos con códigos ya prediseñados que pueden ser usados como plantillas (véase: [Figura 17](#)). [[74](#), [72](#), [75](#), [77](#)]

```
1 go.Figure(data=[go.Table(  
2     header=dict(  
3         values=style.lettering.headers,  
4         align=style.figure.headers.align.value,  
5         fill_color=style.figure.headers.color,  
6         font=dict(color='black', size=14)  
7     ),  
8     cells=dict(  
9         values=self.dataframe[self.model.src.col_names].T,  
10        align=..., fill_color=..., font=...  
11    )  
12 )]).update_layout(  
13     title=f"{style.lettering.title}<br><sup>{style.lettering.subtitle}</sup>"  
14     if style.lettering.subtitle else  
15     f"{style.lettering.title}",  
16     margin=dict(l=style.margin.left, r=style.margin.right, t=style.margin.top, b=  
17         style.margin.bottom)
```

Figura 17: Código con un ejemplo de construcción de gráfico con Plotly

Cabe destacar, que hay aspectos remarcables a tener en cuenta durante este proceso. Uno de ellos es la creación de un sistema de generación de descripciones dinámicas para los elementos del gráfico en base a la información proporcionada por las peticiones. Estas descripciones se definen en el cuerpo de la petición como una lista asociativa con los elementos `label`, dándole una etiqueta inicial a la descripción, y `build`, con una lista de objetos JSON con los elementos `text_before`, con un texto anterior; `name`, el nombre de la columna de la que extraer la información dinámica; y `text_after`, con un texto posterior. Con esta información, dentro del servicio generador de gráficos, se usa la función `_generate_point_info()` que itera por cada una de las filas del DF y basándose en el patrón especificado en la petición, va construyendo las cadenas de texto correspondientes.

Otro aspecto a tener en cuenta es que el gráfico tipo mapa de calor ha requerido un diseño mucho más complejo que el resto, dado que Plotly no permitía la creación de manera directa en base a las plantillas que proporciona. Así, se ha diseñado un algoritmo propio para su confección que diseña un mapa de calor en función de la variación del valor medido a lo largo de una superficie, sin embargo, este proceso presenta la problemática de que no se poseen valores para cada punto que se desea representar, por lo que es necesario abordar la construcción del mapa mediante un proceso de interpolación que arroje valores aproximados en base a los valores cercanos. Además, se desea representar la información siguiendo los márgenes geográficos de la Península Ibérica y de las Islas Canarias, por lo que será necesario diseñar un sistema capaz de componer una imagen con esta forma deseada. Puede consultarse en el apéndice A, [Sección A.8](#), una explicación detallada de la implementación.

Para finalizar este punto, una vez que se ha logrado obtener el objeto `Figure` de Plotly tras la creación de la estructura interna del gráfico, se procede a su exportación al sistema de almacenamiento con la función `save_plot()`, que hace uso del *back-end* que da soporte a la exportación de gráficos. Así pues, siempre se exporta en formato HTML con la función `export_html()`, permitiendo generar un componente web interactivo, y en caso de indicar en la petición que se quiere exportar como imagen en formato PNG, se usa también la función `export_png()`.

Descripción específica de ficheros de configuración

El contenido de este punto se centra en los ficheros del directorio `Config`, que contiene dos ficheros con datos estáticos utilizados durante todo el programa: el primer fichero, `constants.py`, contiene valores constantes, centrados en claves para el acceso a variables de entorno, rutas para el almacenamiento, `Strings` con identificadores o valores de posicionamiento para la construcción de mapas. El segundo fichero, `enumerations.py`, define enumerados, cuyos valores pueden ser accedidos con la función `get_enum_value()`. Entre estos se encuentran, formatos de datos, tipos de alineamiento para tablas, meses del año y localizaciones geográficas.

Descripción específica del módulo de utilería

Este módulo se centra en el contenido del directorio `Utils`, con funciones de utilidad para el resto de elementos del sistema.

Por lo que respecta a las funciones básicas, se agrupan en los siguientes ficheros:

- **`dataframe_formatter.py`**: se define una función `format_df()`, que en base a un DF de Pandas, formatea algunas de sus columnas, usando un diccionario con pares columna y nombre del formato, así aplicando una función que modifica la columna. Entre estas se encuentran ejemplos como `to_datetime()`.
- **`file_utils.py`**: define las funciones `get_src_path()`, que devuelve una ruta de datos de entrada relativa a su formato absoluto; `get_dest_path()`, igual que la anterior pero con datos de salida; y `get_response_dest_path()` que normaliza la ruta devuelta en las respuestas del servidor.
- **`validator.py`**: define una clase `Validator` para usarse como estructura de datos auxiliar en el proceso de validación del modelo de datos. Brinda la posibilidad de

almacenar mensajes de error o datos personalizados, así como obtenerlos para poder ser enviados como respuestas ante una validación errónea.

Por otro lado, se ha implementado un sistema de almacenamiento gemelo al ya explicado en la sección del programa principal, tan solo cambiando su desarrollo a Python. De este modo, la arquitectura e implementación es la misma en prácticamente todos los casos, salvo en la sección de funciones para el trabajo sobre el almacenamiento remoto con AWS S3, que usa la librería Boto3 proporcionando una API de trabajo con Python sobre S3, pero con un funcionamiento igual al del AWS SDK de Scala. [85]

Respecto a los *back-ends* que permiten el trabajo sobre formatos de archivos concretos, se ha implementado uno llamado `ParquetStorageBackend` para el trabajo sobre archivos `Parquet` usando `Pandas`, que contiene la función `read_parquet()` que hace uso de la función de `Pandas` `read_parquet()` con el motor `pyarrow`. [33] Por otro lado, se ha implementado otro *back-end* llamado `PlotExportStorageBackend` para permitir la exportación de figuras de la librería `Plotly`, dando soporte a `HTML` con la función `export_html()` y `PNG` con `export_png()`. Estas hacen uso de las funciones del objeto `Figure` `write_html()` y `write_image()` respectivamente. [73, 76]

3.1.4. Sitio web para visualización de resultados

Como complemento para este proyecto, se ha creado una web para visualizar los gráficos obtenidos. Su creación se centra en la construcción de un *front-end* basado en `Next.js`, un *framework* de `React`. A grandes rasgos, su arquitectura se fundamenta en el uso de componentes, donde cada uno define una sección de código que es interpretada por el navegador de forma independiente. Este mecanismo permite construir webs potentes, lo que facilita la carga de trabajo al momento de renderizar la misma, solo actualizando los componentes necesarios en cada momento. [52, 88].

Cada componente permite la definición de una plantilla que puede ser reutilizada en varias secciones del código, lo que permite una mayor escalabilidad del sistema. Cabe destacar que para facilitar la construcción de estos elementos se ha usado una librería de componentes ya prediseñados llamada `Shadcn`, que aporta todos los aspectos necesarios para la construcción de cualquier web moderna. Entre los componentes usados destacan: menús interactivos, secciones colapsables o selectores. El uso de los mismos es muy simple, ya que se basa en la instalación y aplicación de las plantillas proporcionadas por la documentación de la librería y modificando los parámetros necesarios para conseguir el resultado esperado. [28]

Una vez diseñada la estructura del sitio web con todos los componentes funcionales, se integran los gráficos dentro de elementos `iframe` que permiten cargar contenido `HTML` e interactuar con el mismo. Este proceso contiene una problemática referente al peso de los archivos a cargar, puesto que la totalidad de los mismos tiene un peso en disco de 3,64 GB, un valor incompatible para ser proporcionados dentro del directorio `public`, por lo que se ha decidido que sean enviados desde un servidor externo proveedor de archivos estáticos.

Para terminar, puede accederse a la web mediante el siguiente enlace. También se puede apreciar parte de su construcción en la Figura 18.

<https://meteorological-dashboard-web.vercel.app>

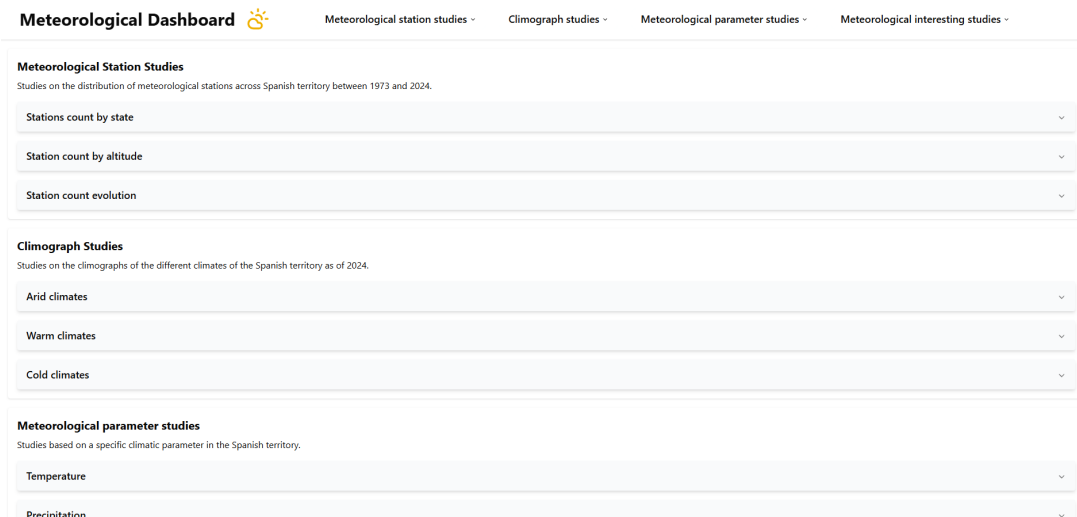


Figura 18: Captura de pantalla de la web de visualización de resultados

3.2. Despliegue

En esta última sección de este capítulo se explican las cuestiones referentes al despliegue de los sistemas y su puesta en marcha. Así pues, se describirá el proceso de compilación del sistema principal, la integración de este con el resto de sistemas en el ecosistema local y finalmente, la prueba de despliegue de la fase de estudios con Spark en AWS EMR.

3.2.1. Compilación del proyecto principal

El proceso de compilación del sistema principal se centra en el uso de SBT como herramienta principal. Este sistema permite definir un archivo `build.sbt` donde se define la estructura del proyecto así como sus dependencias y directivas de compilación.

El proyecto mantiene una estructura que consta de tres subsistemas y una librería de utilería. Cada uno de estos elementos compone un subproyecto independiente dentro de un proyecto general.

Para definir cada componente del sistema se utilizan varias funciones de SBT encadenadas. En primer lugar, se define la localización del mismo con la directiva `project in file` (todos estarán en una ruta del estilo `modules/[NOMBRE-MODULO]`). Posteriormente, se indican las dependencias con la función `dependsOn()`, pasando como parámetro los módulos pertinentes. A continuación, se activan los *plug-ins* a usar por SBT, que en el caso de este proyecto, se hará uso de `sbt-assembly`, necesario para la compilación. Finalmente, con `settings()` se definen las propiedades del módulo, entre estas, su nombre con `name`, la clase principal usada como punto de entrada con `mainClass`, sus dependencias con `libraryDependencies` (admitiendo una `Seq` de dependencias en formato `groupId%% artifactId%% version`) y el nombre del archivo JAR con el código compilado con `assembly / assemblyJarName`. [27]

Posteriormente, se definen los componentes `utils`, como librería de utilería, y dependientes de esta: `dataExtraction`, `sparkApp` y `plotGeneration`. Finalmente todos los componentes son integrados en uno general llamado `root` mediante la función `aggregate()`.

Para terminar de definir el archivo, es necesaria una estrategia de compilación para evitar conflictos entre archivos repetidos de varias librerías. Su creación se basa en el uso de *pattern matching* sobre los nombres de las librerías, y así se elige una estrategia, ya sea descartando, escogiendo el primer elemento del conflicto o el último.

Una vez definido el archivo, SBT puede compilar el proyecto mediante el comando `sbt clean compile`, generando archivos `class` con el código fuente compilado, pero sin incluir las librerías externas. Ha de usarse el comando `sbt clean assembly` para construir el archivo `fat JAR` con todo el contenido del proyecto, preparado para ser ejecutado por la JVM. Cabe destacar que si solo se quiere aplicar la compilación o ensamblaje a un módulo puede ejecutarse `sbt clean [NOMBRE_MÓDULO]/[COMANDO]`.

3.2.2. Despliegue en el ecosistema local

En este punto se explicará el proceso de despliegue de cada uno de los elementos del sistema dentro de un ecosistema local.

Tanto para el sistema principal, como para Auto-Plot, se ha diseñado un proceso de despliegue basado en Docker cuyos ficheros están en un directorio llamado `deployment`. Este directorio cuenta con tantos subdirectorios como elementos contenga el sistema, y estos a su vez, contienen un despliegue para ser usado en local y otro para utilizar la herramienta Localstack para realizar las pruebas de almacenamiento en la nube sin necesidad de usar el sistema de AWS real.

El proceso de despliegue de cada elemento individual se basa en la definición de un archivo `Dockerfile` donde se define la imagen base a usar y las directivas de construcción de la imagen, siendo principalmente: copiado de archivos, instalación de dependencias y definición de comandos de arranque. Este último archivo se ve complementado por un *script* de ejecución que permite construir la imagen Docker y ejecutarla. Cabe destacar que existe una versión de este archivo `.sh` para sistemas Linux y `.bat` para sistemas Windows. [36]

Es importante mencionar que a las imágenes de Docker se les insertan diferentes variables de entorno necesarias para el funcionamiento de los programas. Estas se definen en un archivo `.env`, en base a la plantilla `.env.template`. Las variables de entorno definen así pues, parámetros que no deben ser escritos en crudo dentro del código por su valor confidencial o su variabilidad. Entre estas se encuentran:

- *API Key* de AEMET.
- Direcciones de recursos externos como Auto-Plot o Localstack.
- Prefijos de las rutas de almacenamiento.
- Directorios base de almacenamiento.

Por lo que respecta a los sistemas que hacen uso de dos o más componentes, como es el caso del sistema de petición de gráficos junto a Auto-Plot o cuando se hace uso de Localstack, ambos cuentan con sus imágenes Docker pero es necesario definir un sistema compuesto, interconectando las imágenes entre sí para poder enviar y recibir información. Este proceso se logra con la definición de un archivo del sistema Docker Compose, que permite crear un entorno de trabajo conjunto para ambas imágenes permitiendo abrir los puertos necesarios, establecer volúmenes de persistencia de datos y variables de entorno fácilmente. Este sistema es lanzado también por *scripts* de manera automática. [35]

Cabe mencionar que estos sistemas con varios componentes necesitan que todos estén preparados para comenzar la ejecución, por lo que también se han diseñado *scripts* para Linux, que irán insertados dentro de la imagen para detectar si sus sistemas dependientes ya están operativos.

Una vez construida toda esta infraestructura, es posible ejecutar todos los programas diseñados en el entorno local, previamente compilando el proyecto. La posibilidad de ejecución haciendo uso de este sistema ha agilizado mucho la labor de desarrollo, lo que permite realizar pruebas de una manera rápida y cercana a un entorno de producción real, así como el aislamiento que brindan estos sistemas ha permitido agilizar los procesos de pruebas sin tener que lidiar con problemas derivados de incompatibilidades.

3.2.3. Despliegue de la fase de estudios con Spark en AWS EMR

Uno de los objetivos de este proyecto incluía el despliegue de la aplicación que hace uso de Spark en un *cluster* mediante los sistemas de AWS. Así pues, en este punto se describirá como se ha desarrollado este proceso.

En primer lugar, para poder utilizar el *cluster*, es necesario realizar algunas modificaciones al proyecto. De este modo, se ha creado un nuevo proyecto, con el mismo código que el general, pero solo dejando los módulos de `Utils` y el referente a los estudios con Spark. Así pues, también se eliminan dependencias innecesarias que ya el propio *cluster* provee y se actualizan de versión todos los elementos para que mantengan una coherencia con el entorno de despliegue. Una vez establecida la nueva estructura se procede a compilar y ensamblar el proyecto obteniendo un nuevo **fat JAR** con nombre `spark-app-cluster-1.0.0.jar`.

A continuación, se procede a montar la infraestructura en AWS, compuesta por un *bucket* S3 que servirá como sistema de almacenamiento, y un *cluster* EMR que servirá como centro de procesamiento. Cabe destacar que el acceso a los recursos de AWS ha sido posible gracias a la posesión de una cuenta de AWS Academy, que brinda acceso los recursos del sistema a estudiantes bajo un presupuesto limitado.

Para poder crear el entorno de despliegue, es necesario entrar en AWS Academy e iniciar una sesión en el *Lab*. Una vez establecida, aparecerá una interfaz con una consola, así como podrá accederse a información útil para conectarse remotamente mediante **AWS CLI**.

Previo paso a la ejecución de los comandos para crear el entorno, es necesario configurar **AWS CLI** con los credenciales del *Lab* con el comando `aws configure`. Una vez configurado el cliente, se procede a crear el *bucket* S3 con el siguiente comando. [84]

```
aws s3 mb s3://[unique-bucket-name]
```

Una vez creado el *bucket*, puede procederse a subir los archivos necesarios para el funcionamiento del sistema, estos son, el archivo **fat JAR**, el directorio con las configuraciones, y los datos procedentes de la fase de extracción de datos mediante el siguiente comando:

```
aws s3 cp ./big-data-core/deployment/spark-app/spark-app-cluster-1.0.0.jar s3://[unique-bucket-name]/
aws s3 cp --recursive ./data/config s3://[unique-bucket-name]/data/config/
aws s3 cp --recursive ./data/data_extraction/aemet_spark_format s3://[unique-bucket-name]/data/data_extraction/aemet_spark_format/
aws s3 cp --recursive ./data/data_extraction/ifapa_spark_format s3://[unique-bucket-name]/data/data_extraction/ifapa_spark_format/
```

Con todos los archivos subidos, se procede a configurar el *cluster* EMR. De este modo, se accede a la interfaz web de AWS y se busca el recurso **AWS EMR Service**. Una vez dentro, se pulsa la opción **Create cluster**. A continuación, se mostrará una interfaz que permite la configuración del mismo. Esta configuración debe aplicarse como sigue. [83, 58]

- **name:** meteo-study-spark-cluster
- **EMR Version:** emr-7.9.0
- **Applications:** Spark Interactive
- **Instance Types:**
 - **Master:** 1 × m5.xlarge (4 vCores 16 GiB)
 - **Core:** 2 × m5.2xlarge (8 vCores [16 vCores] 32 GiB [64 GiB])
- **Roles:**
 - **Service Role:** EMR_DefaultRole
 - **EC2 Role:** EMR_EC2_DefaultRole
- **Log Storage:** s3://[unique-bucket-name]/logs/

Tras aplicar la configuración, se pulsa en **Create cluster** y se espera mientras el estado del *cluster* sea **Waiting**. Cuando esté creado, se puede insertar un nuevo *step* con la tarea del programa de Spark. Así pues, se debe seguir esta configuración para el mismo:

- **Step Type:** Spark application
- **Name:** meteo-spark-step
- **Deploy Mode:** cluster
- **Application Location:**

`s3://[unique-bucket-name]/spark-app-cluster-1.0.0.jar`

- **Deploy Mode:** cluster
- **Spark-Submit Arguments:**

```
--master yarn --class Spark.Main --executor-memory 20G --executor-cores 5 --num
-executors 2 --conf spark.dynamicAllocation.enabled=false --conf spark.yarn
.appMasterEnv.STORAGE_PREFIX=[unique-bucket-name] --conf spark.yarn.
appMasterEnv.STORAGE_BASE=/data --conf spark.executorEnv.STORAGE_PREFIX=[
unique-bucket-name] --conf spark.executorEnv.STORAGE_BASE=/data
```

- **Action on Failure:** Continue

Se finaliza pulsando en **add** y esperando a que su estado cambie a **Completed**. Una vez terminada la ejecución, puede accederse al *bucket* S3 para revisar los archivos y registros generados.

Capítulo 4

Resultados y métricas

Este capítulo pretende servir de validación sobre el proyecto una vez ejecutado y obtenidos sus resultados. Consta de dos secciones: la primera, resultados de la ejecución, donde se mencionarán los resultados más relevantes mediante datos y gráficos. La segunda sección, métricas del desarrollo, expondrá diferentes resultados sobre eficiencia y rendimiento.

4.1. Resultados de la ejecución

Los resultados a mostrar están organizados respectivamente por los estudios sobre estaciones, distribución climática, sobre variables individuales y aspectos climáticos de interés. Cabe destacar que, si bien solo se mostrarán los gráficos más relevantes, puede consultarse el resto en la web de resultados (véase: [Web de resultados](#))

4.1.1. Estudios sobre estaciones meteorológicas

Los resultados aportados por este estudio han mostrado aspectos referentes a la distribución de las estaciones meteorológicas. Por un lado, se ha obtenido su número por provincia y su distribución por altitud, y por otro, su evolución temporal desde 1973 hasta 2024.

En cuanto a la distribución por provincias, hay un total de 853 estaciones repartidas en mayor concentración en las provincias insulares, con Santa Cruz de Tenerife en primer lugar con 65, seguida por Illes Balears con 43 y Las Palmas con 37. En cuanto a los territorios de la península, destacan Huesca con 30 y Cáceres junto a Murcia con 29, frente a La Rioja, Palencia y Segovia, que solo tienen 8. En cuanto a las provincias con menos estaciones, destacan Ceuta y Melilla, con solo 1. Finalmente, el resto de las provincias mantienen normalmente entre 10 y 20 estaciones.

Por lo que respecta a la evolución en número, arranca en el año 1973 con 121, manteniendo una tendencia exponencial hasta 2009 con 764, y a partir de este momento se adquiere una tendencia asintótica hasta el fin de la serie en 2024 con 853. Tal vez, el cambio de tendencia en 2009 se debió al comienzo de la crisis económica que afectó al país, viéndose recortado el presupuesto de la AEMET.

Para terminar, la distribución por altitud muestra que el 51,1 % de las estaciones se encuentran en el intervalo de 0 a 500 metros, le sigue el intervalo de 500 a 1000 metros con el 36 % y el de 1000 a 1500 metros con el 8,58 %. El resto de intervalos no llegan al 5 %. Estos datos muestran que las estaciones se concentran en las altitudes con mayor densidad de población, siendo las más cercanas al nivel del mar, frente a las grandes altitudes, donde hay una escasez de estaciones, posiblemente debido a la dificultad de su mantenimiento. Puede observarse en la [Figura 19](#) el gráfico del que se extraen estos datos.

Distribution of the station count by altitude
Year 2024

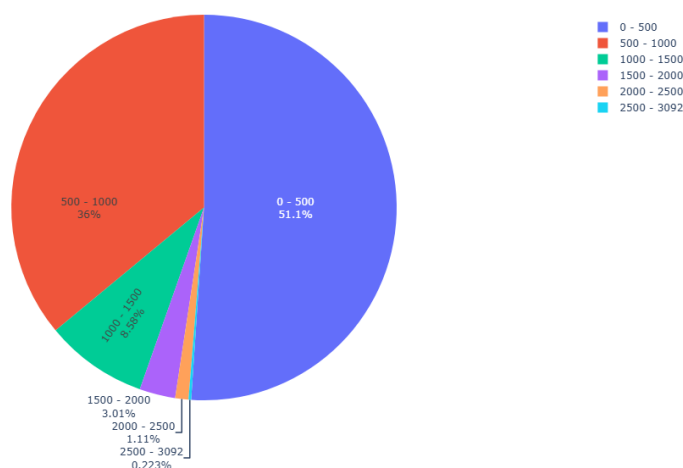


Figura 19: Gráfico de sectores circulares con la distribución de estaciones por intervalos de altitud

4.2. Estudios sobre distribución climática

Los resultados aportados por este estudio pretenden explicar los parámetros característicos de cada clima del territorio español. En capítulos anteriores se mencionaron todos los climas existentes y su distribución (véase: [Tabla 6](#)). Dado que hay una cantidad considerable de ellos, se explicará solo un climograma, pudiéndose consultar el resto en la web de resultados. Así pues, el gráfico escogido explica un clima cálido tipo Cfb, localizado en Bilbao, Bizkaia (véase: [Figura 20](#)).

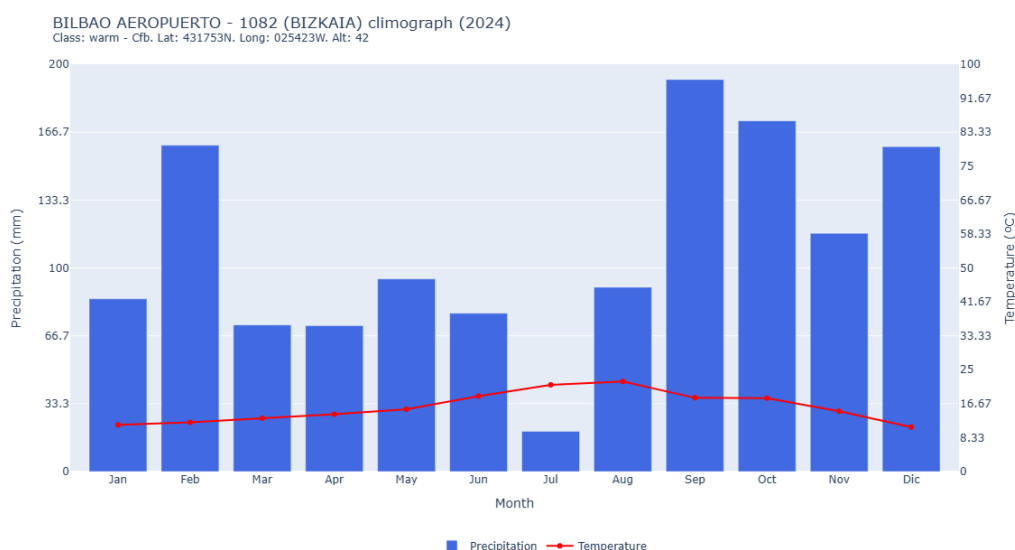


Figura 20: Gráfico tipo climograma de Bilbao, Bizkaia, durante el año 2024

Como se aprecia en la figura, en el eje izquierdo se representan las precipitaciones acumuladas en milímetros, en el derecho las temperaturas en grados Celsius y en el horizontal, los meses del año. Se muestran tendencias lluviosas durante los meses de otoño e invierno,

superando los 100mm, frente al resto del año, que se mantiene estable sin superar esta última cifra, con una bajada considerable durante el mes de Julio que no llega a superar los 20mm. En cuanto a las temperaturas, se mantienen estables durante todo el año en torno al intervalo de 10°C a 15°C, con un repunte en verano en torno a los 20°C.

4.3. Estudios sobre variables climáticas individuales

Los resultados de estos estudios buscan analizar las variables climáticas de la temperatura, precipitación, velocidad del viento, presión atmosférica, radiación solar y humedad ambiental y por cada una su *top 10* valores extremos, *top 5* incrementos, evolución temporal y distribución en el territorio. Dado que la cantidad de estudios es muy elevada, se mostrarán los más destacados, pudiéndose visualizar el resto en la web de resultados.

En cuanto al estudio *top 10* valores extremos, se ha decidido analizar la humedad relativa durante el año 2024. En la [Figura 21](#) puede apreciarse como el primer puesto lo gana Vimianzo, A Coruña, con una media de humedad relativa del 89,2 %, el segundo, La Mola, Maó, Illes Balears, con 88,6 %, y el tercero, Mazaricos, A Coruña, con 86,7 %. El resto de posiciones prácticamente se localizan en el norte peninsular con humedades siempre por encima del 80 %.

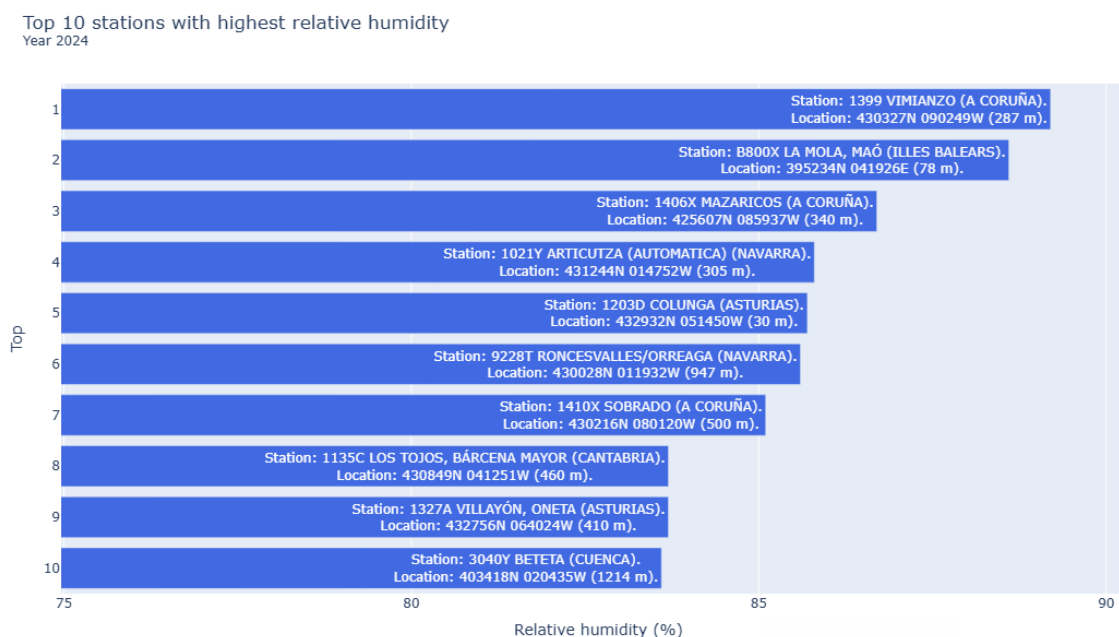


Figura 21: Gráfico de barras con los 10 lugares con mayor humedad durante 2024

Respecto al estudio de *top 5* incrementos, se han analizado las precipitaciones en su versión decremental durante el año 2024. Como se aprecia en la [Figura 22](#), la mayor pérdida de precipitaciones la sufrió Vigo, Pontevedra, con 451,5mm de precipitación menos que en años anteriores, seguido por Folgoso do Courel, Lugo, con 318,4mm menos. El resto de puestos apenas sobrepasan 200mm en pérdidas, localizados en Ceuta, Girona y León.

Top 5 stations with lowest precipitation increment
From the beginning of the records

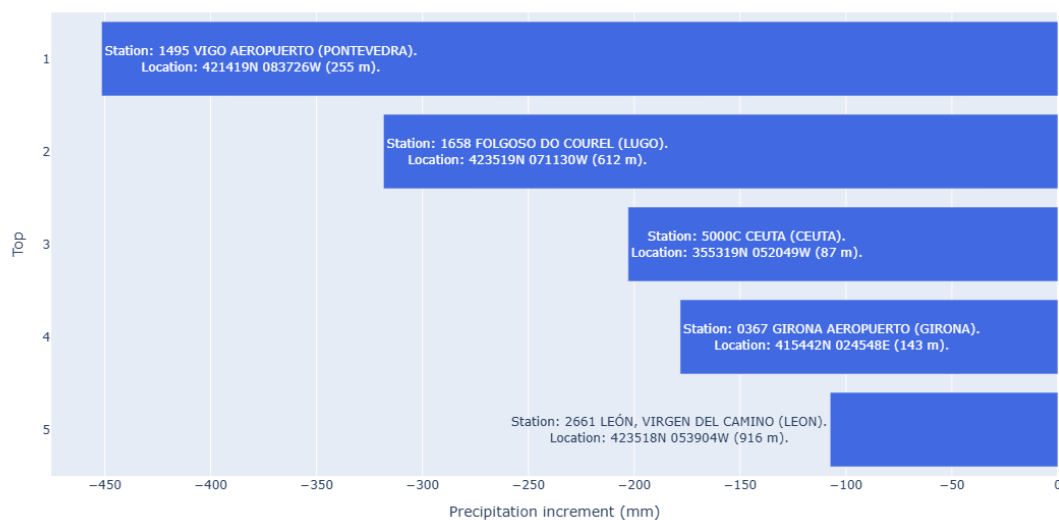


Figura 22: Gráfico de barras con los 5 lugares con mayores decrementos en sus precipitaciones durante 2024

En cuanto al estudio de la evolución temporal, se ha decidido estudiar la temperatura global desde el inicio de los registros en 1973 hasta 2024 en Sevilla. Como se aprecia en la [Figura 23](#), hay un gráfico lineal acompañado de una recta de regresión que muestra la tendencia de crecimiento al alza de esta variable climática, con una media anual de 17,8°C en 1973 frente a los 19,6°C en 2024, lo que corresponde a un incremento de la temperatura en 1,8°C en los últimos 50 años. Esta tendencia demuestra como el cambio climático está provocando un cambio en la temperatura global, y sobre todo en ciertas áreas de España, como es Andalucía, con importante riesgo de desertificación.

SEVILLA AEROPUERTO - 5783 (SEVILLA) temperature evolution (from the beginning of the records)
Lat: 372500N. Long: 055245W. Alt: 34

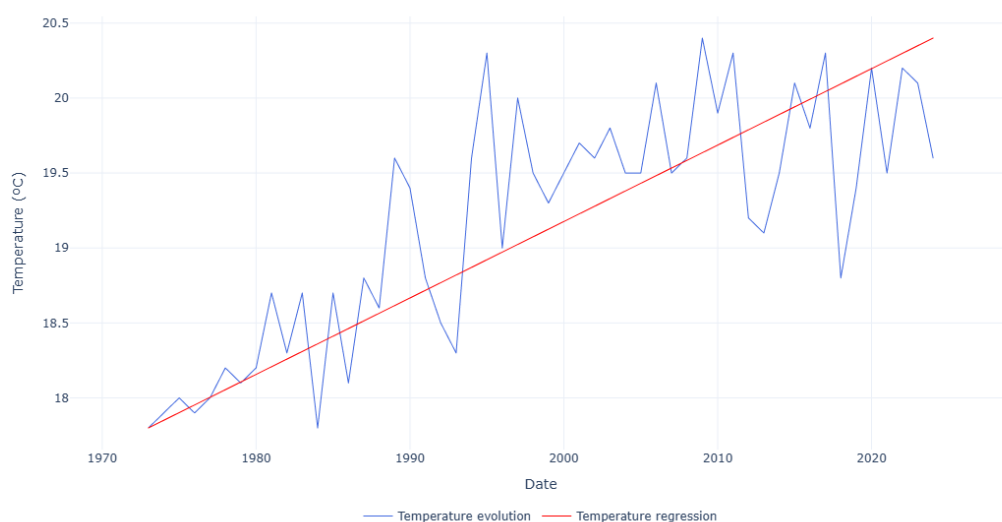


Figura 23: Gráfico lineal con recta de regresión que muestra la evolución de las temperaturas desde 1973 hasta 2024 en Sevilla

Finalmente, en cuanto al estudio que permite la representación de un mapa de calor, se ha escogido el mapa de las horas de máxima radiación solar en el territorio continental durante el año 2024. Como puede apreciarse en la [Figura 24](#), las horas de máxima radiación se alcanzan en la zona sur del territorio continental con más de 8 horas diarias frente al norte donde no llega a las 6 horas. El resto del territorio mantiene un valor medio entorno a las 6 o 7 horas. Este estudio muestra claramente la separación entre los climas cercanos al mar Cantábrico, mucho más fríos y lluviosos, frente a los del sur, con temperaturas mucho más elevadas y con menos humedad ambiental.

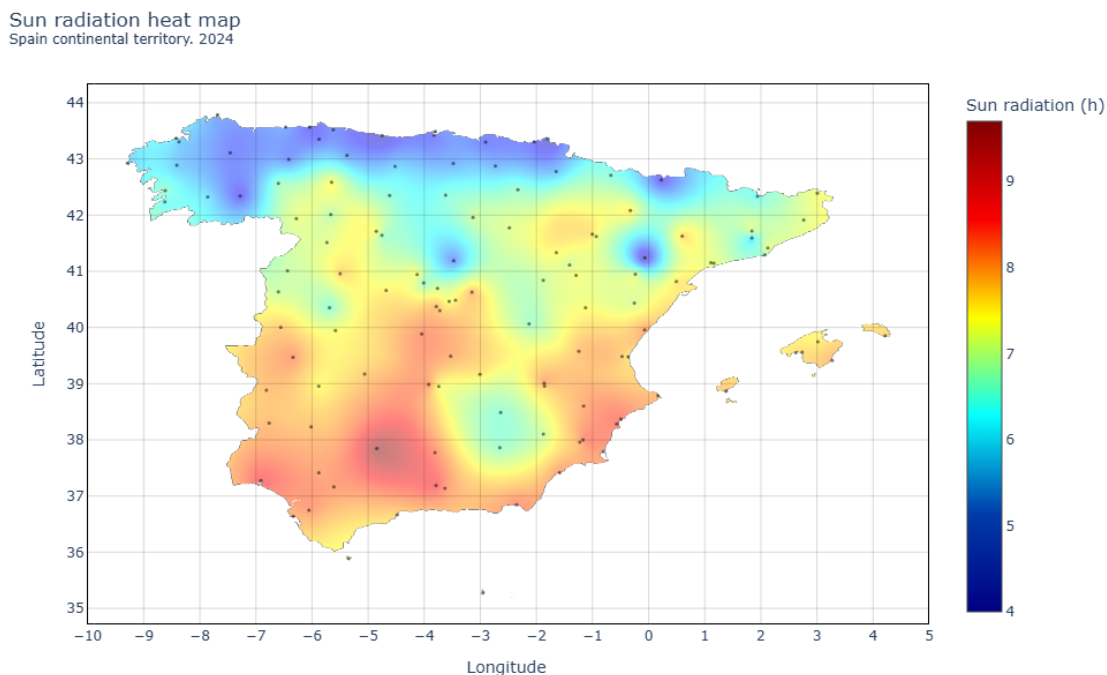


Figura 24: Gráfico con un mapa de calor de la distribución de la radiación solar a lo largo del territorio continental español durante 2024

4.4. Estudios climáticos de interés

Estos últimos estudios buscan datos interesantes sobre el clima, en concreto se ha escogido estudiar la relación entre precipitaciones y presión atmosférica, y por otro lado, el estudio *top 10* lugares con condiciones climáticas especiales.

En cuanto al estudio sobre la relación entre precipitaciones y presión atmosférica puede observarse en la [Figura 25](#) como contiene dos gráficos: en la parte superior, las precipitaciones, y en la parte inferior, la presión atmosférica, tomados dichos valores en Getafe, Madrid, durante el año 2024. El interés de este estudio era buscar algún tipo de correlación entre ambos valores, ya que cuando se produce un periodo caracterizado por lluvias y vientos, hay variaciones en la presión, pero no se muestra ningún patrón significativo, tan solo en algunos casos concretos no concluyentes. Por lo que respecta a la evolución anual de la presión, se aprecia que durante los primeros meses del año es más inestable, en verano se estabiliza y a principios de otoño comienza a mantener otro periodo de inestabilidad con tendencia creciente.

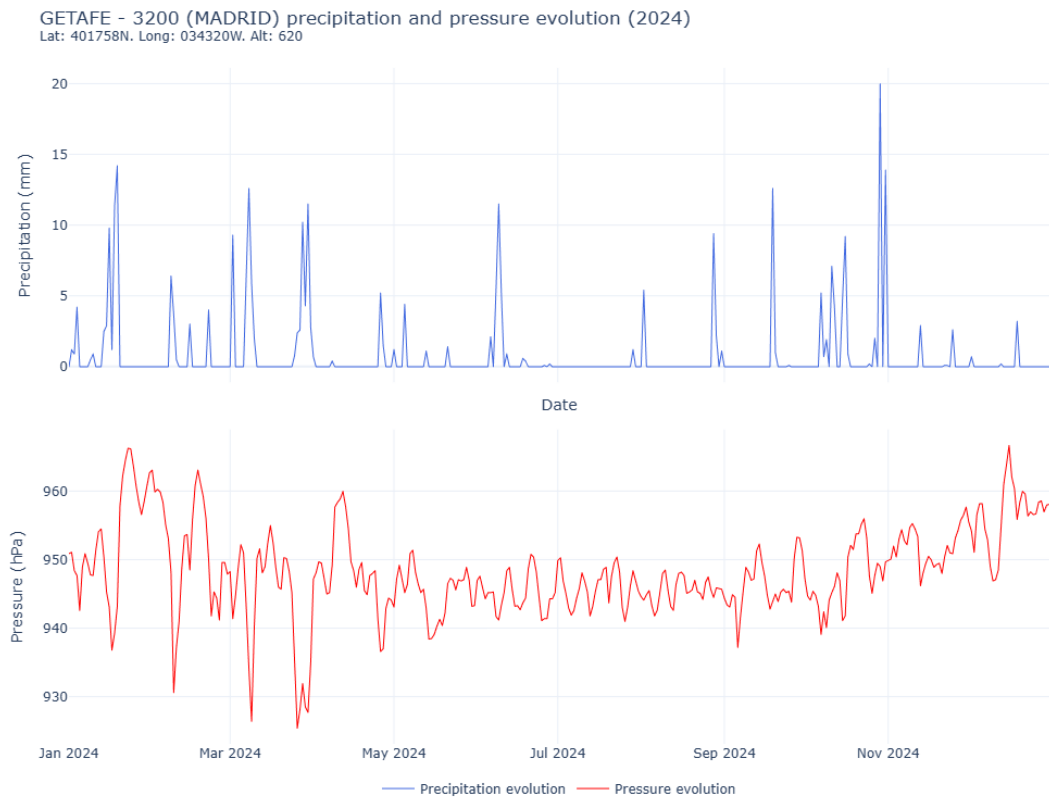


Figura 25: Gráfico lineal doble con la evolución de las precipitaciones y la presión atmosférica durante el año 2024 en Getafe, Madrid

En cuanto al estudio *top* 10 lugares con interés climático, se han escogido mostrar las provincias con mayor tendencia a padecer lluvias torrenciales durante el periodo comprendido entre los años 2014 y 2024. Así pues, como se aprecia en la [Figura 26](#), Valencia está en el primer puesto, lo que corresponde con la realidad al haberse producido muchos episodios como las DANAS. En los siguientes puestos se encuentran Illes Balears y A Coruña donde también es frecuente ver estos episodios climáticos.

Top 10 states with the highest incidence of torrential rains
Last decade

Top	State
1	VALENCIA
2	ILLES BALEARS
3	A CORUÑA
4	PONTEVEDRA
5	GIRONA
6	CACERES
7	CADIZ
8	NAVARRA
9	MALAGA
10	MURCIA

Figura 26: Gráfico en formato tabla con las 10 provincias más propensas a padecer lluvias torrenciales entre 2014 y 2024

4.5. Métricas del desarrollo

Respecto a las métricas, en primer lugar se expondrán los tiempos que ha tardado en desarrollarse el proyecto, así como otros aspectos de interés. Seguidamente, se mostrarán los tiempos de ejecución de cada una de las fases en su despliegue local. Finalmente, se mostrarán algunas métricas de interés para la ejecución de la fase de estudios con Spark tanto en local como en el *cluster* de AWS.

4.5.1. Tiempos de desarrollo y otras métricas

Para calcular los tiempos que ha demorado la realización del proyecto se han usado como guía las estadísticas del repositorio de GitHub donde se encuentra el código fuente. Así pues, se aporta la siguiente información aproximada (véase: [Tabla 7](#)).

Actividad	Tiempo de desarrollo (horas)	Porcentaje sobre el total
Extracción de datos	120	16,22 %
Estudios con Spark	250	33,78 %
Desarrollo de Auto-Plot	100	13,51 %
Cliente de Auto-Plot	50	6,76 %
Web para resultados	50	6,76 %
Despliegue	30	4,05 %
Documentación	140	18,92 %
TOTAL	740	100 %

Tabla 7: Tiempos que ha demorado el desarrollo para cada fase del proyecto

Tras exponer esta información, se aprecia que aproximadamente el proyecto completo ha demorado un total de 740 horas, donde la actividad que más peso ha tenido ha sido la de estudios con Spark debido a su gran complejidad de planificación y desarrollo individual de cada tipo de consulta. Le siguen otras actividades importantes, como la documentación del proyecto, incluyendo la redacción de esta memoria, el desarrollo de la extracción de datos o el desarrollo del sistema Auto-Plot. El resto de tareas muestran un peso menor. A modo de aclaración, para realizar este cálculo, se ha tomado un promedio de 4 a 5 horas de desarrollo por día.

Respecto a las otras métricas características del proyecto, cabe mencionar que sobre el repositorio de GitHub se han realizado 111 *commits* con un balance de 139.083 añadidos y 73.281 eliminaciones. En cuanto al volumen de datos procesados, consta de 8199 archivos con un peso total de 6,80 GB repartido de tal manera que los datos extraídos contienen 3,15 GB, los resultados de los estudios con Spark 2,34 MB y los gráficos finales 3,64 GB.

4.5.2. Tiempos de ejecución en local

Durante todas las fases del proyecto se han realizado mediciones de los tiempos de ejecución para dar a conocer una estadística que permita visibilizar el rendimiento del sistema, así pues, se aprecia en la [Tabla 8](#) cada uno de estos tiempos.

Proceso	Tiempo
Extracción de datos	01:19:11.784
Estudios con Spark	00:11:02.957
Generación de gráficos	00:18:54.198
TOTAL	01:49:08.939

Tabla 8: Tiempos de ejecución por proceso

En total, el proceso completo de ejecución dura cerca de 2 horas. Esta demora se debe a la fase de extracción de datos desde la API de AEMET, donde se deben realizar pausas frecuentes entre las peticiones para no obtener una expulsión, no obstante, muchas veces la API experimenta fallos, por lo que deben repetirse las peticiones varias veces hasta ser correctas, he ahí la demora que se experimenta de casi una 1,5 horas. En cuanto al proceso de estudios con Spark, es capaz de realizar todo el cómputo en poco más de 10 minutos gracias a la posibilidad de serializar y persistir la información utilizada en muchas labores de cómputo, lo que acelera los procesos de ejecución, sin embargo, utilizar esta estrategia solo ha sido posible en el caso concreto de este proyecto donde el total de datos no es tan masivo. Finalmente, el proceso de generación de gráficos demora cerca de 20 minutos, dado que el cliente tiene que pedir a Auto-Plot generar uno por cada estudio, donde destaca una mayor labor de cómputo en la creación de los gráficos tipo mapa de calor.

4.5.3. Métricas específicas de la fase de estudios con Spark

En esta última sección se pretende abordar la explicación de algunas estadísticas extraídas de la interfaz que proporciona Spark durante la ejecución, SparkUI, y así comprender mejor el rendimiento de la aplicación. En primer lugar, se expondrán una serie de métricas generales para dar una visión global del rendimiento del sistema, y por otro lado, se expondrán las métricas para una consulta en concreto. Todas las métricas han sido tomadas en una versión ejecutada en local y otra en el *cluster* EMR de AWS, lo que permite visualizar el rendimiento de ambos entornos.

Métricas generales

Puede apreciarse en la [Tabla 10](#) todas las métricas extraídas que a continuación serán descritas.

Entorno	Task Time	Complete Tasks	Shuffle Read	Shuffle Write
Local	11 min	29691	17.7 MiB	16.1 MiB
EMR	18 min	24243	23.3 MiB	21.9 MiB

Tabla 10: Comparación de métricas generales de ejecución entre el entorno local y EMR en AWS

- **Task Time:** mide el tiempo de ejecución de todas las *tasks* (acciones atómicas de Spark). En el caso de la ejecución en local, se ha empleado un tiempo de 11 minutos y en EMR 18 minutos. La diferencia entre ambos valores puede deberse a que no se haya empleado una configuración óptima para desarrollar la ejecución de Spark en EMR, tanto por parte del propio diseño de las consultas como por las características escogidas por el *cluster*.

- **Complete Task:** mide la cantidad de *tasks* realizadas. En local se obtiene un total de 29691 frente a 24243 en el *cluster*. Puede apreciarse una pequeña diferencia en este último debido posiblemente a la gestión de Spark sobre el sistema distribuido.
- **Shuffle Read y Shuffle Write:** mide la cantidad total de datos leídos o escritos en operaciones de *shuffle* (intercambio de datos entre diferentes nodos). Se leen, en local, 17,7 MiB, y en EMR, 23,3 MiB. En cuanto a escritura, en local, 16,1 MiB, y en EMR, 21,9 MiB. Estos valores muestran que no se están realizando operaciones pesadas que requieran un intercambio de datos entre los diferentes nodos, además, puede apreciarse un ligero aumento en EMR posiblemente debido al menor número de tareas que se realizan, lo que implicaría incluir más datos por cada operación de intercambio.

En cuanto a otras métricas interesantes que son comunes a ambos entornos destacan la realización de 5855 *Jobs* (tareas a realizar a partir de una consulta) y 5855 *Stages* (subtareas derivadas). Ambos números son iguales porque por cada *Job* solo se realiza 1 *Stage*. Esto se debe a que no se ha aprovechado totalmente la naturaleza de Spark para tratar datos masivos en el momento de diseñar las consultas, realizando multitud de ellas basadas en llamadas a una función que realiza la consulta en base a datos reducidos. Agrupando los datos y haciendo todo a la vez, se podría haber mejorado el rendimiento notablemente. Este aspecto también se ha visto reflejado en que se han leído 1,3 TiB en total, un valor natural para la naturaleza iterativa del planteamiento descrito en el sistema.

Métricas concretas en consultas

En este punto se explican las métricas específicas para una consulta en particular, la extracción de la media anual durante 2024 del valor de la radiación solar en cada estación del territorio continental. Así pues, desde la Spark UI se puede acceder a un resumen de la ejecución de la misma donde muestra sus estadísticas acompañadas del Directed Acyclic Graph (DAG), un grafo que muestra todo el plan de ejecución de Spark para realizar la consulta (puede consultarse en la [Figura 58](#)).

Cabe destacar que la consulta, a grandes rasgos, toma los datos sobre registros meteorológicos filtrando por solo los valores de radiación solar en el año 2024 en todas las provincias del territorio continental. A continuación calcula la media anual agrupando por el indicativo de la estación donde fueron extraídos. Finalmente se realiza la unión con los datos sobre estaciones para añadir valores como las coordenadas de localización. Finalmente se almacenan los datos en **Parquet**.

Entrando en detalle respecto a las métricas extraídas, la consulta consta de 3 *Jobs* diferentes y cada uno compuesto por un *Stage*, tanto en el entorno local como en EMR. El primero de todos carga los datos sobre registros, realiza los filtrados y comienza el cómputo de la media de los valores con agrupación; el segundo carga los datos sobre estaciones y realiza la unión; y el tercero, termina de realizar el cálculo de los datos medios y almacena el resultado. Esta explicación es importante para comprender los valores tomados de cada *Job* que se muestran a continuación en la [Tabla 11](#).

Entorno	Job ID	Duration	Tasks	Input	Output	Shuffle Read	Shuffle Write
Local	10	1 s	13	766.25 MiB	0.0 MiB	0.0 MiB	8.8 MiB
	11	59 ms	2	75 KiB	0.0 KiB	0.0 KiB	0.0 KiB
	12	0.3 s	1	0.0 KiB	10.2 KiB	8.8 KiB	0.0 KiB
EMR	22	0.3 s	10	766.25 MiB	0.0 MiB	0.0 MiB	9.9 MiB
	23	24 ms	2	75 KiB	0.0 KiB	0.0 KiB	0.0 KiB
	24	0.3 s	1	0.0 KiB	10.2 KiB	9.9 KiB	0.0 KiB

Tabla 11: Comparación de métricas de los Jobs en el entorno local y EMR en AWS

- **Job ID:** identificador unívoco que asigna Spark a cada *Job*.
- **Duration:** duración de cada *Job*. En local se obtienen tiempos para cada uno respectivamente de 1 segundo, 59 milisegundos y 0.3 segundos. En EMR, 0.3 segundos, 24 milisegundos y 0.3 segundos. Puede apreciarse una mejoría clara en el *cluster* para este caso.
- **Tasks:** cantidad de *Tasks* por *Stage*. En local, respectivamente hay 13, 2 y 1. En EMR, 10, 2, y 1. Se aprecia la tendencia ya vista de realizar menos *tasks* en EMR.
- **Input:** cantidad de datos leídos. Tanto en local como en EMR, respectivamente, 766,25 MiB, para los datos sobre registros meteorológicos; 75 MiB, para los datos sobre estaciones; y 0,0 MiB ya que en este *Job* solo se escribe en memoria.
- **Output:** cantidad de datos escritos. Tanto en local como en EMR no se escribe nada hasta el tercer *Job*, con 10.2 KiB.
- **Shuffle Read:** cantidad de datos leídos en operaciones de *shuffle*. Tanto en local como en EMR no se lee nada hasta el tercer *Job*, respectivamente, con 8.8 KiB y 9.9 KiB. Se aprecia la tendencia de que en EMR haya un ligero aumento del coste de las operaciones de *shuffle*, aunque en ambos sistemas no se traspasan grandes cantidades de datos. Es importante mencionar que solo se realiza lectura de intercambio en este *Job* porque en el primer *Job* hubo una operación que necesitó reorganización de datos y el segundo solo se encargaba de leer los datos sobre estaciones y unir, por lo que antes de escribir los datos finales, hay que cargar la porción del intercambio.
- **Shuffle Write:** cantidad de datos escritos en operaciones de *shuffle*. Tanto en local como en EMR solo se lee en el primer *Job*, respectivamente, 8.8 KiB y 9.9 KiB. Al igual que en la métrica anterior, se muestra la tendencia de bajo intercambio y ligero aumento en EMR. La razón por la que solo hay escritura en el primer *Job* se deduce de la explicación de la anterior métrica.

Se puede concluir que la ejecución de las consultas con Spark en EMR puede dar un mejor rendimiento en algunos casos, como el estudiado en la consulta expuesta, sin embargo, el tiempo global de ejecución no consigue llegar a ser menor que el aportado por la ejecución en local, lo que presupone que haya algún tipo de problemática de diseño o configuración que no se haya tenido en cuenta. Tal vez un diseño de consultas basado en la ejecución directa sobre grandes conjuntos de datos habría aportado mejor paralelización que podría haberse visto reflejada en una mejoría en el entorno distribuido de EMR. Por otro lado, la elección de unas mejores características para el *cluster* podrían haber aportado también mejorías. Finalmente es importante mencionar que si bien un entorno distribuido como EMR debería aportar un mayor rendimiento, no siempre puede ser posible si la cantidad de datos no supera cierto umbral, gastando entonces más recursos el sistema distribuido en su propia gestión que en el cómputo.

Capítulo 5

Conclusiones

Finalmente, se expondrán en este capítulo las conclusiones del proyecto con la descripción de todos los objetivos que han sido cumplidos, así como los que no, y de que manera podría mejorarse el sistema para lograrlos. También, en base a estos objetivos incompletos y otra serie de ideas, se expondrán los posibles trabajos a futuro que podrían realizarse con este proyecto.

5.1. Objetivos cumplidos

Como apoyo a este punto, pueden consultarse los objetivos marcados para la realización del proyecto en el capítulo de introducción (véase: [Objetivos](#)).

1. Se han logrado extraer completamente todos los datos deseados tanto desde AEMET como desde IFAPA, diseñando un sistema capaz de realizar peticiones a sus APIs. Posteriormente, la información ha sido unificada en el caso de IFAPA, adaptando su esquema al de AEMET, y finalmente, se ha modificado el formato de todos los archivos a **Parquet** para un mejor tratamiento de la información en las siguientes fases.
2. Completado con éxito con la realización de estudios usando diferentes consultas con Spark sobre la distribución de estaciones meteorológicas, distribución de los diferentes climas en el territorio español, estudios sobre variables climáticas individuales y finalmente, estudios de interés climático.
3. Completado con éxito con la creación de dos sistemas, un servidor con Python capaz de generar los gráficos con el uso de Plotly, y un cliente que envía peticiones a dicho servidor para crear los gráficos en base a los resultados de los estudios con Spark.
4. Completado con éxito, aunque podría ser mejorable. Se ha logrado desplegar y ejecutar todo el sistema en un entorno local con Docker, con resultado exitoso. Por lo que respecta a la ejecución del sistema que usa Spark en un *cluster* EMR de AWS, se han obtenido unos resultados de rendimiento similares a los del sistema local, por lo que no se aprecia gran mejoría en cuanto a la capacidad de cómputo.
5. Completado con éxito con la creación de un sitio web con Next.js donde visualizar los gráficos obtenidos fácilmente.

6. Completado con éxito con la redacción de esta memoria donde se explica con detalle el sistema completo, además, cuenta con múltiples diagramas para un mejor entendimiento.

5.2. Trabajos futuros

Para terminar, es importante mencionar que este proyecto se ha concebido desde la visión del *software* libre, por lo que se mantendrá abierto a futuros desarrolladores para complementar y mejorar el sistema ya creado. En base a esta premisa, algunos de los posibles trabajos futuros a realizar podrían ser los siguientes:

- Mejoría del sistema que usa Spark para aumentar la compatibilidad con EMR en AWS, así logrando ejecutar el proceso con una mayor eficiencia.
- En base a la anterior propuesta, el resto de fases están preparadas para ser ejecutadas en recursos de AWS ayudándose de S3 con el sistema de almacenamiento personalizado que se ha creado, por lo que sería una buena idea crear un proceso completo de ejecución desde la fase de extracción de datos a la de representación de gráficos que funcione en AWS.
- Existen otras APIs de meteorología con datos muy interesantes que podrían complementar a los que ya se estudian en este proyecto. Su añadido daría más valor al producto, aportando mucha más información meteorológica. Además, esta información podría extraerse en tiempo real y, con el uso de la web de resultados, podría visualizarse.
- Otra mejora interesante podría ser el añadido la IA dentro del proyecto, de modo que, en base a la información proporcionada por el sistema y por medio de un modelo de lenguaje, se pudiesen realizar consultas desde un apartado en la web de resultados.

Apéndice A

Información complementaria

A.1. Tablas de requisitos

A.1.1. Requisitos funcionales de la fase de extracción de datos

Extracción de datos desde AEMET

Identificador	Descripción
RF-01	El sistema debe extraer metadatos de estaciones meteorológicas desde la API de datos abiertos de AEMET.
RF-02	El sistema debe extraer datos de estaciones meteorológicas desde la API de datos abiertos de AEMET.
RF-03	El sistema debe extraer metadatos de registros meteorológicos diarios desde la API de datos abiertos de AEMET.
RF-04	El sistema debe extraer registros meteorológicos diarios desde la API de datos abiertos de AEMET.
RF-05	La extracción de registros meteorológicos diarios se realizará desde el 1 de enero de 1973 hasta el 31 de diciembre de 2024.
RF-06	La extracción de registros meteorológicos diarios se hará de tal manera que se ejecuten peticiones con intervalos de quince días.
RF-07	En la extracción de registros meteorológicos diarios se almacena la fecha de la última petición exitosa.
RF-08	Si el sistema se reinicia, la extracción de registros meteorológicos diarios se reanuda desde quince días en adelante a la última fecha guardada.

Tabla 12: Listado de requisitos funcionales del sistema de extracción de datos de AEMET

Extracción de datos desde IFAPA

Identificador	Descripción
RF-09	El sistema debe extraer metadatos de la estación meteorológica del desierto de Tabernas desde la API de datos abiertos de IFAPA.
RF-10	El sistema debe datos de la estación meteorológica del desierto de Tabernas desde la API de datos abiertos de IFAPA.
RF-11	El sistema debe extraer metadatos de registros meteorológicos del desierto de Tabernas desde la API de datos abiertos de IFAPA.
RF-12	El sistema debe extraer datos de registros meteorológicos del desierto de Tabernas desde la API de datos abiertos de IFAPA.
RF-13	El sistema debe extraer registros meteorológicos correspondientes al año 2024.

Tabla 13: Listado de requisitos funcionales del sistema de extracción de datos de IFAPA

Unificación de los datos de IFAPA al esquema de AEMET

Identificador	Descripción
RF-14	El sistema debe convertir los datos de estaciones y registros meteorológicos diarios de IFAPA al esquema definido por AEMET.

Tabla 14: Listado de requisitos funcionales del sistema de conversión de los datos de IFAPA al esquema definido por AEMET

Transformación del formato de los datos extraídos

Identificador	Descripción
RF-15	El sistema debe agrupar los registros meteorológicos diarios de AEMET en bloques de datos (chunks) para reducir el número de archivos generados.
RF-16	El sistema debe convertir los datos de estaciones meteorológicas de AEMET a formato Parquet .
RF-17	El sistema debe convertir los registros meteorológicos diarios de AEMET a formato Parquet .
RF-18	El sistema debe convertir los datos de estaciones meteorológicas de IFAPA a formato Parquet .
RF-19	El sistema debe convertir los registros meteorológicos diarios de IFAPA a formato Parquet .

Tabla 15: Listado de requisitos funcionales del sistema de transformación del formato de los datos extraídos

A.1.2. Requisitos funcionales de la fase de estudios con los datos

Estudios sobre estaciones meteorológicas

Identificador	Descripción
RF-20	El sistema debe permitir realizar la consulta sobre la distribución de las estaciones meteorológicas por altitud según los intervalos: 0-500, 500-1000, 1000-1500, 1500-2000, 2000-2500, 2500-3092.
RF-21	El sistema debe permitir realizar la consulta sobre la distribución de las estaciones meteorológicas por provincia según las 52 provincias que integran el territorio español.
RF-22	El sistema debe permitir realizar la consulta sobre la evolución temporal del número de estaciones meteorológicas desde el comienzo de los registros.

Tabla 16: Listado de requisitos funcionales de los estudios sobre estaciones meteorológicas

Estudios sobre distribución climática

Identificador	Descripción
RF-23	Deben buscarse estaciones candidatas para cada clima descrito en la Tabla 6 .
RF-24	El sistema debe permitir realizar la consulta que obtenga las precipitaciones acumuladas y la media de las temperaturas por cada mes del año en cada estación candidata de climas áridos, templados y fríos.

Tabla 17: Listado de requisitos funcionales de los estudios de distribución climática

Estudios sobre variables climáticas individuales

Identificador	Descripción
RF-25	Los estudios se realizarán sobre las variables de temperatura, precipitación, velocidad del viento, presión atmosférica, radiación solar y humedad ambiental.
RF-26	El sistema debe permitir realizar la consulta que obtenga los diez lugares con mayores o menores valores extremos de la variable estudiada, durante el año 2024, durante la última década y desde el comienzo de los registros.

Campo	Descripción
RF-27	El sistema debe permitir realizar la consulta que obtenga los cinco lugares con mayores o menores incrementos de la variable estudiada desde el comienzo de los registros.
RF-28	El sistema debe permitir realizar la consulta que obtenga la evolución temporal del valor de la variable, diariamente durante el año 2024 y anualmente desde el comienzo de los registros.
RF-29	El sistema debe permitir realizar la consulta que obtenga los registros meteorológicos diarios de todas las estaciones durante el año 2024.

Tabla 18: Listado de requisitos funcionales de los estudios sobre variables climáticas individuales

Estudios meteorológicos interesantes

Identificador	Descripción
RF-30	El sistema debe permitir realizar la consulta que obtenga la evolución del valor de las variables climáticas de precipitación y presión atmosférica a la vez, diariamente durante el año 2024 y anualmente desde el comienzo de los registros.
RF-31	El sistema debe permitir consultas sobre lugares con las condiciones especiales para la energía solar, eólica y agricultura, y los lugares más propensos a lluvias torrenciales, tormentas, olas de calor, heladas, incendios y sequías.

Tabla 19: Listado de requisitos funcionales de los estudios meteorológicos interesantes

A.1.3. Requisitos funcionales de la fase de representación de resultados

Requisitos generales

Identificador	Descripción
RF-32	Se debe diseñar un sistema que provea los gráficos que sean pedidos mediante una serie de instrucciones de cómo crearlo.
RF-33	Se debe crear un cliente para el sistema que crea los gráficos.
RF-34	Se necesita crear gráficos de barras, lineales univariable con y sin regresión, lineales bivariante, sectores circulares, tablas, climogramas y mapas de calor.

Tabla 20: Listado de requisitos funcionales generales para la representación de resultados

Resultados de los estudios sobre estaciones meteorológicas

Identificador	Descripción
RF-35	El sistema debe permitir la creación del gráfico de la distribución del número de estaciones meteorológicas por altitud mediante un gráfico de sectores circulares.
RF-36	El sistema debe permitir la creación del gráfico de distribución del número de estaciones meteorológicas por provincia mediante un gráfico de tabla.
RF-37	El sistema debe permitir la creación del gráfico de evolución del número de estaciones meteorológicas mediante un gráfico lineal.

Tabla 21: Listado de requisitos funcionales de los resultados de los estudios sobre estaciones meteorológicas

Resultados de los estudios sobre distribución climática

Identificador	Descripción
RF-38	El sistema debe permitir la creación de los gráficos de los climogramas por cada estación representativa de cada tipo de clima.

Tabla 22: Listado de requisitos funcionales de los estudios de distribución climática

Resultados de los estudios sobre variables climáticas individuales

Identificador	Descripción
RF-39	El sistema debe permitir la creación de los gráficos sobre valores máximos y mínimos de la variable mediante un gráfico de barras.
RF-40	El sistema debe permitir la creación de los gráficos sobre mayores o menores incrementos de la variable mediante un gráfico de barras.
RF-41	El sistema debe permitir la creación del gráfico de evolución del valor de la variable durante el año 2024 con un gráfico lineal simple y la evolución desde el comienzo de los registros con un gráfico lineal con una recta de regresión.
RF-42	El sistema debe permitir la creación del gráfico de todos los valores meteorológicos diarios de las estaciones durante el año 2024 mediante un mapa de calor dependiendo de la localización de las estaciones pudiendo ser del territorio español continental o de las Islas Canarias.

Tabla 23: Listado de requisitos funcionales de los estudios sobre variables climáticas individuales

Resultados de los estudios meteorológicos interesantes

Identificador	Descripción
RF-43	El sistema debe permitir la creación de los gráficos de evolución temporal de las variables de precipitación y presión atmosférica tanto en 2024 como desde el comienzo de los registros mediante un gráfico doble lineal.
RF-44	El sistema debe permitir la creación de los gráficos que indiquen los lugares donde es más frecuente que se den condiciones climáticas especiales mediante un gráfico de tabla.

Tabla 24: Listado de requisitos funcionales de los estudios meteorológicos interesantes

A.1.4. Requisitos funcionales sobre aspectos comunes en todas las fases

Valores constantes

Identificador	Descripción
RF-45	Todas las fases del proyecto deben tener asociados ficheros con constantes.
RF-46	El sistema debe permitir que al iniciar una fase, se carguen todas las constantes en una representación interna que permita su uso.

Tabla 25: Listado de requisitos funcionales sobre valores constantes

Almacenamiento

Identificador	Descripción
RF-47	El sistema debe permitir el almacenamiento de todos los datos producidos.
RF-48	El sistema debe permitir la lectura y escritura de archivos JSON, Parquet, y lectura de archivos de configuración.
RF-49	El sistema debe permitir que las acciones de lectura y escritura se puedan desarrollar en un entorno local o remoto, sirviendo una interfaz de trabajo común independiente del entorno usado.

Tabla 26: Listado de requisitos funcionales de almacenamiento

Funciones auxiliares del sistema

Identificador	Descripción
RF-50	El sistema debe permitir la realización de peticiones HTTP, concretamente GET y POST para poder enviar y recibir información al exterior.
RF-51	El sistema debe contener una interfaz de tratamiento para JSON que habilite herramientas que permitan realizar acciones de modificación sobre los mismos.
RF-52	El sistema debe contener una interfaz de tratamiento del tiempo que permita realizar acciones como saltos entre fechas o contabilizar tiempo transcurrido.
RF-53	El sistema debe contener una interfaz de tratamiento de la estética de los mensajes que se imprimen por consola que permita realizar acciones como cambio de color o añadir tabulaciones fácilmente.

Tabla 27: Listado de requisitos funcionales sobre funciones auxiliares del sistema

A.1.5. Requisitos funcionales sobre aspectos complementarios

Identificador	Descripción
RF-54	Se creará una web interactiva que permita visualizar todos los resultados.

Tabla 28: Listado de requisitos funcionales sobre aspectos complementarios

A.1.6. Requisitos no funcionales

Arquitectura

Identificador	Descripción
RNF-01	El sistema principal debe estar desarrollado en Scala siguiendo los estándares de la programación funcional.
RNF-02	La fase de estudios sobre los datos se deberá desarrollar con Apache Spark.
RNF-03	El sistema proveedor de gráficos estará basado en un servidor web en Python, generando los mismos mediante la librería Plotly.

Tabla 29: Listado de requisitos no funcionales de arquitectura

Fiabilidad

Identificador	Descripción
RNF-04	El sistema cargará los ficheros de configuración al inicio de cada fase manteniendo una representación interna.
RNF-05	El sistema debe gestionar errores temporales en APIs de obtención de datos sin finalizar la ejecución.
RNF-06	Ante un fallo fatal, el sistema finalizará la ejecución de forma controlada mostrando la causa del error.
RNF-07	El sistema debe garantizar la integridad de los datos ante cualquier fallo en la ejecución de una fase.
RNF-08	El almacenamiento de los datos del sistema estará integrado por carpetas y subcarpetas dependiendo de la fase a la que pertenezcan y subcarpetas dependiendo del ámbito de los mismos.

Tabla 30: Listado de requisitos no funcionales de fiabilidad

Despliegue

Identificador	Descripción
RNF-09	Todas las fases del proyecto se desplegarán con un sistema de contenedores con Docker y se realizarán pruebas que permitan conocer sus tiempos de ejecución.
RNF-10	Se alojarán los archivos que contienen los gráficos generados en un sistema capaz de proveerlos remotamente.
RNF-11	La fase de estudios sobre los datos se desplegará en un <i>cluster</i> EMR de AWS y se estudiará su diferencia frente a la ejecución en local.

Tabla 31: Listado de requisitos no funcionales de despliegue

Documentación

Identificador	Descripción
RNF-12	Se documentará todo el proceso de creación del proyecto mediante una memoria complementada con múltiples diagramas de componentes y ejecución.

Tabla 32: Listado de requisitos no funcionales de documentación

A.2. Jerarquía de ficheros y directorios del sistema principal

Se explica la jerarquía de directorios y ficheros. Así pues, la raíz del proyecto muestra la siguiente estructura:

```
big-data-core
├── modules
├── project
├── .gitignore
└── build.sbt
```

Figura 27: Estructura de directorios en la raíz del sistema principal

Como puede apreciarse en la [Figura 27](#):

- **modules**: código fuente del proyecto.
- **project**: información útil para la compilación, como la versión de SBT o *plug-ins*.
- **.gitignore**: información sobre los ficheros y directorios a excluir por Git.
- **build.sbt**: estructura del proyecto, dependencias y directrices de compilación.

A continuación, se explica la jerarquía del directorio **modules**. En su raíz se encuentra el siguiente contenido (véase: [Figura 28](#))

```
modules
├── data-extraction
├── plot-generation
├── spark
└── utils
```

Figura 28: Estructura de directorios del módulo modules

En cuanto a la descripción de su contenido:

- **data-extraction**: código fuente para el proyecto de extracción de datos.
- **plot-generation**: código fuente para el proyecto cliente solicitador de gráficos.
- **spark**: código fuente para el proyecto de estudios sobre los datos.
- **utils**: librería de utilería.

Cabe destacar que se muestran los tres proyectos independientes y la librería de utilería como parte de un mismo sistema de archivos. Esto se debe a que SBT permite crear composiciones de proyectos, facilitando las labores de desarrollo, gestión de dependencias y compilación.

A continuación, se describe el contenido de cada uno de los directorios mencionados. Sin embargo, a modo de simplificación, se mostrarán primero los directorios comunes que contienen. Así pues, se encuentra el siguiente contenido. (véase: [Figura 29](#))

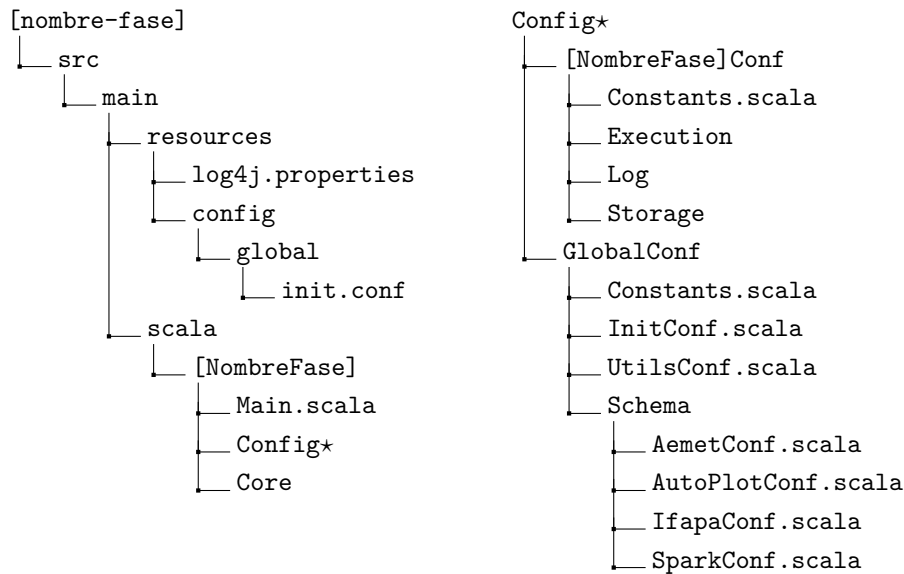


Figura 29: Estructura de directorios y archivos general para todas las fases

Como puede apreciarse en la figura:

- **resources**: archivos de configuración necesarios para el arranque de la aplicación. El más destacado es `init.conf`, que integra las constantes con nombres de variables de entorno necesarias para la ejecución.
- **scala**: archivos `.scala` con el código fuente:
 - **Main.scala**: punto de entrada del código.
 - **Config**: archivos `.scala` con la representación interna de las constantes.
 - ▷ **[NombreFase]Conf**: configuración, con varios subdirectorios para la representación interna de las constantes. En base a su cometido, los ficheros estarán en un directorio u otro. Por ejemplo, `execution`, para constantes de ejecución; `storage`, para rutas o nombres de archivos; `log`, para mensajes por consola...etc. En cada directorio los archivos contarán con un nombre característico indicando a qué fichero del código fuente se destina. Finalmente, el archivo `Constants.scala` lee las constantes y las carga en la estructura interna valiéndose de las representaciones internas pudiendo ser accedidas desde el código fuente.
 - ▷ **GlobalConf**: similar al anterior, pero destinado a constantes globales para todas las fases. `InitConf.scala` representa las constantes que hay en `resources`; `UtilsConf.scala`, representa constantes de utilería, y por último, `Schema`, representa constantes relacionadas con esquemas de nombres, normalmente claves de JSON. Finalmente `Constants.scala` realiza la misma acción que en el directorio anterior.
 - **Core**: código fuente principal del subproyecto.

Se prosigue describiendo la jerarquía de archivos del código fuente principal de cada subproyecto, contenido en **Core**. En primer lugar, el subproyecto `data-extraction`, referente a la extracción de datos, cuenta con la siguiente jerarquía (véase: [Figura 30](#))

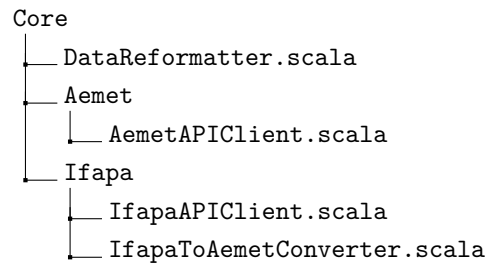


Figura 30: Estructura de directorios y archivos del código fuente principal de la fase de extracción de datos

Como puede apreciarse en la figura:

- **DataReformatter.scala**: conversión de JSON a Parquet para Spark.
- **Aemet**: extracción de datos de la API de AEMET.
 - **AemetAPIClient.scala**: código fuente.
- **Ifapa**: extracción de datos de la API de IFAPA.
 - **IfapaAPIClient.scala**: código fuente.
 - **IfapaToAemetConverter.scala**: transformación del esquema de IFAPA al de AEMET.

Seguidamente, se describe la jerarquía de archivos para el subproyecto de Spark, es decir, el destinado a los estudios sobre los datos (véase: [Figura 31](#))

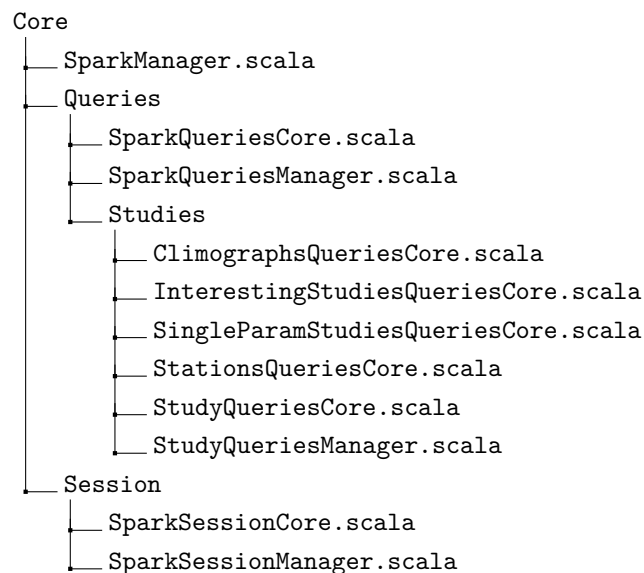


Figura 31: Estructura de directorios y archivos del código fuente principal de la fase de estudios sobre los datos

Como puede apreciarse en la figura:

- **SparkManager.scala**: inicializa todos los recursos necesarios para la ejecución de los estudios con Spark y orquesta el funcionamiento del sistema.
- **Queries**: consultas sobre los datos.

- **SparkQueriesCore.scala** y **SparkQueriesManager.scala**: definición de cada una de las consultas a realizar.
- **Studies**: código para la petición consultas.
 - ▷ **ClimographsQueriesCore.scala**: estudios sobre distribución climática.
 - ▷ **InterestingStudiesQueriesCore.scala**: estudios sobre información climática interesante.
 - ▷ **SingleParamStudiesQueriesCore.scala**: estudios climáticos sobre variables individuales.
 - ▷ **StationsQueriesCore.scala**: estudios sobre estaciones meteorológicas.
 - ▷ **StudyQueriesCore.scala** y **StudyQueriesManager.scala**: código general para la petición de consultas y almacenaje de la información obtenida.
- **Session**: contiene los ficheros **SparkSessionCore.scala** y **SparkSessionManager.scala** con el código fuente referente a la sesión de Spark y la carga de datos al sistema.

En cuanto al subproyecto `plot-generation`, centrado en la petición de generar gráficos con los resultados, cuenta con un único archivo, `PlotGenerator.scala` encargado en realizar las peticiones al servidor. (véase: [Figura 32](#))

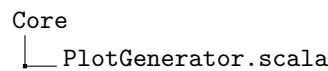


Figura 32: Estructura de directorios y archivos del código fuente principal de la fase de representación de resultados

Finalmente, la librería de utilería `utils` cuenta con la siguiente jerarquía (véase: [Figura 33](#))

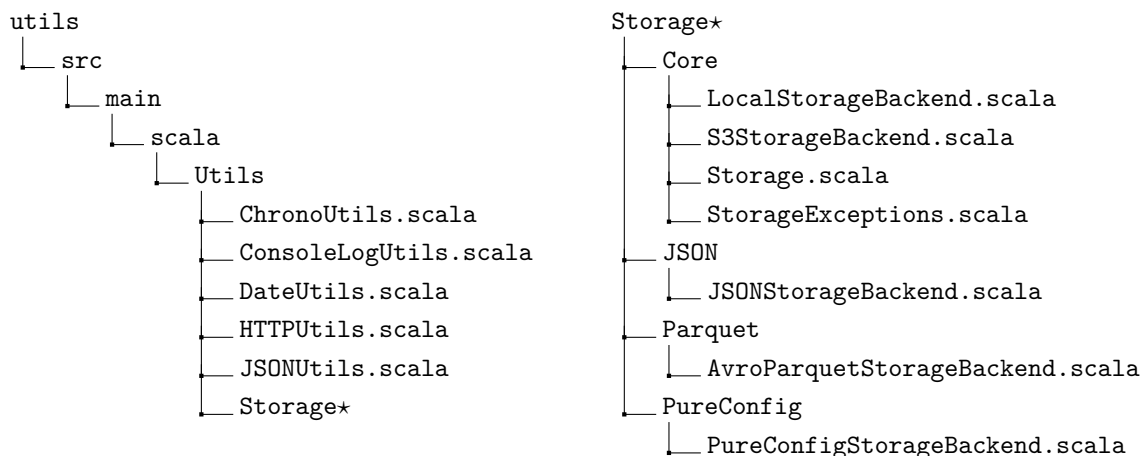


Figura 33: Estructura de directorios y archivos del módulo `utils` (dividida a partir de `Storage`)

Como puede apreciarse en la figura:

- **ChronoUtils.scala**: funciones sobre el manejo del tiempo.

- **ConsoleLogUtils.scala**: manejo de los estilos del texto por consola.
- **DateUtils.scala**: operaciones sobre fechas.
- **HTTPUtils.scala**: cliente HTTP para realizar peticiones en Internet.
- **JSONUtils.scala**: acciones de modificación sobre archivos JSON.
- **Storage**: código fuente del sistema de almacenamiento común entre AWS S3 y local.
 - **Core**: ficheros principales del sistema.
 - ▷ **LocalStorageBackend.scala**: funciones del sistema local.
 - ▷ **S3StorageBackend.scala**: funciones del sistema AWS S3.
 - ▷ **Storage.scala**: asume sobre que sistema operar.
 - ▷ **StorageExceptions.scala**: excepciones del sistema.
 - **JSON**: cuenta con el fichero **JSONStorageBackend.scala** que permite el trabajo con archivos JSON.
 - **Parquet**: cuenta con el fichero **AvroParquetStorageBackend.scala** que permite el trabajo con archivos Parquet.
 - **PureConfig**: cuenta con el fichero **PureConfigStorageBackend.scala** que permite el trabajo con archivos de configuración.

A.3. Visión global del sistema principal

Se describe la arquitectura global del sistema, mostrando sus diferentes interconexiones, mediante la interpretación del diagrama de componentes presente en la [Figura 38](#).

Visualizando el diagrama se muestran cuatro módulos principales dentro de un módulo llamado **main app**, estos son: **DataExtraction**, **SparkApp**, **Utils** y **PlotGenerator**. En el exterior se encuentran: **AEMET API**, **IFAPA API**, **Storage system** y **AUTO-PLLOT**.

En cuanto al módulo **DataExtraction**, referente al proceso de extracción de datos, integra dentro de sí mismo varios paquetes. **Core**, por su parte, integra los elementos principales. También destacan los paquetes **Config** y **Resources**. **Core** se conecta a las constantes con **Config** y este a su vez a **Resources** para obtener el archivo de configuraciones inicial. También hay conexiones con las APIs de extracción de datos **AEMET API** e **IFAPA API**.

El módulo **SparkApp**, referente al proceso de estudios, contiene varios elementos, a parte de los ya explicados anteriormente referentes a constantes. **Core** contiene los paquetes **Session** y **Queries**, y este a su vez, **Studies**. **SparkManager** tiene control sobre el resto de *managers* o orquestadores que sirven como punto de entrada para el resto de paquetes. Se conectan con los *cores* o núcleos de cada sección donde están las funciones principales. El principal de ellos es **SparkSessionCore**, del cual dependen el resto. Este se comunica directamente con la sesión de Spark y accede a sus funciones. Seguidamente, **SparkQueriesCore** contiene las funciones que definen el plan de las consultas a realizar. Continuando con **StudyQueriesCore**, define un método genérico para la petición de ejecución de una consulta y almacenaje de la información obtenida. Este será usado por el resto de núcleos destinados a cada estudio a realizar, es decir **StationsQueriesCore**, **ClimographQueriesCore**, **SingleParamStudiesQueriesCore** y **InterestingStudiesQueriesCore**.

En cuanto al módulo **Utils**, referente a la librería de utilería, destaca la presencia del paquete **Storage**, destinado al almacenamiento y otros componentes secundarios exteriores

que fueron explicados en la jerarquía de archivos. Por su parte el módulo para el almacenamiento cuenta con varios paquetes en su interior: **Core** con las funcionalidades principales del sistema y otros paquetes que hacen uso de este, **JSON**, **Parquet** y **PureConfig**. Cada uno de estos paquetes secundarios cuentan con un *back-end* que permite usar el sistema de almacenamiento para un tipo de archivo determinado. El paquete principal, por su parte, cuenta con el componente **Storage**, siendo el elemento de entrada al sistema que aplica la elección de usar el sistema local o el sistema AWS S3, así como otras operaciones secundarias. Este elemento hace uso de **LocalStorageBackend**, para almacenamiento local y **S3StorageBackend**, para almacenamiento remoto en AWS S3. Finalmente cada elemento hace uso de una definición de excepciones creadas para la resistencia ante posibles errores en **StorageExceptions**.

Finalmente, el módulo **PlotGenerator**, referente a el proceso de representación de los resultados, integra una estructura general similar a **DataExtraction** y **SparkApp**, pero mucho más simplificada. El sistema de constantes actúa de la misma manera y su núcleo solo cuenta con el elemento **PlotGenerator**, capaz de comunicarse con el sistema proveedor de gráficos, pasándole la información necesaria para su creación.

Por lo que concierne a los elementos externos, las APIs de AEMET e IFAPA se conectan con los procesos de extracción de datos. En cuanto al sistema de almacenamiento es el único punto de acceso al sistema de archivos, encontrando una carpeta común donde será almacenada toda la información llamada */data*. Finalmente, **PlotGenerator** se conecta un servidor externo llamado **AUTO-PLOT** que internamente tiene una estructura compleja independiente, con un sistema para recibir peticiones, interpretarlas y construir gráficos.

A.4. Detalles sobre consultas en Spark

getStationInfoById(): el plan de acción comienza con el DF de estaciones y se filtra con **filter()** con la condición de que la columna **indicativo** sea igual al identificador. Después, se seleccionan todas las filas con **select()** y se aplica un alias a cada una con **alias()**.

Véase este funcionamiento en el diagrama de la [Figura 44](#). La siguiente [Tabla 33](#) muestra un posible resultado de la consulta.

station_id	station_name	state	lat_dms	long_dms	altitude
B954	IBIZA, AEROPUERTO	ILLES BALEARS	385235N	012304E	6

Tabla 33: Ejemplo del resultado de la consulta **getStationInfoById()**

getStationCountByColumnInLapse(): el plan de acción se desarrolla partiendo del DF de registros meteorológicos, ya que aporta la fecha de observación y se aplica **filter()** con el siguiente criterio: si se recibe fecha de fin de intervalo, la columna **fecha** debe estar entre esta y la fecha de inicio (se hace uso de la función **between()**). Para el caso contrario, se filtra con el año de la fecha de inicio. A continuación, se agrupa con **groupBy()** sobre la columna del criterio de búsqueda y se aplica agregación con **agg()** y **countDistinct()**, contando los identificadores, lo que añade la columna **count**. Finalmente, se hace **select()** de la columna del criterio de búsqueda y **count**, ordenando sobre la primera con **orderBy()**.

Véase este funcionamiento en el diagrama de la [Figura 45](#). La siguiente [Tabla 34](#) muestra un posible resultado de la consulta.

state	count
A CORUÑA	18
ALBACETE	12
...	...

Tabla 34: Ejemplo del resultado de la consulta `getStationCountByColumnInLapse()`

getStationsCountByParamIntervalsInALapse(): el plan de acción comienza con el DF de registros meteorológicos aplicando el mismo filtrado de fechas de la consulta anterior y seleccionando el identificador y la columna de búsqueda. Se añade un índice a cada entrada de la lista de tuplas con `zipWithIndex()` y se transforma con `map()`, buscando si los límites de los intervalos contienen infinitos, reemplazando por los valores máximos o mínimos de la columna de búsqueda, o en su defecto, por su valor propio. Al resultado se aplica un filtrado para escoger si los límites deben ser incluidos o no y con `agg()` que añaden las columnas `min_value` con el límite inferior y `max_value` con el superior, y usando `countDistinct()` por cada identificador, añade la columna `count` con la cuenta de estaciones por intervalo.

Finalmente, la función `map()` inicial ha generado resultados individuales por cada elemento de la lista de intervalos, por lo que se aplica `union()` para generar un único resultado al que se le termina aplicando `orderBy()` con criterio de ordenación la columna `min_value`.

Véase este funcionamiento en el diagrama de la [Figura 46](#). La siguiente [Tabla 35](#) muestra un posible resultado de la consulta.

min_value	max_value	count
0	500	458
500	1000	323
...	...	...

Tabla 35: Ejemplo del resultado de la consulta `getStationsCountByParamIntervalsInALapse()`

getStationMonthlyAvgTempAndSumPrecInAYear(): el plan de acción comienza con el DF de registros meteorológicos al que se le aplica `filter()` con las condiciones de que las columnas `t_med` y `prec` no contengan nulos, que `indicativo` sea el identificador de la estación y que `fecha` sea el año de observación requerido. Posteriormente, se agrupa en base al mes extraído de `fecha` mediante `month()`, generando una nueva columna `month` con el mes del registro. Después, se aplica agregación con `agg()` de la columna `t_med` con la función `avg()` para obtener la media de las temperaturas, y de la columna `prec` con la función `sum()` para obtener la suma de las precipitaciones. Finalmente, se ordena en base a la columna `month`.

Véase este funcionamiento en el diagrama de la [Figura 47](#). La siguiente [Tabla 36](#) muestra un posible resultado de la consulta.

month	temp_monthly_avg	prec_monthly_sum
1	20.8	5.8
2	20.7	3.9
...	...	...

Tabla 36: Ejemplo del resultado de la consulta `getStationMonthlyAvgTempAndSumPrecInAYear()`

getClimateParamInALapseById(): cabe destacar que los parámetros climáticos se pasan a la función como una lista de tuplas con el nombre de la columna y la temporalidad de la observación. También se pasa una lista de la misma longitud que la anterior, donde cada elemento es un texto que indica el método de agregación que tiene cada parámetro climático.

El plan de acción comienza con `foldLeft()` sobre la lista de tuplas. Como valor inicial, se toma el resultado de filtrar el DF de registros meteorológicos con la estrategia de consultas anteriores sobre intervalos de fechas. En cuanto a la función de plegado, aplica al resultado acumulado `filter()` con la condición de que el valor de la columna con el nombre del primer elemento de la tupla. Una vez completado, se vuelve a aplicar `filter()` con la condición de que la columna `indicativo` sea el identificador especificado. Finalmente, se selecciona del resultado la totalidad de los nombres de columna de la lista de tuplas y se ordena en base a la columna `date`.

Véase este funcionamiento en el diagrama de la [Figura 48](#). La siguiente [Tabla 37](#) muestra un posible resultado de la consulta.

date	sun_rad_daily_avg
19723	4.1
19724	0.0
...	...

Tabla 37: Ejemplo del resultado de la consulta `getClimateParamInALapseById()`

getTopNClimateParamInALapse(): el plan de acción comienza con el DF de registros meteorológicos, aplicando la técnica de filtrado ya vista sobre las fechas, y también se filtra para que la columna de la variable estudiada no contenga nulos. Seguidamente, se aplica `groupBy()` por la columna `indicativo` y se realiza agregación sobre la columna de la variable generando una nueva columna con el resultado. La agregación se aplica en base a un `String` que se pasa como parámetro, y es proporcionado a un mapa que devuelve la función a aplicar para la columna en cuestión. Al resultado se aplica `join()` con el criterio `inner` y clave para el vínculo, `indicativo`, añadiendo todas las columnas extra del DF de estaciones a cada fila del resultado. Se prosigue realizando `select()` de las columnas de estaciones. Para terminar, se ordena sobre la columna de la variable con el criterio `desc` si se desean los valores extremos más altos o `asc` si se desean los más bajos y también, se limita el resultado a los N deseados y se aplica con `withColumn()` la función `monotonically_increasing_id() + 1` para numerar los resultados, creando una nueva columna llamada `top`.

Véase este funcionamiento en el diagrama de la [Figura 49](#). La siguiente [Tabla 38](#) muestra un posible resultado de la consulta.

station_id	station_name	state	press_avg	lat_dms	long_dms	altitude	top
5511	PRADOLLANO, PARQUE NAC...	GRANADA	706.3	370352N	032217W	3092	1
C430E	IZAÑA	SANTA CRUZ DE TENERIFE	772.4	281830N	162959W	2369	2
...	...	...	...	...	...	...	...

Tabla 38: Ejemplo del resultado de la consulta `getTopNClimateParamInALapse()`

getTopNClimateConditionsInALapse(): cabe destacar que a la función se le pasan los parámetros climáticos siguiendo la misma estrategia ya vista de la lista de tuplas con nombre de columna y extremos inferior superior del intervalo.

El plan de acción comienza con el DF de registros meteorológicos al que se aplica el filtrado de fechas ya visto y otro usando `map()` para evitar valores nulos y que se encuentren entre los límites del intervalo. Se obtiene una lista de sentencias condicionales a la que con `reduce()` que aplica la conjunción a todas ellas resultando en una única expresión booleana de filtrado. Con este resultado se aplican dos posibles ramificaciones condicionales, o bien se ha elegido agrupar por provincias, tomando como candidatos para el recuento de cada una todas las estaciones que pertenezcan a la misma, o bien, se eligen todas las estaciones individualmente. En cuanto a la primera opción, al resultado del filtro se aplica `groupBy()` por la columna `provincia` y se aplica agregación con mediante la función `count()` que aporta en una nueva columna `days_with_conds` la cuenta de los registros. Finalmente se ordena y se numera cada registro con la misma estrategia que la consulta anterior y al resultado se aplica `select()` de las columnas `provincia` y `top`. En cuanto a la segunda opción de la ramificación, al resultado del filtro se realizan acciones similares, tan solo teniendo en cuenta que en la agrupación se aplica sobre la columna `indicativo` y finalmente en la selección se escogen las columnas típicas sobre estaciones y `top`. Para terminar todo el proceso, después de la ramificación condicional, se ordena por la columna `top` en sentido ascendente y se limita el número de registros a N.

Véase este funcionamiento en el diagrama de la [Figura 50](#). La siguiente [Tabla 39](#) muestra un posible resultado de la consulta.

state	top
LAS PALMAS	1
SANTA CRUZ DE TENERIFE	2
...	...

Tabla 39: Ejemplo del resultado de la consulta `getTopNClimateConditionsInALapse()`

getClimateYearlyGroupById(): cabe destacar que a la función se le pasan los parámetros climáticos y las funciones de agregación a aplicar por cada uno mediante la estrategia de listas de tuplas vista en la función `getTopNClimateParamInALapse()`.

El plan de acción comienza con la lista de tuplas con parámetros climáticos a la que se aplica la estrategia de plegado mediante `foldLeft()` ya vista, tan solo teniendo en cuenta que al valor de inicialización ahora se va a añadir una columna nueva llamada `year` con el año del registro y filtrado por la columna `indicativo` igual al identificador. Al resultado se aplica `map()` a la que se le suma con `zip()` la lista con los nombres de las funciones de agregación. Así, la transformación toma el nombre de la función y aplica *pattern matching* para conseguir la función en cuestión aplicada sobre la columna cuyo

nombre es el parámetro climático de la iteración actual. Finalmente, se aplica agrupación por la columna `year` y se aplican las agregaciones de la lista conseguida y el resultado se ordena por `year`.

Véase este funcionamiento en el diagrama de la [Figura 51](#). La siguiente [Tabla 40](#) muestra un posible resultado de la consulta.

year	prec_yearly_sum
1973	464.0
1974	548.7
...	...

Tabla 40: Ejemplo del resultado de la consulta `getClimateYearlyGroupById()`

`getAllStationsByStatesAvgClimateParamInALapse()`: el plan de acción comienza con el DF de registros meteorológicos al que se realiza el filtrado por fechas, otro para que se tomen registros que estén presentes en la lista de provincias (si esta está vacía se toman todos), y un último para evitar valores del parámetro nulos. Con el resultado agrupa por la columna `indicativo` y se aplica agregación con la técnica ya descrita para la elección de funciones de agregación. A continuación se aplica `join()` sobre el DF de estaciones con el criterio `inner` y como clave `indicativo` y finalmente se seleccionan las columnas de estaciones y `latDec` y `longDec` con los valores de latitud y longitud, respectivamente, en formato decimal.

Véase este funcionamiento en el diagrama de la [Figura 52](#). La siguiente [Tabla 41](#) muestra un posible resultado de la consulta.

station_id	station_name	state	temp_avg	lat_dms	long_dms	lat_dec	long_dec	altitude
C929I	HIERRO AEROPUERTO	SANTA CRUZ DE TENERIFE	22.1	274908N	175320W	27.818889	-17.888889	32
C925F	SAN ANDRÉS, VALVERDE	SANTA CRUZ DE TENERIFE	15.5	274608N	175737W	27.768889	-17.960278	1070
...	...	...	...	...	...	...	...	...

Tabla 41: Ejemplo del resultado de la consulta `getAllStationsByStatesAvgClimateParamInALapse()`

`getStationClimateParamRegressionModelInALapse()`: el plan de acción comienza con el DF de estaciones al que se le aplica un filtrado por `indicativo`, por fecha con la estrategia ya vista, y evitando nulos del parámetro buscado. Después se aplica `year()` sobre la columna `fecha` y con `withColumn()` se crea una nueva columna `year` con el año del registro. El resultado se agrupa por la nueva columna y se aplica agregación con la estrategia de elección de métodos ya vista. Finalmente se selecciona la columna `year` ahora con alias `x` y la columna con el parámetro tras la agregación ahora con alias `y`. Estos resultados permiten definir un modelo de regresión lineal, calculado como muestra la [Ecuación A.4.1 \[111\]](#):

$$\begin{aligned}\hat{y} &= \beta_0 + \beta_1 x \\ \beta_1 &= \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \\ \beta_0 &= \bar{y} - \beta_1 \bar{x}\end{aligned}\tag{A.4.1}$$

Para informatizar el modelo, se calculan las medias de las columnas **x** e **y** aplicando agregación con **avg()**, generando dos nuevas columnas, **mx** y **my** de donde se extrae directamente su valor con la función **first()**. Después, se calculan β_1 y β_0 , aplicando las funciones **withColumn()** y de agregación correspondientes para realizar el cómputo usando los valores anteriores. Finalmente con estos resultados se genera un nuevo DF con las columnas **station_id**, **beta_1** y **beta_0**.

Véase este funcionamiento en el diagrama de la [Figura 53](#). La siguiente [Tabla 42](#) muestra un posible resultado de la consulta.

station_id	beta_1	beta_0
1505	-0.034253217079059005	71.02325421442191

Tabla 42: Ejemplo del resultado de la consulta
`getStationClimateParamRegressionModelInALapse()`

getTopNClimateParamIncrementInAYearLapse(): el plan de acción parte de un DF con los datos sobre regresión lineal logrados tras la ejecución de la consulta anterior. A este se le aplica un filtrado por la columna **indicativo** par que solo se contengan las estaciones del conjunto dado y a continuación con **withColumn()** se calculan los incrementos con los valores estimados por el modelo de regresión aplicado a las ambas fechas del intervalo, así obteniendo una nueva columna **inc**. A continuación se aplica **join()** del DF de registros meteorológicos con algunas modificaciones, con el criterio **inner** y como clave **indicativo**. Las modificaciones del DF se basan en aplicar filtros para obtener solo los indicativos presentes en el conjunto de estaciones, no contener valores nulos y conseguir que las fechas estén entre los valores del intervalo temporal. Posteriormente se realizan operaciones de agrupación y agregación sobre el parámetro climático para conseguir su valor medio anual. Sobre este resultado se aplica un nuevo **join()** de mismas características pero sobre el DF de estaciones, añadiendo una nueva columna **inc_prec** con el porcentaje de incremento respecto al valor medio anual. Se seleccionan las columnas **inc**, **inc_prec**, el parámetro medio anual y las referentes a estaciones y se realizan las operaciones de ordenado, limitación de registros y numeración típicas de las consultas tipo *top* ya vistas.

Véase este funcionamiento en el diagrama de la [Figura 54](#). La siguiente [Tabla 43](#) muestra un posible resultado de la consulta.

inc	inc_prec	global_prec_yearly_avg	station_id	station_name	state	lat_dms	long_dms	altitude	top
130.6	35.1	371.7	2374X	CARRIÓN DE LOS CONDES	PALENCIA	422103N	043702W	830	1
103.9	22.7	457.7	5270B	JAÉN	JAEN	374639N	034833W	580	2
...	...	...	...	...	...	...	...	...	...

Tabla 43: Ejemplo del resultado de la consulta
`getTopNClimateParamIncrementInAYearLapse()`

A.5. Detalles sobre el sistema de almacenamiento

En cuanto al núcleo del sistema, comienza en `Storage.scala`, que define la clase `Storage` con las funciones de interfaz común, capaces de discernir que sistema usar al ser llamadas. Este aspecto se gestiona en base a un prefijo que contienen las rutas de los archivos a usar. Todas las rutas son relativas al directorio `/data` y dentro de la misma se encuentran los subdirectorios por cada fase del proyecto con sus receptivos archivos. A la ruta relativa se le agrega al inicio un prefijo necesario que se pasa como parámetro al iniciar el sistema de archivos, que puede ser de dos tipos: un prefijo característico de un sistema local como `C:/directorio1/directorio2` en Windows o `/directorio1/directorio2/` en Unix, o `s3://` para AWS S3. Si el prefijo de almacenamiento es este último, el sistema redirige el flujo a las funciones específicas para trabajar sobre el sistema remoto y en caso contrario, a las del sistema local. Este comportamiento se logra gracias a `selectS3orLocal()`. Cabe destacar que en caso de detectar una ruta de AWS S3, esta sigue una estructura tal que `s3://[BUCKET]/[KEY]`, siendo importante extraer el componente `BUCKET`, es decir, la dirección del sistema de almacenamiento de S3; y el componente `KEY`, es decir, la dirección del recurso dentro del *bucket*. Este comportamiento se logra gracias a `parseS3Path()`.

A continuación se muestran las funciones que aporta el sistema y la acción que permiten ejecutar.

- `read()`: lectura de un archivo.
- `readDirectoryRecursive()`: lectura recursiva de un directorio.
- `write()`: escritura de un archivo.
- `copy()`: copiado de un archivo.
- `deleteLocalDirectoryRecursive()`: borrado recursivo de un directorio local.

En cuanto a su implementación específica para el sistema local y remoto, solo ha sido necesario definir las funciones `read()`, `write()` y `readDirectoryRecursive()` ya que el resto tienen una implementación basada en la combinación de las anteriores.

El sistema local está en el fichero `LocalStorageBackend.scala` y se basa en el uso de funciones de Java para el tratamiento de ficheros usando el paquete `java.nio.file` [13]. En lo referente al sistema remoto, todas las operaciones han de realizarse descargando o enviando datos, de modo que se tienen en cuenta directorios y archivos temporales para este fin. Este aspecto ha de ser tenido en cuenta también por el sistema local con el propósito de definir una interfaz de trabajo común. Así pues, se describen a continuación cada una de sus funciones.

- `read()`: esta función lee un archivo y devuelve su dirección. Obtiene la dirección con `Paths.get()`, y si es una dirección correcta, realiza un copiado del fichero a un directorio temporal con `Files.copy()`, devolviendo la dirección de este último.
- `readDirectoryRecursive()`: esta función lee un directorio y recursivamente todos sus subdirectorios y archivos, devolviendo su dirección. Primero obtiene la dirección raíz, y mediante `Files.walk()` itera y va generando directorios temporales y archivos en base a si deben ser leídos o no por una especificación de directorios a incluir pasada como parámetro.

- **write()**: esta función añade un fichero. Se recibe una dirección temporal donde se encuentra escrito el fichero y se crean las carpetas necesarias dentro del sistema con `Files.createDirectories()` y se realiza un copiado del mismo, devolviendo esta nueva dirección.

El sistema remoto para AWS S3 está integrado en el fichero `S3StorageBackend.scala`, con un funcionamiento similar al local, sin embargo, se debe tener en cuenta que los archivos se encuentran en un servidor remoto y hay que acceder a él para realizar las operaciones de lectura y escritura. Esta labor la realiza la librería AWS SDK, presentando una interfaz de trabajo sobre S3 con diferentes funciones. [86, 14]

En primer lugar, para conectar con S3 debe usarse un cliente, AWS SDK ya proporciona uno con la función `S3Client.create()`, sin embargo, antes de usar el sistema real de AWS, se ha decidido usar un sistema de pruebas que recrea S3 en local, pudiendo realizar pruebas sin gastar los recursos limitados de AWS. La herramienta usada es Localstack, que permite abrir un servidor usando Docker con iguales características que S3, admitiendo peticiones programadas con AWS SDK, pero como el servidor es completamente diferente al oficial de AWS, se necesita definir un cliente personalizado. De este modo se ha creado la función `createDummyClient()` que se basa en la función `S3Client.builder()` que permite definir la región a usar, el *endpoint* a donde llegaran las peticiones y los credenciales de AWS. Respecto a estos valores a definir, la región usada será `US_EAST_1`, el *endpoint*, aquel que abre el contenedor de Docker, normalmente, `localhost:4566`, y los credenciales son triviales dado que Localstack no realiza validación. [63]

En cuanto a las funciones del sistema remoto, mantienen la misma idea del sistema local, pero ahora con la necesidad de acceder a AWS S3. Así pues, describen a continuación:

- **read()**: descarga el contenido de una *key* con `GetObjectRequest.builder()` y construye una petición que es enviada al servidor devolviendo un objeto como respuesta que es pasado a `client.getObject()` junto con la dirección temporal donde se descargará el archivo.
- **readDirectoryRecursive()**: trabaja de manera similar a su versión en local, realizando un filtrado sobre los directorios a incluir. Su particularidad con el trabajo sobre S3 es la necesidad de recorrer los archivos con una función recursiva que se ha diseñado llamada `listAndDownload()` que internamente realiza llamadas a `ListObjectsV2Request.builder()`. Esta función devuelve *tokens* por cada iteración con fragmentos del contenido que deberá ser acumulado durante las iteraciones en un directorio temporal mediante el uso de lecturas con la función `read()`. [10]
- **write()**: se basa en el uso de `PutObjectRequest.builder()` para pedir la subida de un archivo a una *key*. Una vez obtenida la respuesta, se procesa la subida con `client.putObject()` junto al fichero que se encuentra en un directorio temporal.

Una vez explicado el núcleo del sistema, hay otros que lo implementan para poder trabajar sobre archivos concretos.

La implementación para trabajar con JSON, está en el fichero `JSONStorageBackend.scala` que permite la lectura, escritura y copiado de los mismos. El proceso de lectura se realiza utilizando la función `read()` de la interfaz de almacenamiento sobre una dirección y esta devuelve otra a la que se le aplica `Files.readString()` para leer el contenido que posteriormente es interpretado por uJSON en un objeto interno. En cuanto al proceso

de escritura, se crea un fichero temporal donde se escribe el contenido de un objeto `JSON` con `ujson.write()` para obtener el `String` y `Files.writeString()` para escribir dicho contenido que posteriormente se traslada al sistema de almacenamiento con la función `write()`. En cuanto al copiado de archivos, simplemente se basa en el uso de la función `copy()` del sistema de almacenamiento. Como conclusión, puede apreciarse en la [Figura 55](#) un diagrama de ejecución de este sistema.

El código para trabajar con `Parquet` está en `AvroParquetStorageBackend.scala` y permite la escritura de archivos haciendo uso de la librería `Avro`. La idea es crear una función que defina un escritor de `Parquet` y que pueda ser usada de manera similar al sistema del módulo `Using` de `Scala`. Así se define la función `withAvroParquetWriter()` que permite la recepción de una función como parámetro en la que trabaja el escritor. En cuanto a su código, primero se establece una dirección temporal de escritura y se define una abstracción de esta usando la función de `Hadoop` `HadoopOutputFile.fromPath()`, que devuelve un objeto necesario para la creación del escritor. Este a su vez se define con la función `AvroParquetWriter.builder()` a la que se le pasa el esquema de `Parquet` y el objeto anterior. Es importante mencionar que la creación del escritor se encuentra dentro de una función `Using.resource()` que permite a `Scala` un manejo automático de los recursos definidos en su ámbito, pudiendo ejecutar código que hace uso de los mismos mediante una referencia. Así, se ejecuta la función que fue pasada como parámetro, que define el comportamiento sobre la creación de un `Parquet`, haciendo uso del escritor. Finalmente, todo el contenido escrito en un directorio temporal se guarda en el sistema de archivos con la función `write()`. [\[42\]](#)

El último tipo de fichero a tratar son los archivos de configuración de la librería `PureConfig`, usada para definir archivos con extensión `.conf` con las constantes del proyecto, almacenados en `/data/conf`. El acceso a estos archivos se realiza desde el código del fichero `PureConfigStorageBackend.scala`. Dentro se define la función `readInternalConfig()`, pero para comprender su funcionamiento, primero es necesario explicar que el sistema de constantes, tiene una representación interna de cada uno de los ficheros de configuración, donde por medio de clases, se define una abstracción que permite acceder a los valores y cada una queda integrada en un archivo general llamado `constants.scala` donde hay abstracciones para almacenamiento, *log* o ejecución. Este archivo necesita definir un sistema de *parsing* para trasladar la información de los ficheros `.conf` a las representaciones, entando en juego la función a explicar.

Cada clase de representación conforma el tipo de dato con el que `readInternalConfig()` trabajará. En cuanto a su código, primero se define un objeto de configuración con el que trabaja el *parser* llamado `TypesafeConfig` bajo dos alternativas. Es importante mencionar para su entendimiento que los archivos `constants.scala` siempre acceden en primer lugar a un fichero de constantes presente en el propio código fuente llamado `init.conf` con el valor de una variables de entorno con la la dirección base a usar por el sistema de almacenamiento. Una vez obtenida se accede a `/data/conf` para cargar el resto de configuraciones. Así, las alternativas de la definición de la configuración radican entre si se debe leer de los archivos presentes en el código fuente o fuera del mismo.

La primera opción genera el objeto mediante `ConfigFactory.parseFile()` leyendo el contenido de un objeto `java.io.File` y la segunda opción genera el objeto mediante `ConfigFactory.parseReader()`, leyendo el contenido de un `bufferedReader` dirigido al archivo `.conf` en el directorio `resources` del código fuente. Finalmente, este objeto de configuración se pasa como parámetro a `ConfigSource.fromConfig()` que ejecuta el

parser y permite cargar el contenido de los archivos `.conf` a cada una de las clases de representación interna. Cabe destacar que en el proceso de *parsing* se utiliza un lector automático para cada tipo de dato de los atributos de las clases representativas, sin embargo, para el caso del tipo `Double` se ha diseñado un lector personalizado para interpretar `Strings` referentes a un valor infinito como puede ser `'+inf'` o `'∞'`. [25, 61]

A.6. Jerarquía de ficheros y directorios de Auto-Plot

La arquitectura del *software* que puede apreciarse en esta jerarquía se compone por modelos de datos, servicios y controladores, interconectados entre si, siguiendo un estilo de programación en Python. En la [Figura 34](#) puede apreciarse la jerarquía de la raíz del proyecto:

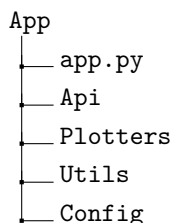


Figura 34: Estructura de directorios en la raíz del sistema Auto-Plot

En cuanto a su contenido:

- **app.py**: punto de entrada a la aplicación.
- **Api**: controladores, modelos de datos y DTOs.
- **Plotters**: servicios encargados de la generación de gráficos.
- **Utils**: módulo de utilería.
- **Config**: contiene los ficheros `constants.py` con constantes y `enumerations.py` con enumeraciones.

A continuación, se explica cada uno de los directorios, primero, el directorio `Api`. Cabe destacar que, para su explicación, se han simplificado las figuras de la jerarquía ya que todas tienen una estructura similar. En todos los directorios se encuentran tantos ficheros como tipos de gráficos que el sistema es capaz de componer. Estos se exponen a continuación con su nomenclatura para el sistema y el gráfico al que hacen referencia:

- **bar**: gráfico de barras.
- **pie**: gráfico de sectores circulares.
- **table**: gráfico en forma tabla.
- **linear**: gráfico lineal.
- **linear_regression**: gráfico lineal con recta de regresión.
- **double_linear**: gráfico lineal doble.
- **climograph**: gráfico tipo climograma.
- **heat_map**: gráfico tipo mapa de calor.

Bajo esta premisa la jerarquía de ficheros se muestra como sigue en la [Figura 36](#):

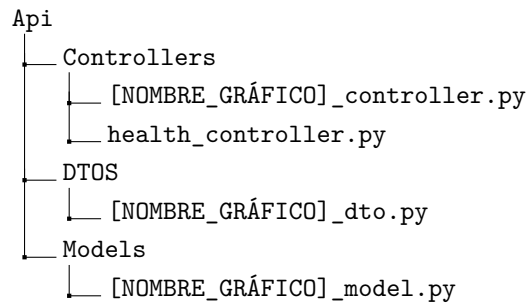


Figura 35: Estructura de directorios de Api en Auto-Plot

En cuanto a su descripción:

- **Controllers:** controladores del sistema. Además, contiene a `health_controller.py` que permite comprobar la salud del sistema.
- **DTOS:** definición de los esquemas para los cuerpos de las peticiones a los *endpoints*, que a su vez permite definir un DTO.
- **Models:** modelo de datos para la información de cada gráfico.

A continuación, se muestra la estructura del directorio **Plotters** en la [Figura 36](#).

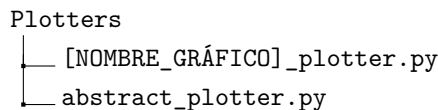


Figura 36: Estructura de directorios de Plotters en Auto-Plot

Contiene los servicios que hacen uso de la librería Plotly para generar gráficos y también destaca el fichero `abstract_plotter.py`, que busca definir varios métodos abstractos que serán implementados por los servicios graficadores.

En cuanto al directorio **Utils**, se aprecia su jerarquía en la [Figura 37](#):

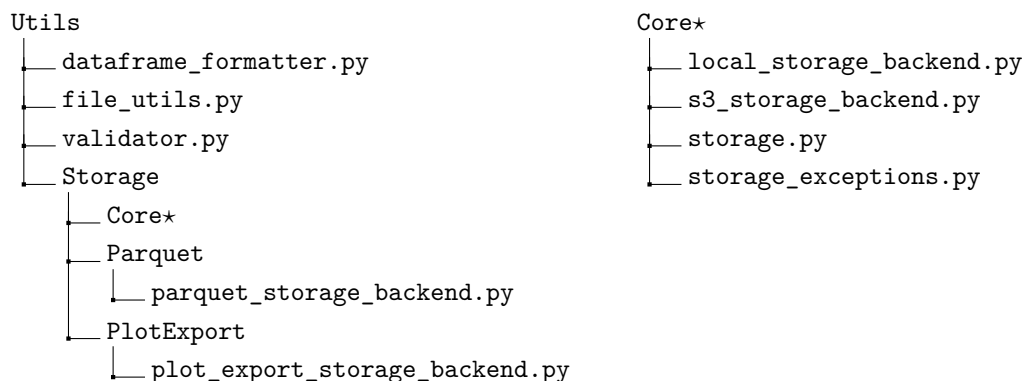


Figura 37: Estructura de directorios de Utils en Auto-Plot

En cuanto a su descripción:

- **dataframe_formatter.py:** utilidad para permitir formatear algunas columnas de los DF leídos.
- **file_utils.py:** funciones para resolver rutas relativas que llegan de las peticiones a absolutas internas al sistema.

- **validator.py**: define una clase que actúa como auxiliar en la validación del modelo de datos.
- **Storage**: sistema gemelo al ya visto en el sistema principal, salvo por algunas particularidades.
 - **Core**: igual que en el sistema principal pero programado en Python.
 - **Parquet**: contiene a `parquet_storage_backend.py` con soporte para Parquet.
 - **PlotExport**: contiene a `plot_export_storage_backend.py` que da soporte a la exportación de gráficos.

A.7. Visión global de Auto-Plot

Se explica el sistema en base al diagrama de sus componentes contenido en la [Figura 56](#). En cuanto a su análisis se aprecia un componente principal donde están la mayoría de componentes del sistema, y externamente se muestran varios recursos adicionales.

En primer lugar, el elemento **app** funciona como punto de entrada a la aplicación y gestiona la recepción de peticiones que son encaminadas a sus correspondientes controladores en base a la ruta con la que llegan. Por ello, se muestra una conexión con **Controllers** que se encuentra dentro del componente **Api**.

Por lo que respecta a los componentes internos de la aplicación, **Api** contiene el núcleo de la misma, integrado por **Controllers**, **DTOS** y **Models**. Este núcleo mantiene una conexión con el componente **Utils** destinado a brindar funciones de utilidad. En cuanto a los componentes internos del módulo que se describe, **Controllers** mantiene conexiones con **Models**, ya que todos los controladores hacen uso del modelo de datos, así como de **DTOS** para poder acceder a los elementos enviados en los cuerpos de las peticiones y así pasarlos a los modelos para su interpretación. Finalmente, los controladores utilizan el componente **Plotters** para acceder a los servicios que generan gráficos.

Plotters, a su vez, mantiene conexiones con **Utils** para poder acceder a las funciones de utilidad, así como su elemento interno `heat_map_plotter`, centrado en la construcción de mapas de calor, se comunica con el fichero externo **Resources**, con otro fichero con información georreferenciada necesaria para la construcción de estos mapas llamada `/countries_shapes`. Cabe destacar que todos los elementos internos de **Plotters** mantienen una conexión con el elemento `abstract_plotter` ya explicado en la sección anterior.

En cuanto al componente **Utils**, contiene algunos elementos individuales con funciones de utilidad y un elemento más destacado llamado **Storage**, que brinda acceso al sistema de almacenamiento. Puede apreciarse que su arquitectura es idéntica a la desarrollada por el sistema principal en Scala. Tan solo muestra diferentes *back-ends* que permiten el trabajo con ficheros **Parquet** y exportaciones de gráficos. Es importante mencionar que este sistema, al igual que su gemelo en Scala, se comunican con el sistema de almacenamiento donde está la carpeta compartida `/data` con todos los datos del programa.

Para terminar, **Config** tiene conexiones con **Controllers**, **DTOS**, **Plotters** y **Utils** y da acceso a datos estáticos como constantes o enumeraciones.

A.8. Algoritmo para crear mapas de calor

Antes de la explicación, es importante mencionar las herramientas que se han usado junto con Python para desarrollar este algoritmo.

- **Plotly**: permite crear gráficos interactivos y exportarlos.
- **Geopandas**: permite trabajar con datos georreferenciados fácilmente.
- **Scipy**: proporciona múltiples algoritmos enfocados en la computación científica y técnica, entre ellos, optimización, interpolación o estadística.
- **Numpy**: ofrece la posibilidad de trabajar de forma optimizada con matrices de N dimensiones, además de otras características matemáticas.
- **Pykrige**: algoritmos de Kriging para interpolación espacial.
- **Rasterio**: permite el tratamiento de datos geoespaciales y el uso de algoritmos para transformaciones geométricas.
- **OpenCV**: proporciona herramientas de trabajo con imágenes y visión artificial.

El desarrollo del algoritmo comienza con la creación de un **array** bidimensional de tamaño 1000×1000 para el caso de que la localización sea el territorio continental, y de tamaño 2000×2000 para las Islas Canarias. Posteriormente se delimitan los márgenes de latitud y longitud de los territorios con valores en formato decimal establecidos como constantes. [89] A continuación se procede a leer un archivo vectorial georreferenciado en el directorio `Resources/countries_shapes`, con la librería Geopandas mediante la función `read_file()`. [24] El archivo tiene el contorno de todos los países del mapa mundial [37], y sobre su representación interna una vez leído, se escoge el territorio español seleccionando la clave `'SOVEREIGNT'` con valor `'Spain'`. A continuación se procede a crear tres **arrays** haciendo uso de la librería Numpy con los valores de latitud y longitud de la localización de las estaciones y el valor en cuestión a representar, así como también se generan las descripciones dinámicas con `_generate_point_info()`. Después, se procesan los datos con un filtrado con el algoritmo K vecinos cercanos, que usa la clase `cKDTree()` de la librería Scipy [32], eliminando datos muy cercanos entre sí (límite de 0,05). Prosigue con la inserción de los valores en la malla de interpolación con `linspace()` de Numpy y se aplica el proceso de interpolación usando un algoritmo de *Kriging* ordinario (muy usado en disciplinas como la minería, geografía o meteorología para procesos de interpolación compleja), proporcionado por la librería PyKriging. [38, 31] De este modo, usando los ejemplos de la librería se confecciona un modelo de interpolación con la clase `OrdinaryKriging`, sin embargo, su coste computacional es muy elevado, por lo que se debe aplicar una estrategia de divide y vencerás para conseguir la interpolación particionando la malla en bloques de 200×200 , aplicando el proceso a cada sección y uniéndolo en una sola posteriormente. Al resultado, con los datos vectoriales de los bordes del territorio que fueron tomados al principio, se aplica una máscara geométrica con la clase `geometry_mask` de la librería Rasterio [26], de modo que todos los valores que caigan fuera del contorno de la máscara, se les aplica valor 0. Finalmente, usando la librería OpenCV se crea una imagen en formato `base64` interpretando los valores de tal modo que los 0 son color blanco y el resto un degradado con tonos térmicos llamado `COLORMAP_JET` mediante la función `applyColorMap()`. [29] Sobre esta imagen generada se usa un algoritmo de aplicación de contorno que pinta una línea negra como borde del territorio con las funciones `findContours()` y `drawContours()`. [30] El gráfico final se genera como una distribución de puntos y como fondo la imagen generada. [43]

Apéndice B

Diagramas del proyecto

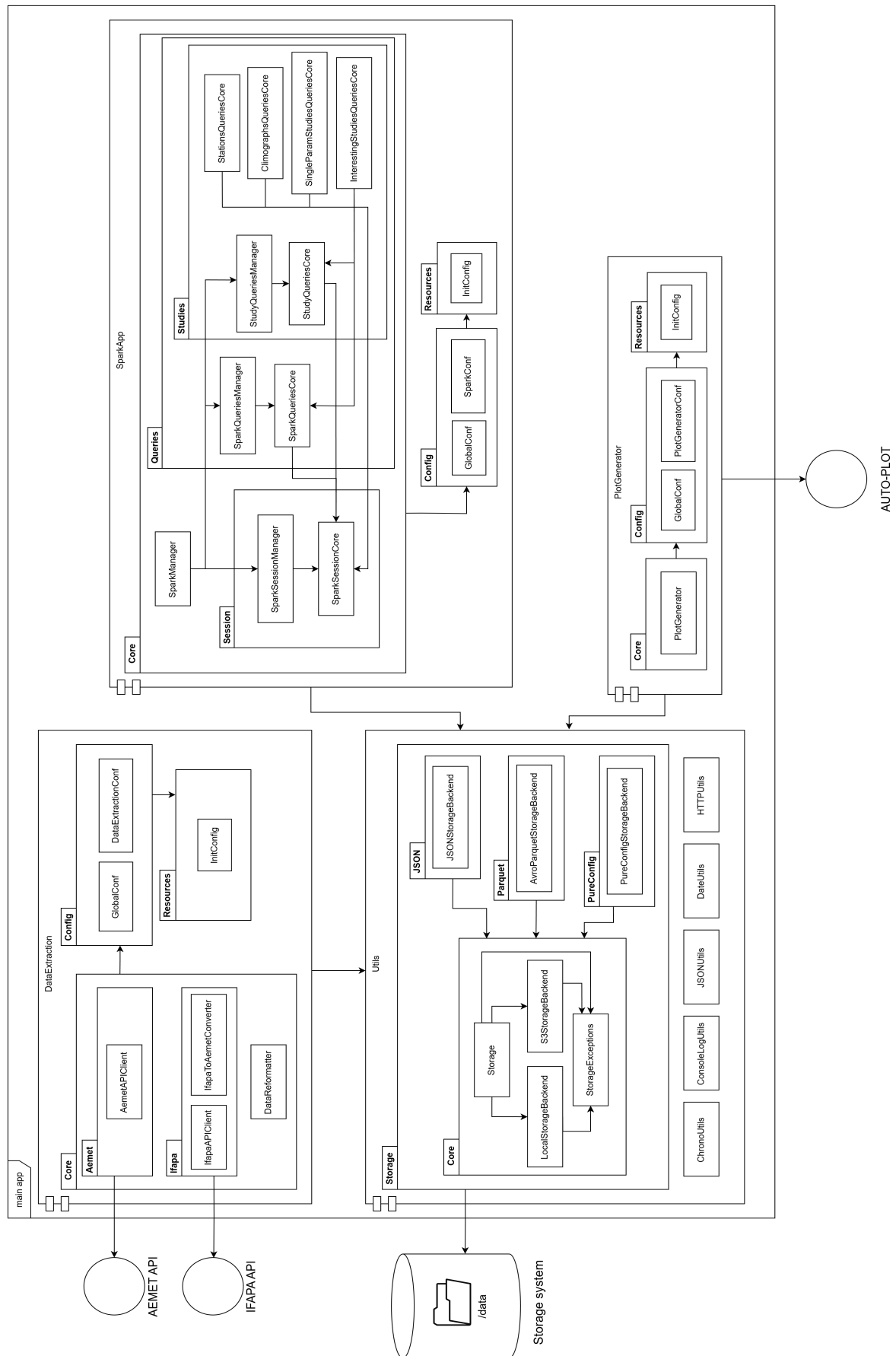


Figura 38: Diagrama de componentes del sistema principal

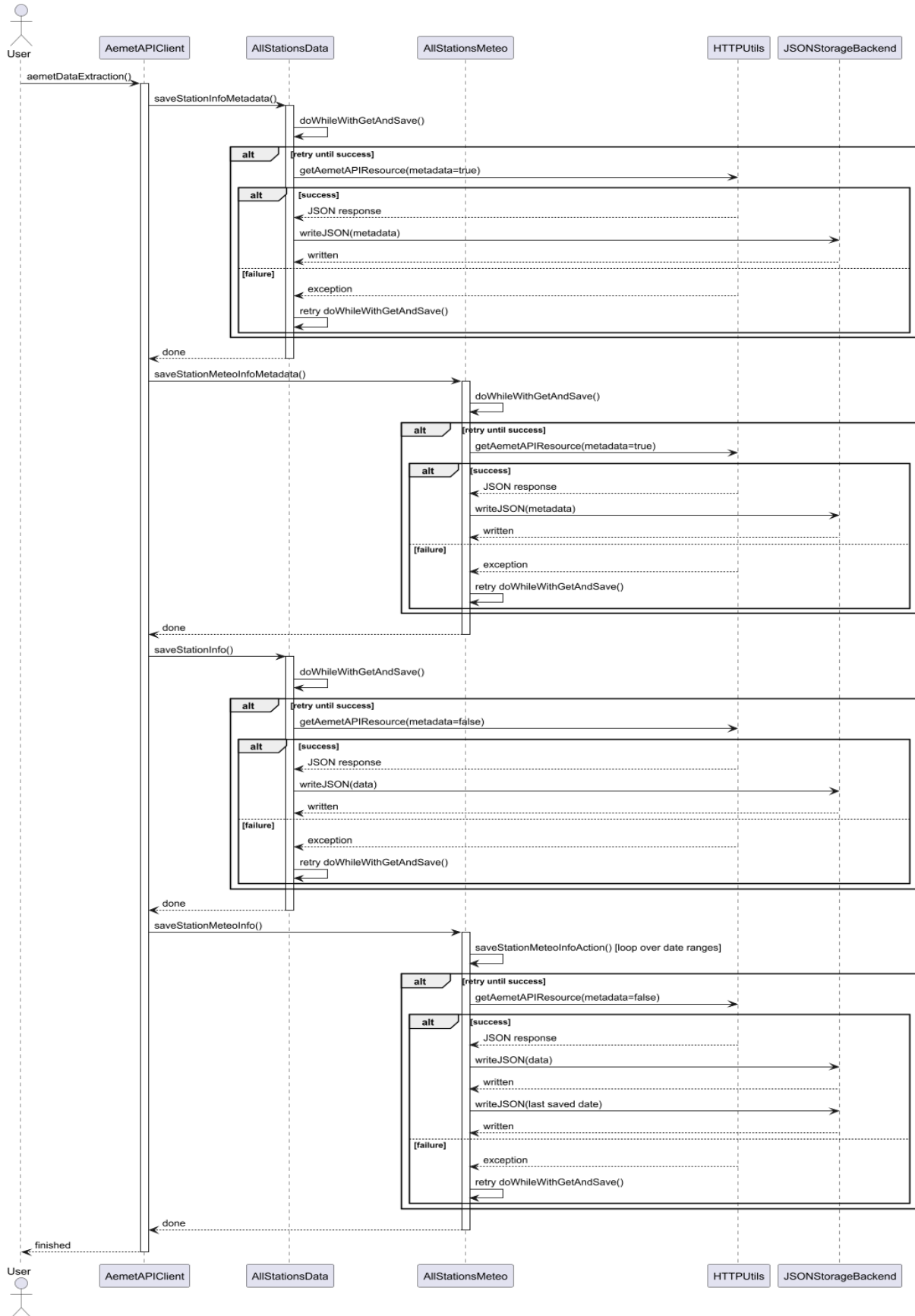


Figura 39: Diagrama de ejecución del proceso de extracción de datos de la API de AEMET

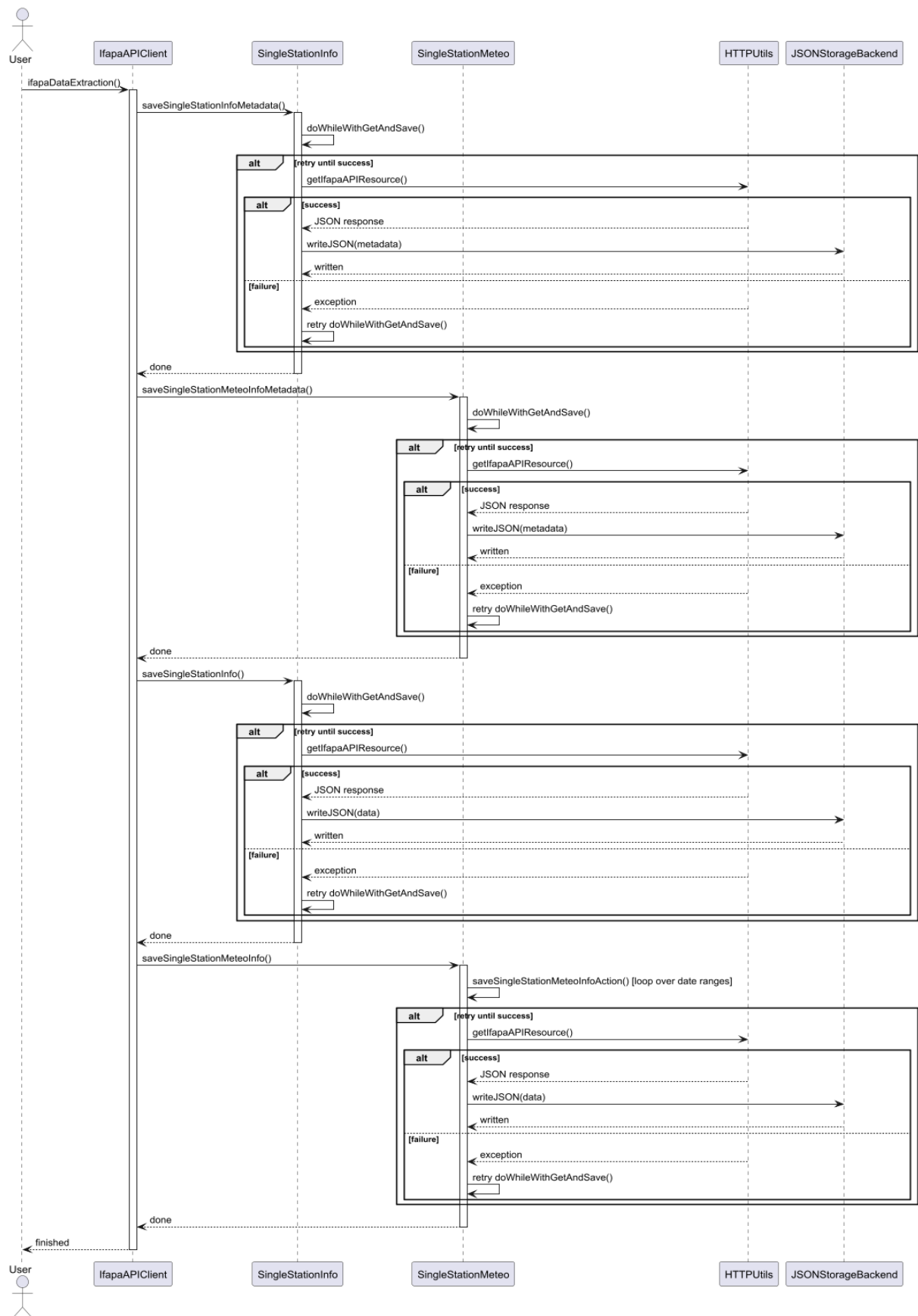


Figura 40: Diagrama de ejecución del proceso de extracción de datos de la API de IFAPA

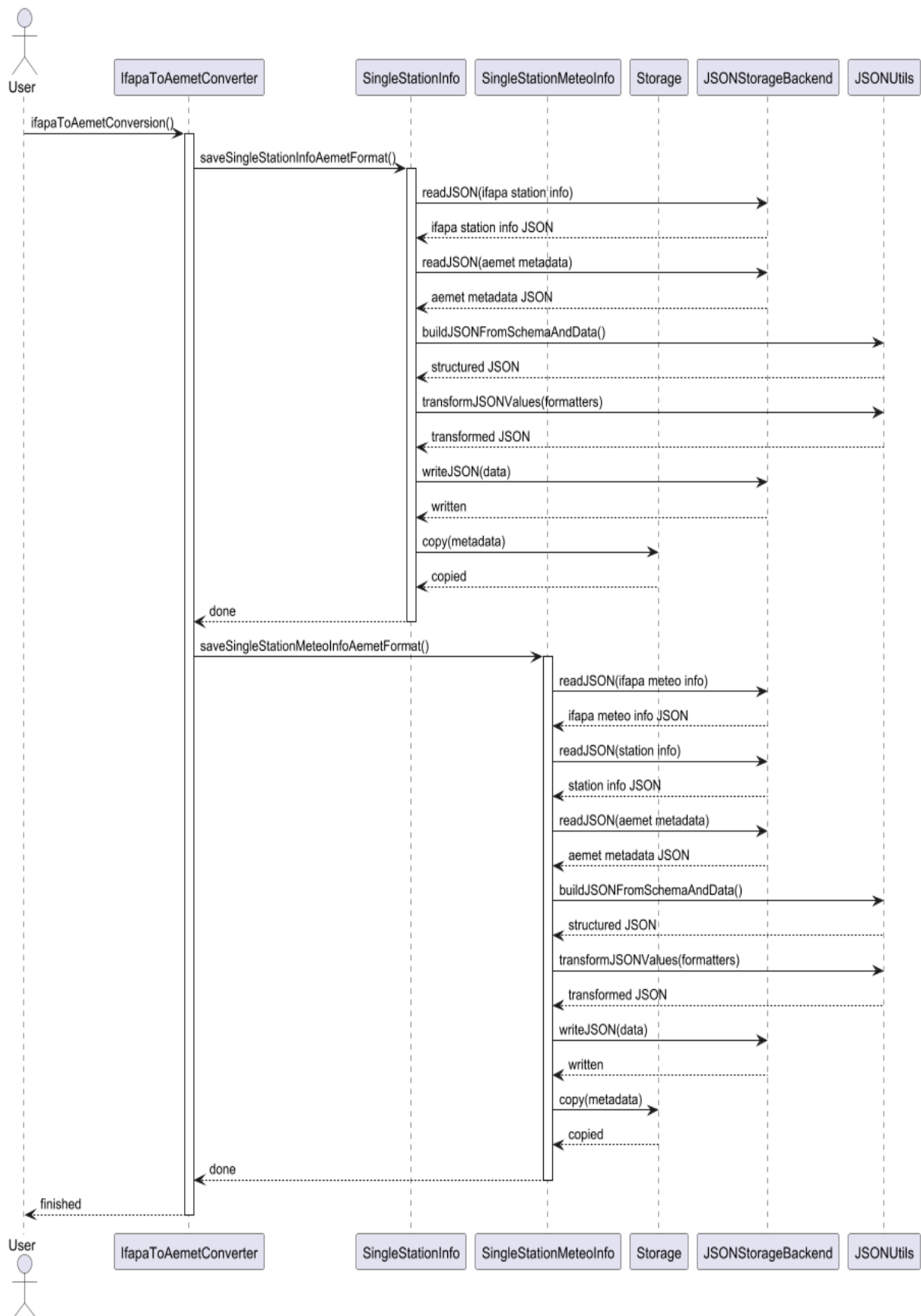


Figura 41: Diagrama de ejecución del proceso de conversión del esquema de datos de IFAPA al de AEMET

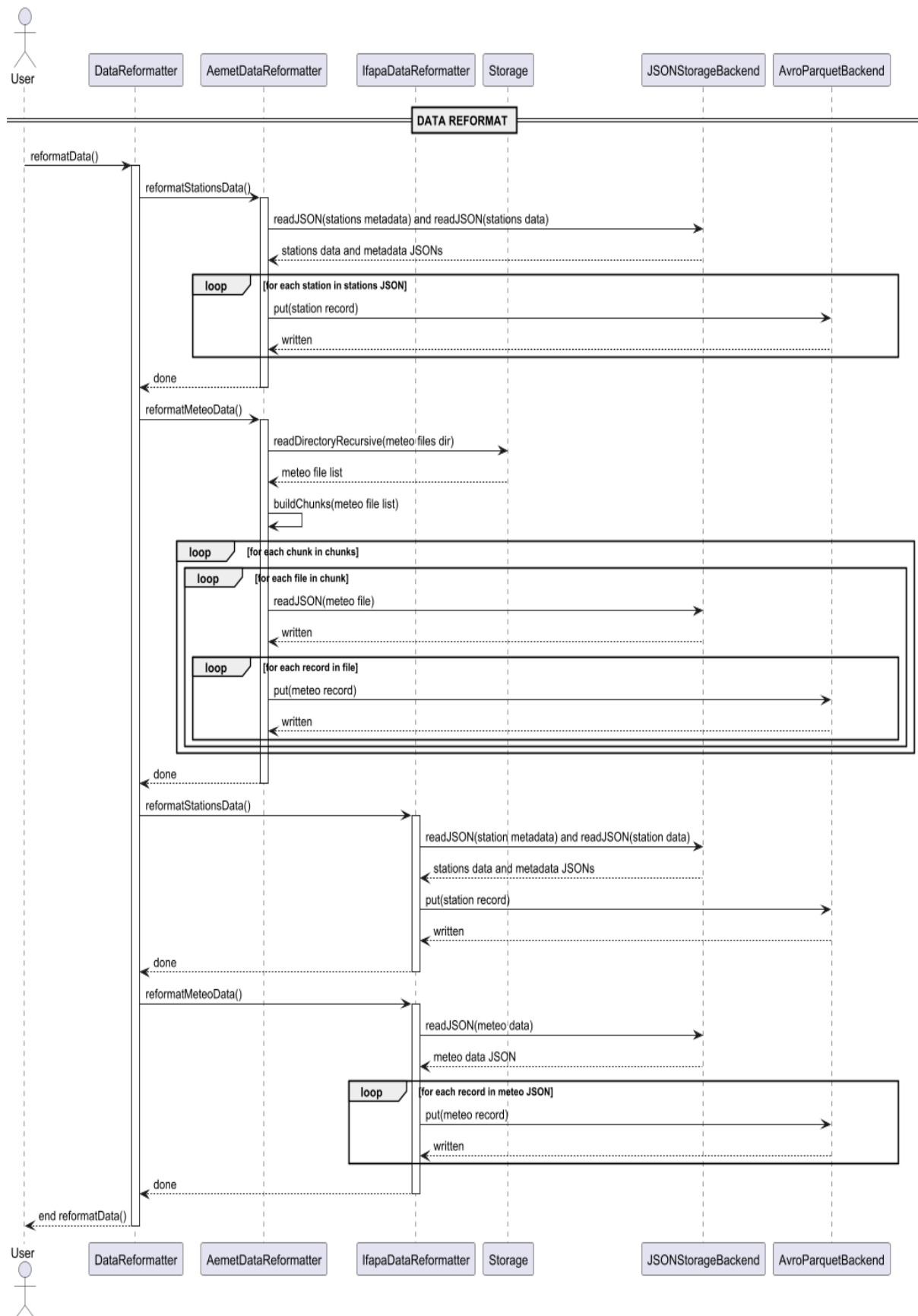


Figura 42: Diagrama de ejecución del proceso de transformación del formato de los datos

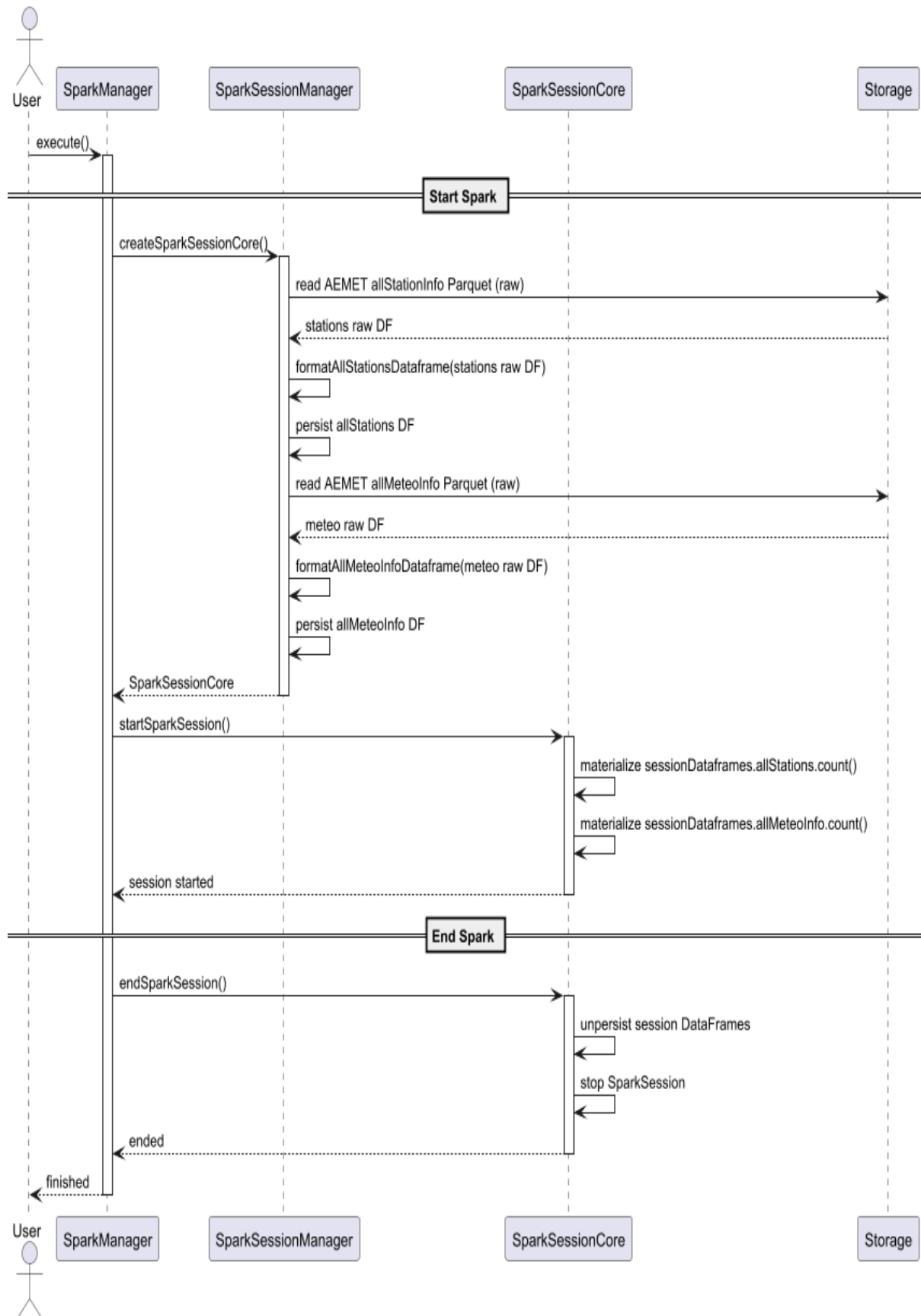


Figura 43: Diagrama de ejecución del proceso de arranque y finalización de la sesión de Spark

Execution of getStationInfoById()

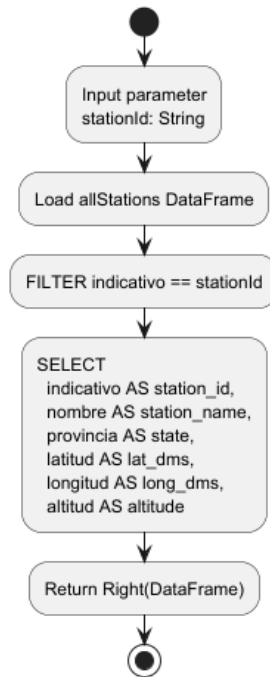


Figura 44: Diagrama de actividad de la consulta `getStationInfoById()`

Execution of getStationCountByColumnInLapse()

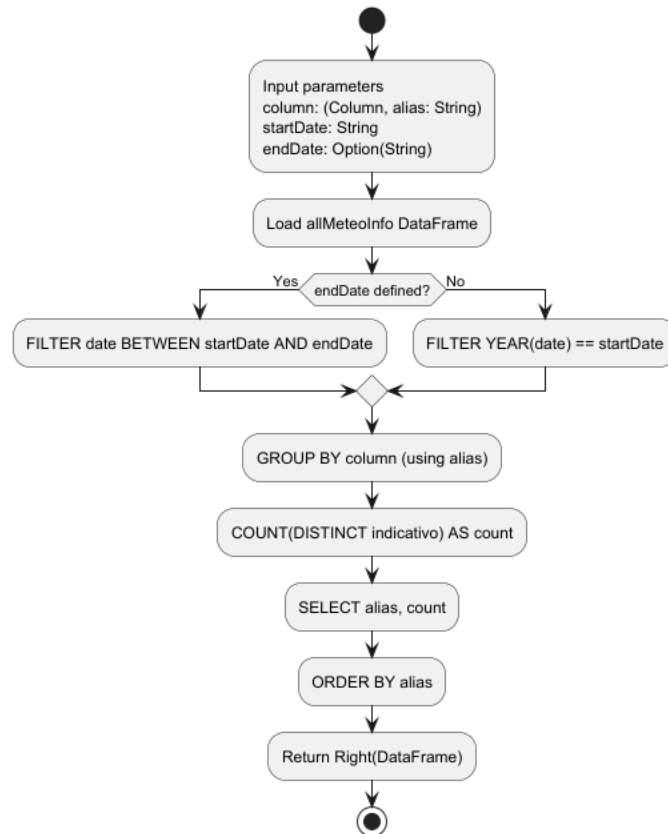


Figura 45: Diagrama de actividad de la consulta `getStationCountByColumnInLapse()`

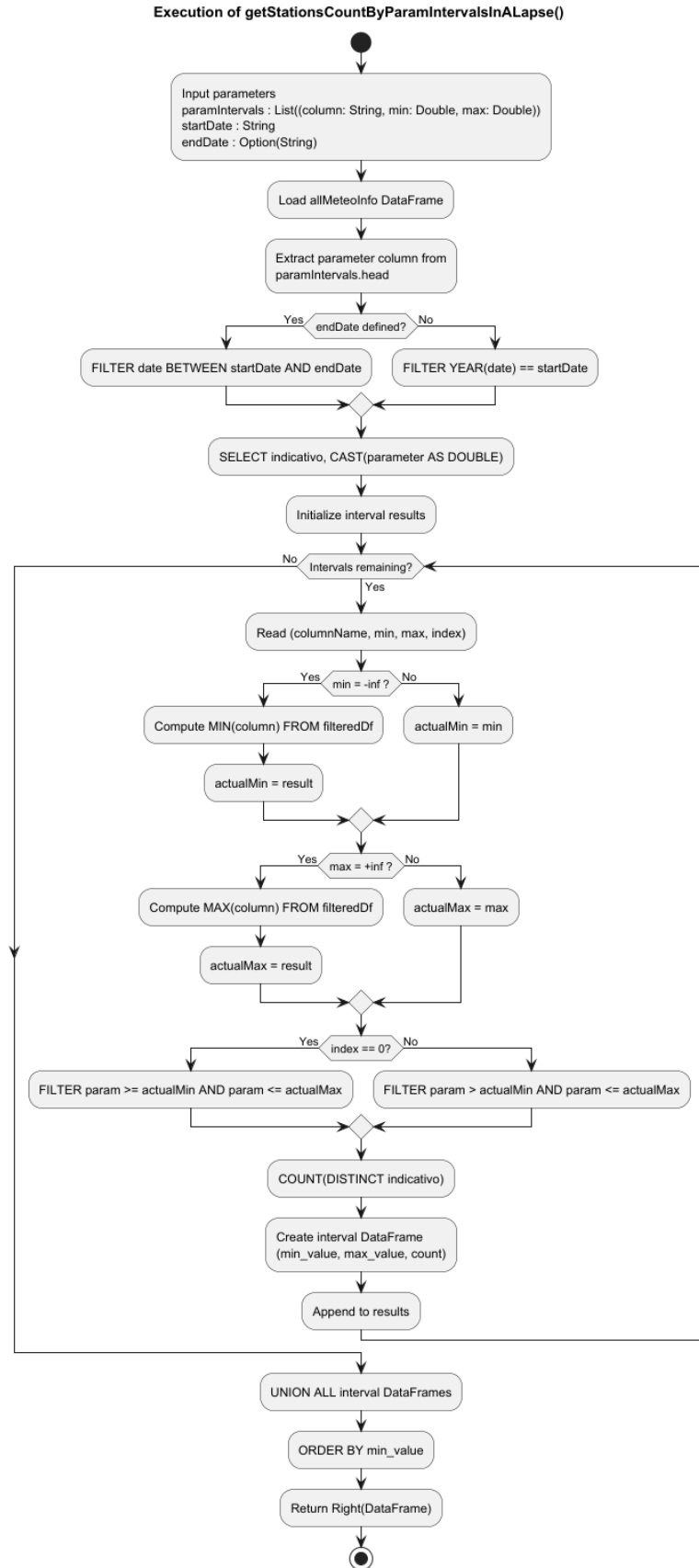


Figura 46: Diagrama de actividad de la consulta `getStationsCountByParamIntervalsInALapse()`

Execution of getStationMonthlyAvgTempAndSumPrecInAYear()

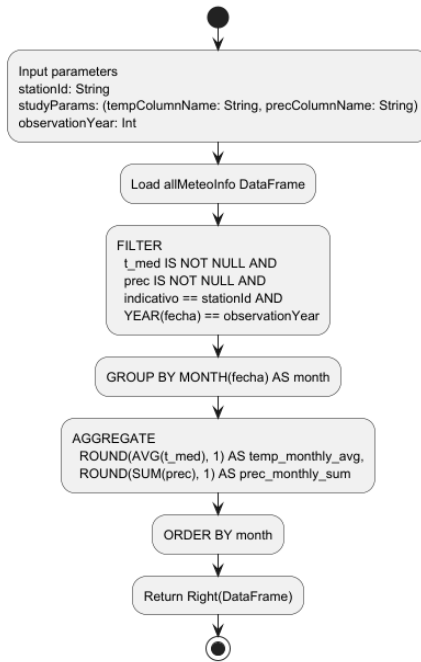


Figura 47: Diagrama de actividad de la consulta `getStationMonthlyAvgTempAndSumPrecInAYear()`

Execution of getClimateParamInALapseById()

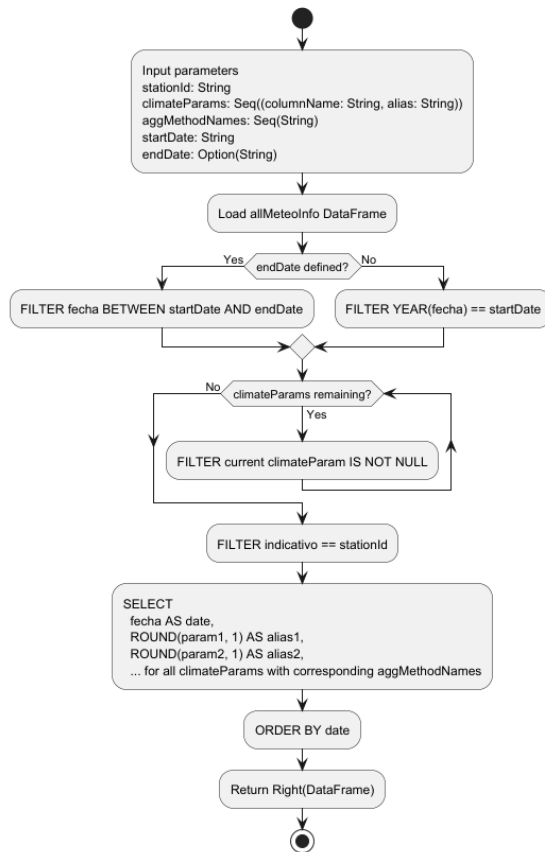


Figura 48: Diagrama de actividad de la consulta `getClimateParamInALapseById()`

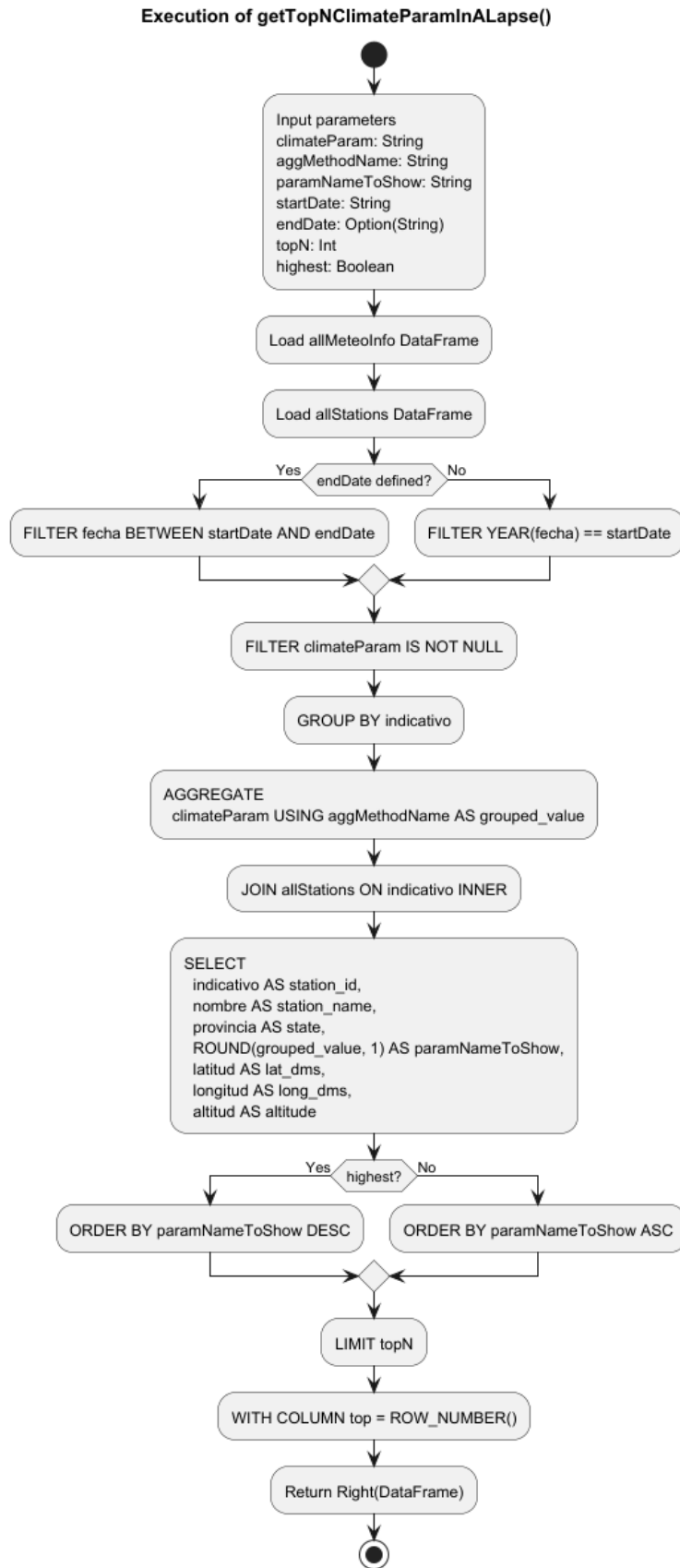


Figura 49: Diagrama de actividad de la consulta `getTopNClimateParamInALapse()`

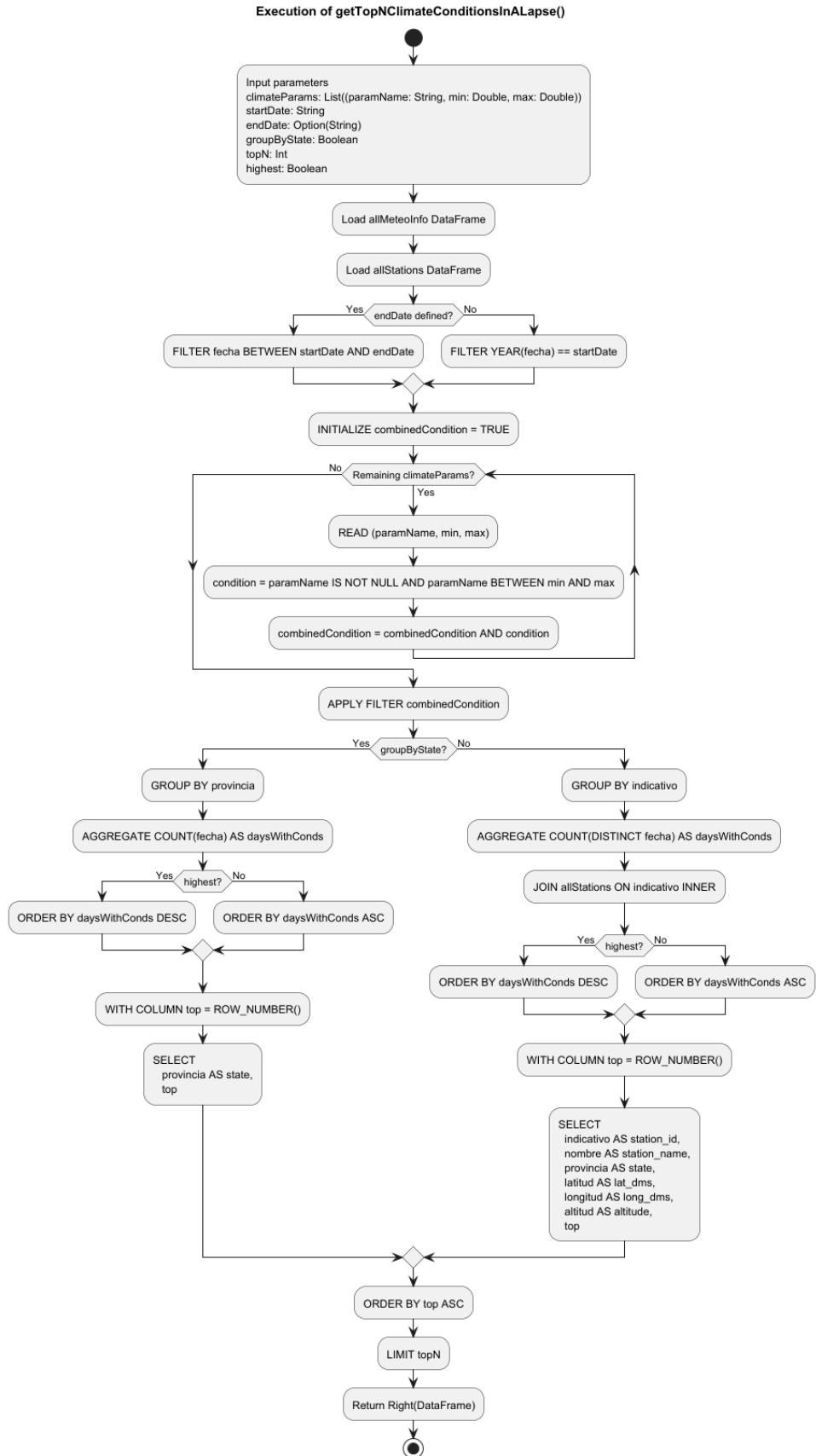


Figura 50: Diagrama de actividad de la consulta getTopNClimateConditionsInALapse()

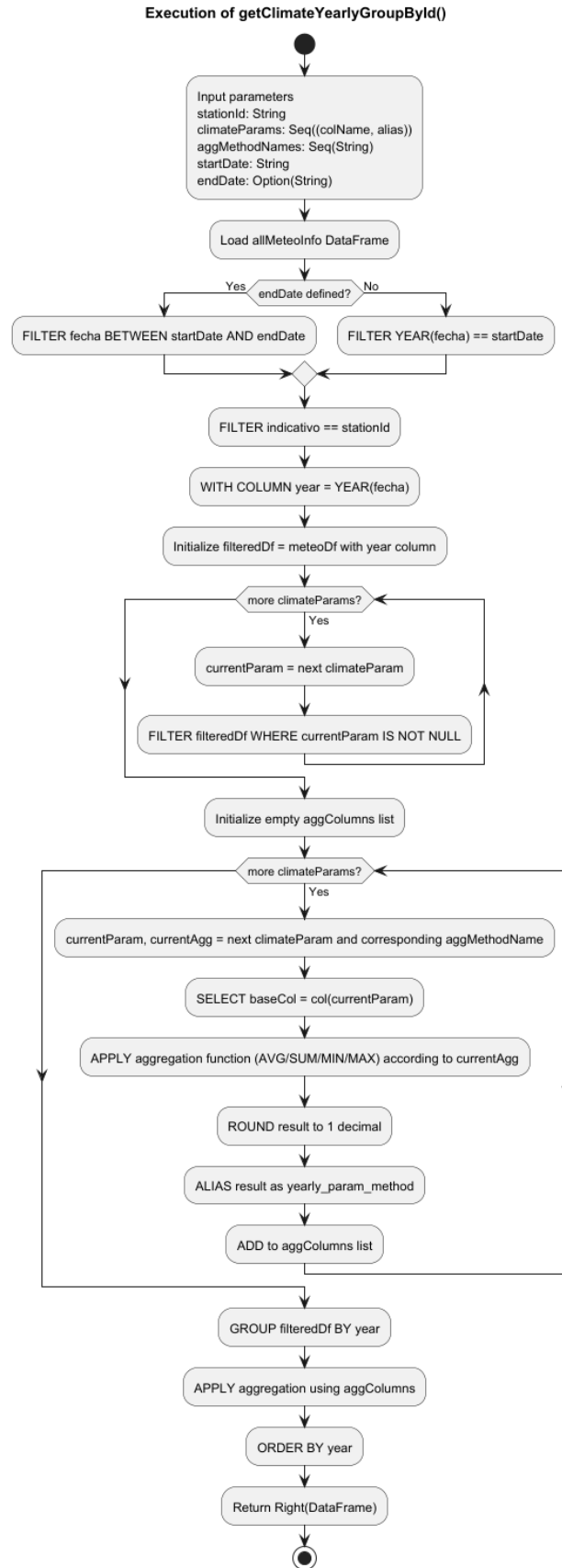


Figura 51: Diagrama de actividad de la consulta `getClimateYearlyGroupById()`

Execution of getAllStationsByStatesAvgClimateParamInALapse()

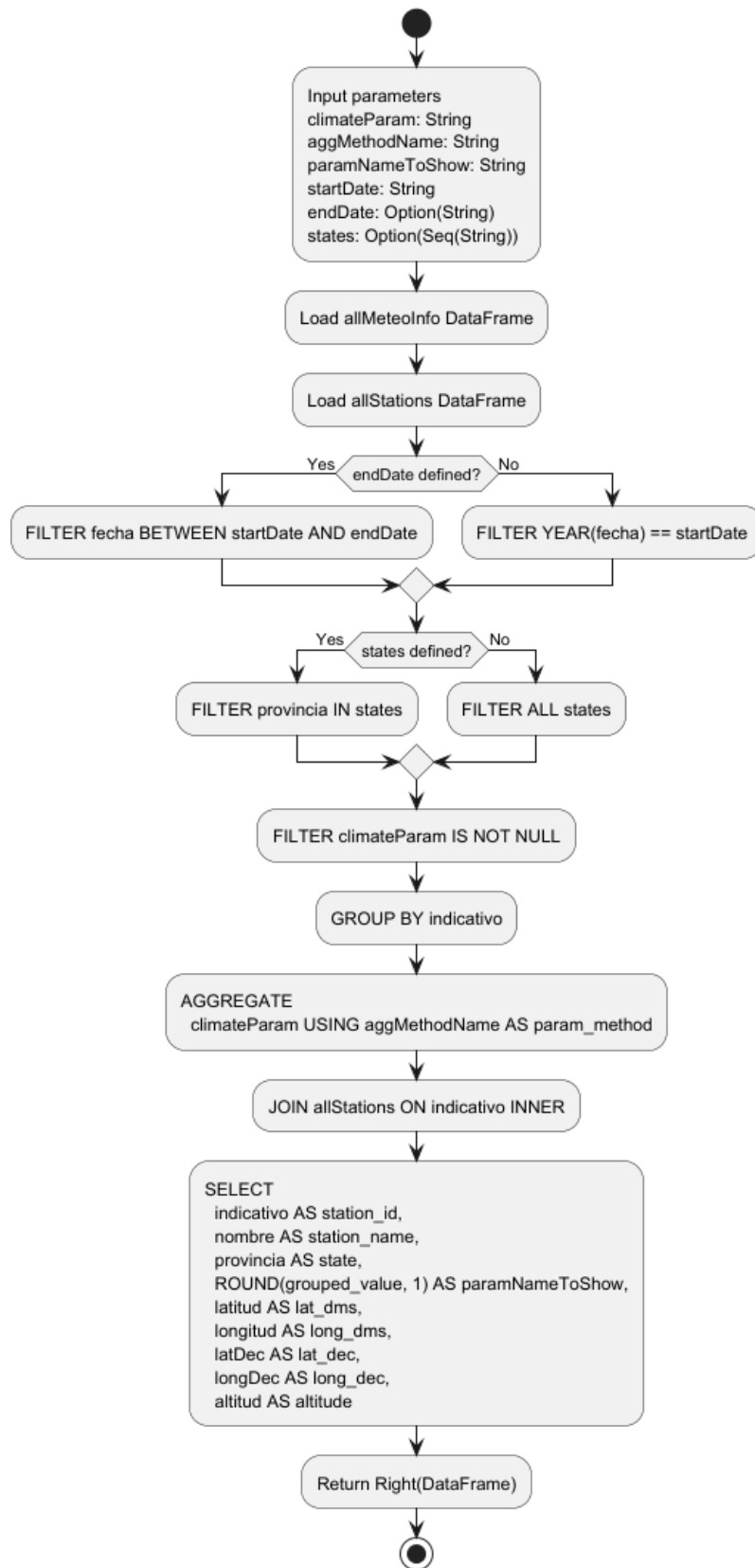


Figura 52: Diagrama de actividad de la consulta `getAllStationsByStatesAvgClimateParamInALapse()`

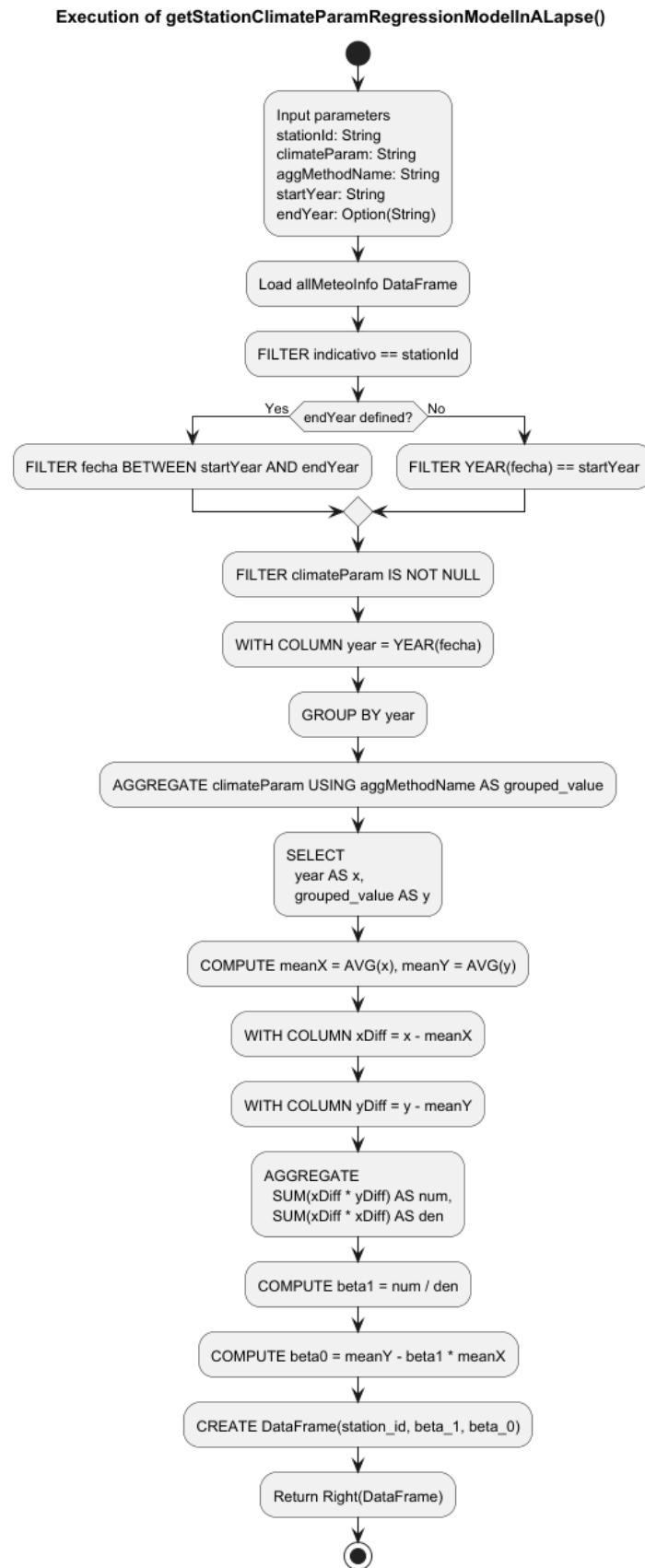


Figura 53: Diagrama de actividad de la consulta `getStationClimateParamRegressionModelInALapse()`

Execution of getTopNClimateParamIncrementInAYearLapse()

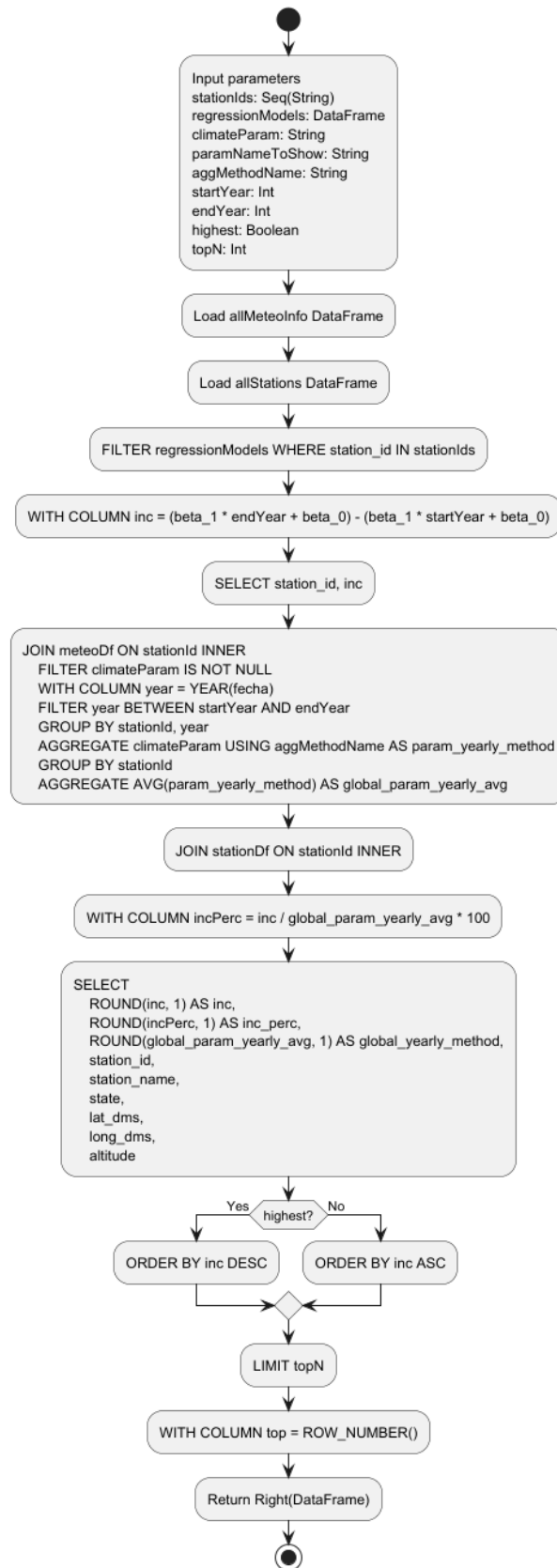


Figura 54: Diagrama de actividad de la consulta getTopNClimateParamIncrementInAYearLapse()

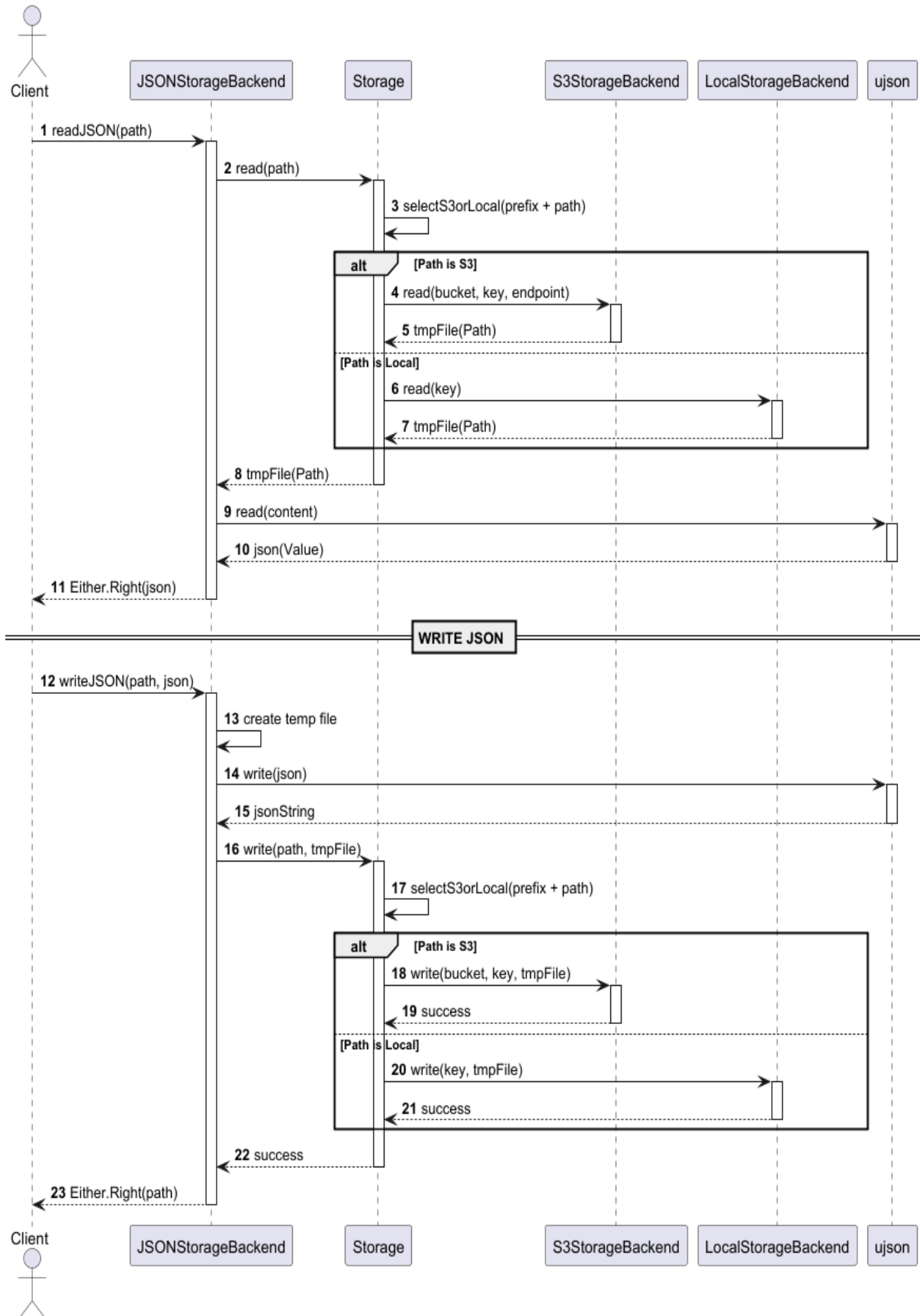


Figura 55: Diagrama de ejecución del sistema de almacenamiento para archivos JSON

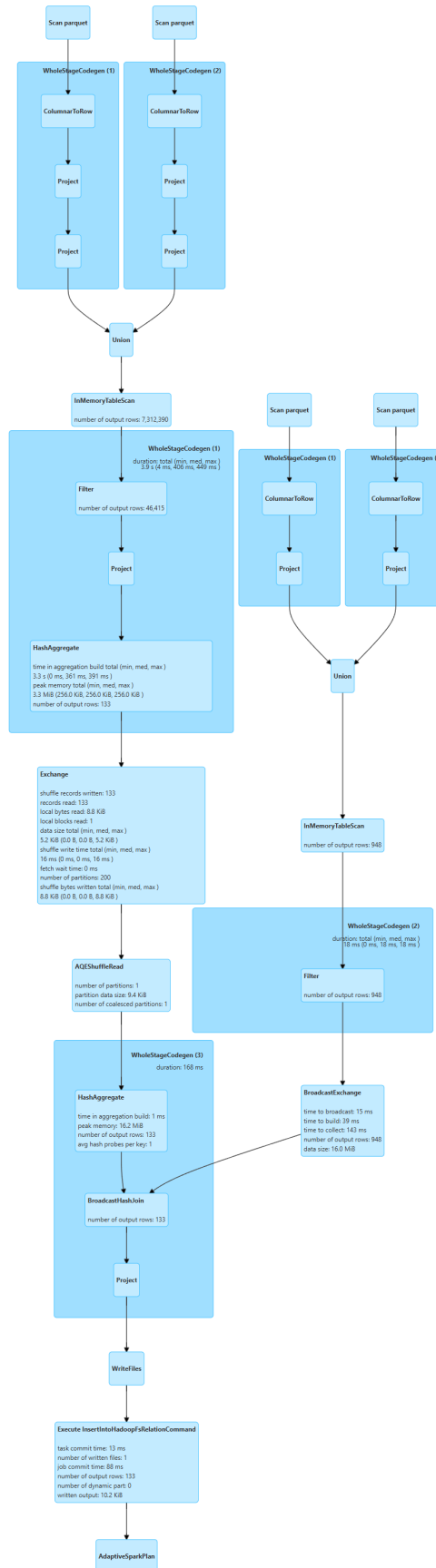


Figura 58: Captura de pantalla de la SparkUI con el DAG de la consulta `getAllStationsByStatesAvgClimateParamInALapse()`


```

1 {
2   "src" : {
3     "path" : "/spark/stations/count_evol",
4     "axis" : {
5       "x" : {
6         "name" : "year",
7         "format" : "timestamp_year"
8       },
9       "y" : {
10        "name" : "count"
11      }
12    }
13  },
14  "dest" : {
15    "path" : "/stations/count_evol",
16    "filename" : "plot",
17    "export_png" : true
18  },
19  "style" : {
20    "lettering" : {
21      "title" : "Evolution of the meteorological stations count in spanish
22        territory",
23      "subtitle" : "From 1973 to 2024",
24      "x_label" : "Year",
25      "y_label" : "Count"
26    },
27    "figure" : {
28      "name" : "Station count",
29      "color" : "#4169E1"
30    },
31    "margin" : {
32      "left" : 80,
33      "right" : 80,
34      "top" : 80,
35      "bottom" : 80
36    },
37    "legend" : {
38      "y_offset" : -0.3
39    }
40  }
41 }

```

Figura 57: JSON presente en el cuerpo de una petición para la generación de un gráfico lineal en Auto-Plot

Bibliografía

- [1] AEMET. *Preguntas frecuentes sobre OpenData AEMET*. <https://opendata.aemet.es/centrodedescargas/docs/FAQs220424.pdf>. Accedido: 14 de enero de 2026. 2025 (vid. pág. 10).
- [2] Agencia Estatal de Meteorología (AEMET). *AEMET OpenData – Centro de Descargas de Productos Meteorológicos*. <https://opendata.aemet.es/centrodedescargas/productosAEMET?>. Accedido: 14 de enero de 2026. 2026 (vid. pág. 10).
- [3] Agencia Estatal de Meteorología (AEMET). *Nuestra Historia*. Consulta: 5 de enero de 2026. Agencia Estatal de Meteorología. 2026. URL: https://www.aemet.es/es/conocenos/nuestra_historia (vid. pág. 2).
- [4] Agencia Estatal de Meteorología (AEMET). *Nuestros Recursos*. Consulta: 5 de enero de 2026. Agencia Estatal de Meteorología. 2026. URL: <https://www.aemet.es/es/conocenos/recursos> (vid. pág. 2).
- [5] Agencia Estatal de Meteorología (AEMET). *Quiénes somos*. Consulta: 5 de enero de 2026. Agencia Estatal de Meteorología. 2026. URL: https://www.aemet.es/es/conocenos/quienes_somos (vid. pág. 2).
- [6] Agencia Estatal de Meteorología (AEMET) e Instituto de Meteorologia de Portugal. *Atlas Climático Ibérico*. Apartado “Climas”, p. 15. Madrid: Ministerio de Medio Ambiente, y Medio Rural y Marino, 2011. ISBN: 978-84-7837-079-5. DOI: 10.31978/784-11-002-5. URL: https://www.aemet.es/documentos/es/conocermas/recursos_en_linea/publicaciones_y_estudios/publicaciones/Atlas-climatologico/Atlas.pdf (vid. pág. 13).
- [7] Amazon Web Services, Inc. *AWS Documentation*. Consulta: 13 de enero de 2026. Amazon Web Services, Inc. 2026. URL: <https://aws.amazon.com/documentation/> (vid. pág. 6).
- [8] Ramiro Romero Fresneda y José Vicente Moreno García Andrés Chazarra Bernabé Belinda Lorenzo Mariño. *Evolución de los climas de Köppen en España en el periodo 1951-2020*. Nota Técnica n.º 37. Consultado el 14 de enero de 2026. Información utilizada del apartado “Clasificación climática de Köppen-Geiger” (página 8). Agencia Estatal de Meteorología (AEMET), 2022. URL: https://www.aemet.es/documentos/es/conocermas/recursos_en_linea/publicaciones_y_estudios/publicaciones/NT_37_AEMET/NT_37_AEMET.pdf (vid. pág. 13).
- [9] Apache Software Foundation. *Apache Spark - Big Data Processing*. Consulta: 13 de enero de 2026. Apache Software Foundation. 2026. URL: <https://spark.apache.org/> (vid. pág. 5).

- [10] Viktor Ardelean. *Listing All AWS S3 Objects in a Bucket Using Java*. Tutorial sobre cómo listar objetos en un bucket de Amazon S3 usando Java y AWS SDK V2. Consultado el 8 de febrero de 2026. 2024. URL: <https://www.baeldung.com/java-aws-s3-list-bucket-objects> (vid. pág. 86).
- [11] Atlassian. *Trello*. Consulta: 13 de enero de 2026. Atlassian. 2026. URL: <https://www.atlassian.com/es/software/trello> (vid. pág. 7).
- [12] Baeldung. *Introduction to the Java Date/Time API*. Consultado el 5 de febrero de 2026. 2023. URL: <https://www.baeldung.com/java-8-date-time-intro> (vid. pág. 40).
- [13] Baeldung. *Introduction to the Java NIO2 File API*. Artículo técnico sobre la API de archivos NIO2 en Java. Consultado el 8 de febrero de 2026. 2024. URL: <https://www.baeldung.com/java-nio-2-file-api> (vid. pág. 85).
- [14] Hardik Singh Behl. *Amazon S3 With Java*. Tutorial sobre el uso de Amazon S3 en aplicaciones Java publicado en Baeldung. Consultado el 8 de febrero de 2026. 2023. URL: <https://www.baeldung.com/java-aws-s3> (vid. pág. 86).
- [15] Jerónimo López Bejarano. *Trabajando con ficheros Parquet en Java usando Avro*. Blog personal. Consultado el 29 de enero de 2026. 2019. URL: <https://www.jerolba.com/trabajando-con-ficheros-parquet-en-java-usando-avro/> (vid. pág. 32).
- [16] Bill Chambers y Matei Zaharia. *Spark: The Definitive Guide: Big Data Processing Made Simple*. Consulta: 13 de enero de 2026. Sebastopol, CA: O'Reilly Media, 2018. ISBN: 9781491912218. URL: <https://www.oreilly.com/library/view/spark-the-definitive-guide/9781491912218/> (vid. pág. 5).
- [17] Clima.com – Meteopedia. *Alta presión atmosférica*. Consulta: 15 de enero de 2026. Clima.com. 2026. URL: <https://www.clima.com/meteopedia/alta-presion-atmosferica> (vid. pág. 14).
- [18] Clima.com – Meteopedia. *Baja presión atmosférica*. Consulta: 15 de enero de 2026. Clima.com. 2026. URL: <https://www.clima.com/meteopedia/baja-presion-atmosferica> (vid. pág. 14).
- [19] Code & Chill (Diego Coder). *Intrdocucción a la programación funcional: conceptos básicos*. Consultado el 24 de enero de 2026. 2023. URL: <https://medium.com/@diego.coder/fundamentos-de-la-programaci%C3%B3n-funcional-137366c85279> (vid. pág. 21).
- [20] Flask-RESTx Contributors. *Export Swagger Specifications — Flask-RESTx Documentation*. Sección de la documentación oficial de Flask-RESTx sobre cómo exportar especificaciones Swagger. Consultado el 12 de febrero de 2026. 2026. URL: <https://flask-restx.readthedocs.io/en/latest/swagger.html#export-swagger-specifications> (vid. pág. 44).
- [21] Flask-RESTx Contributors. *Flask-RESTx Examples*. Ejemplos de uso en la documentación oficial de Flask-RESTx. Consultado el 12 de febrero de 2026. 2026. URL: <https://flask-restx.readthedocs.io/en/latest/example.html> (vid. pág. 46).
- [22] Flask-RESTx Contributors. *Flask-RESTx Quickstart*. Guía rápida de Flask-RESTx — documentación oficial. Consultado el 12 de febrero de 2026. 2026. URL: <https://flask-restx.readthedocs.io/en/latest/quickstart.html> (vid. pág. 43).

-
- [23] Flask-RESTx Contributors. *Flask-RESTx Scaling*. Guía sobre escalado en Flask-RESTx — documentación oficial. Consultado el 12 de febrero de 2026. 2026. URL: <https://flask-restx.readthedocs.io/en/latest/scaling.html> (vid. pág. 43).
- [24] GeoPandas Contributors. *Input/Output (I/O) — GeoPandas User Guide*. Guía de entrada/salida en la documentación oficial de GeoPandas. Consultado el 13 de febrero de 2026. 2026. URL: https://geopandas.org/en/stable/docs/user_guide/io.html (vid. pág. 91).
- [25] PureConfig Contributors. *PureConfig Documentation*. Documentación oficial de la librería PureConfig para gestión de configuraciones en Scala. Consultado el 11 de febrero de 2026. 2025. URL: <https://pureconfig.github.io/docs/> (vid. pág. 88).
- [26] Rasterio Contributors. *rasterio.features — Rasterio API Documentation*. Documentación oficial de Rasterio sobre el módulo rasterio.features. Consultado el 13 de febrero de 2026. 2026. URL: <https://rasterio.readthedocs.io/en/latest/api/rasterio.features.html> (vid. pág. 91).
- [27] sbt Contributors. *sbt Documentation*. Documentación oficial de sbt (Scala Build Tool). Consultado el 16 de febrero de 2026. 2026. URL: <https://www.scala-sbt.org/1.x/docs/index.html> (vid. pág. 51).
- [28] shadcn/ui Contributors. *shadcn/ui — Component Documentation*. Documentación de componentes de la librería shadcn/ui. Consultado el 16 de febrero de 2026. 2026. URL: <https://ui.shadcn.com/docs/components> (vid. pág. 50).
- [29] OpenCV Developers. *Colormap — Image Processing (imgproc) Module in OpenCV*. Documentación oficial de OpenCV sobre mapas de color en el módulo de procesamiento de imágenes. Consultado el 13 de febrero de 2026. 2026. URL: https://docs.opencv.org/4.x/d3/d50/group__imgproc__colormap.html (vid. pág. 91).
- [30] OpenCV Developers. *Finding Contours — OpenCV Python Tutorials*. Tutorial oficial de OpenCV sobre cómo encontrar contornos con Python. Consultado el 13 de febrero de 2026. 2026. URL: https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html (vid. pág. 91).
- [31] PyKriging Developers. *Ordinary Kriging Example — PyKriging Documentation*. Ejemplo de uso de Ordinary Kriging en la documentación oficial de PyKriging. Consultado el 13 de febrero de 2026. 2026. URL: https://geostat-framework.readthedocs.io/projects/pykriging/en/stable/examples/00_ordinary.html#sphx-glr-examples-00-ordinary-py (vid. pág. 91).
- [32] SciPy Developers. *scipy.spatial.cKDTree — SciPy v1 Reference Guide*. Documentación oficial de SciPy para la clase cKDTree. Consultado el 13 de febrero de 2026. 2026. URL: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.html> (vid. pág. 91).
- [33] The pandas Development Team. *pandas.read_parquet — pandas v2.x Documentation*. Documentación oficial de pandas para la función read_parquet. Consultado el 13 de febrero de 2026. 2026. URL: https://pandas.pydata.org/docs/reference/api/pandas.read_parquet.html (vid. pág. 50).

- [34] Docker, Inc. *Docker Documentation*. Consulta: 14 de enero de 2026. Docker, Inc. 2026. URL: <https://docs.docker.com/> (vid. pág. 6).
- [35] Inc. Docker. *Compose file reference*. Documentación oficial de referencia del fichero Compose de Docker. Consultado el 16 de febrero de 2026. 2026. URL: <https://docs.docker.com/reference/compose-file/> (vid. pág. 52).
- [36] Inc. Docker. *Dockerfile Reference*. Documentación oficial de referencia de Dockerfile. Consultado el 16 de febrero de 2026. 2026. URL: <https://docs.docker.com/reference/dockerfile/> (vid. pág. 52).
- [37] Natural Earth. *10m Cultural Vectors — Admin 0 — Countries*. Datos vectoriales de países (nivel admin 0) en resolución 1:10 m de Natural Earth. Consultado el 13 de febrero de 2026. 2026. URL: <https://www.naturalearthdata.com/downloads/10m-cultural-vectors/10m-admin-0-countries/> (vid. pág. 91).
- [38] Esri. *Understanding Ordinary Kriging*. Guía de Esri ArcGIS Pro sobre el método de Kriging ordinario. Consultado el 13 de febrero de 2026. 2026. URL: <https://pro.arcgis.com/es/pro-app/latest/help/analysis/geostatistical-analyst/understanding-ordinary-kriging.htm> (vid. pág. 91).
- [39] Apache Software Foundation. *org.apache.spark.sql Scala API Documentation*. Documentación oficial de Apache Spark. Consultado el 1 de febrero de 2026. 2025. URL: <https://spark.apache.org/docs/latest/api/scala/org/apache/spark/sql/index.html> (vid. pág. 35).
- [40] Apache Software Foundation. *pyspark.sql.DataFrameReader.parquet — PySpark Documentation*. Documentación oficial de Apache Spark. Consultado el 31 de enero de 2026. 2025. URL: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrameReader.parquet.html> (vid. pág. 34).
- [41] Apache Software Foundation. *Spark SQL, DataFrames and Datasets Guide*. Documentación oficial de Apache Spark. Consultado el 1 de febrero de 2026. 2025. URL: <https://spark.apache.org/docs/latest/sql-programming-guide.html> (vid. pág. 35).
- [42] Francesco Galgani. *Introduction to Java Parquet (Formerly Parquet MR)*. Baeldung. Consultado el 29 de enero de 2026. 2025. URL: <https://www.baeldung.com/java-intro-to-apache-parquet> (vid. págs. 32, 87).
- [43] Stack Overflow user G-Corral y contributors. *Interpolated map using points Python*. Pregunta y respuestas en Stack Overflow sobre cómo crear un mapa interpolado a partir de puntos en Python. Consultado el 13 de febrero de 2026. 2020. URL: <https://stackoverflow.com/questions/63236525/interpolated-map-using-points-python> (vid. pág. 91).
- [44] GeeksforGeeks. *How to Create Spark Session in Scala*. GeeksforGeeks. Consultado el 31 de enero de 2026. 2022. URL: <https://www.geeksforgeeks.org/scala/how-to-create-spark-session-in-scala/> (vid. pág. 34).
- [45] Git SCM. *Git Documentation*. Consulta: 13 de enero de 2026. Git SCM. 2026. URL: <https://git-scm.com/doc> (vid. pág. 7).
- [46] GitHub. *GitHub Copilot*. Página oficial. Consultado el 9 de marzo de 2026. 2026. URL: <https://github.com/features/copilot> (vid. pág. 7).

-
- [47] GitHub, Inc. *GitHub Documentation*. Consulta: 13 de enero de 2026. GitHub, Inc. 2026. URL: <https://docs.github.com/> (vid. pág. 7).
- [48] Li Haoyi. *fansi: Scala/Scala.js library for manipulating Fancy Ansi colored strings*. Repositorio de GitHub de Fansi. Consultado el 5 de febrero de 2026. 2025. URL: <https://github.com/com-lihaoyi/fansi> (vid. pág. 41).
- [49] Li Haoyi. *uJson — uPickle Documentation*. Documentación oficial de uPickle (sección uJson). Consultado el 5 de febrero de 2026. 2024. URL: <https://com-lihaoyi.github.io/upickle/#uJson> (vid. pág. 42).
- [50] Li Haoyi. *uPickle Documentation - Defaults*. Documentación oficial de uPickle. Consultado el 5 de febrero de 2026. 2024. URL: <https://com-lihaoyi.github.io/upickle/#Defaults> (vid. pág. 39).
- [51] IBM. *¿Qué es la ciencia de datos?* Consulta: 13 de enero de 2026. IBM. 2026. URL: <https://www.ibm.com/es-es/topics/data-science> (vid. pág. 3).
- [52] Dmitry Ignatovich. *Web Components — A Dive Into Modern Component Architecture*. Artículo técnico en Medium sobre Web Components y arquitectura de componentes modernos. Consultado el 15 de febrero de 2026. 2021. URL: <https://medium.com/@ignatovich.dm/web-components-a-dive-into-modern-component-architecture-529b8e983671> (vid. pág. 50).
- [53] Instituto Andaluz de Investigación y Formación Agraria, Pesquera, Alimentaria y de la Producción Ecológica (IFAPA). *Funciones, Actividad y Planificación estratégica*. Consulta: 5 de enero de 2026. Instituto Andaluz de Investigación y Formación Agraria, Pesquera, Alimentaria y de la Producción Ecológica. 2026. URL: <https://www.juntadeandalucia.es/agriculturaypesca/ifapa/web/funciones-%20actividad-y-planificacion-estrategica> (vid. pág. 3).
- [54] Instituto Andaluz de Investigación y Formación Agraria, Pesquera, Alimentaria y de la Producción Ecológica (IFAPA). *Quiénes somos*. Consulta: 5 de enero de 2026. Instituto Andaluz de Investigación y Formación Agraria, Pesquera, Alimentaria y de la Producción Ecológica. 2026. URL: <https://www.juntadeandalucia.es/agriculturaypesca/ifapa/web/quienes-somos> (vid. pág. 3).
- [55] Instituto Andaluz de Investigación y Formación Agraria, Pesquera, Alimentaria y de la Producción Ecológica (IFAPA). *Red de Información Agroclimática de Andalucía*. Consulta: 5 de enero de 2026. Instituto Andaluz de Investigación y Formación Agraria, Pesquera, Alimentaria y de la Producción Ecológica. 2026. URL: <https://www.juntadeandalucia.es/agriculturaypesca/ifapa/riaweb/web/> (vid. pág. 3).
- [56] Instituto de Investigación y Formación Agraria y Pesquera (IFAPA). *Servicio Web REST – RIAWS API (Swagger UI) – Provincia Controller*. <https://www.juntadeandalucia.es/agriculturaypesca/ifapa/riaws/swagger-ui.html>. Accedido: 14 de enero de 2026. 2026 (vid. pág. 11).
- [57] JetBrains s.r.o. *JetBrains Product Documentation*. Consulta: 13 de enero de 2026. JetBrains s.r.o. 2026. URL: <https://www.jetbrains.com/help/> (vid. pág. 7).
- [58] jserranohidalgo. *spark-intro/workflow-example*. Repositorio de ejemplo para flujo de trabajo con Apache Spark. Consultado el 19 de febrero de 2026. 2026. URL: <https://github.com/jserranohidalgo/spark-intro/tree/master/workflow-example> (vid. pág. 54).

- [59] Julia Martins. *¿Qué es la metodología Kanban y cómo funciona?* Consultado el 22 de enero de 2026. 2026. URL: <https://asana.com/es/resources/what-is-kanban> (vid. pág. 19).
- [60] Niko Karanatsios. *Introduction to React Library*. Consulta: 13 de enero de 2026. Medium. 2023. URL: <https://medium.com/%40karanatsios.n/introduction-to-react-library-410f3ad57101> (vid. pág. 6).
- [61] Yadu Krishnan. *Load Configuration Files in Scala Using PureConfig*. Tutorial sobre cómo cargar archivos de configuración en Scala usando la librería PureConfig. Consultado el 12 de febrero de 2026. 2025. URL: <https://www.baeldung.com/scala/pureconfig-load-config-files> (vid. pág. 88).
- [62] LaTeX Project Team. *An introduction to LaTeX*. Consulta: 13 de enero de 2026. LaTeX Project. 2026. URL: <https://www.latex-project.org/about/> (vid. pág. 8).
- [63] LocalStack. *AWS Java — Integración de AWS SDKs con LocalStack*. Documentación oficial de LocalStack sobre cómo usar AWS SDK para Java con LocalStack. Consultado el 8 de febrero de 2026. 2025. URL: <https://docs.localstack.cloud/aws/integrations/aws-sdks/java/> (vid. pág. 86).
- [64] Martin Odersky, Philippe Altherr, Vincent Cremet, Gilles Dubochet, Burak Emir, Philipp Haller, Stéphane Micheloud, Nikolay Mihaylov, Adriaan Moors, Lukas Rytz, Michel Schinz, Erik Stenman, Matthias Zenger, Iain McGinniss. *Scala Language Specification, Version 2.13*. Consulta: 13 de enero de 2026. Scala Center / Lightbend. 2026. URL: <https://scala-lang.org/files/archive/spec/2.13/> (vid. pág. 4).
- [65] Mozilla Developer Network. *JavaScript Guide*. Consulta: 13 de enero de 2026. Mozilla Foundation. 2026. URL: <https://developer.mozilla.org/es/docs/Web/JavaScript/Guide> (vid. pág. 5).
- [66] National Geographic Society. *The Science and Art of Meteorology*. Consulta: 5 de enero de 2026. National Geographic Education. 2026. URL: <https://education.nationalgeographic.org/resource/science-art-meteorology/> (vid. pág. 2).
- [67] Naveen Nelamali. *PySpark withColumn() Usage with Examples*. Spark By Examples. Consultado el 31 de enero de 2026. 2024. URL: <https://sparkbyexamples.com/pyspark/pyspark-withcolumn/> (vid. pág. 34).
- [68] Oracle. *¿Qué es Big Data?* Consulta: 13 de enero de 2026. Oracle. 2025. URL: <https://www.oracle.com/es/big-data/what-is-big-data/> (vid. pág. 4).
- [69] Overleaf Ltd. *About Overleaf*. Consulta: 13 de enero de 2026. Overleaf. 2026. URL: <https://www.overleaf.com/about> (vid. pág. 8).
- [70] Pallets Projects. *Flask Documentation*. Consulta: 13 de enero de 2026. Pallets Projects. 2026. URL: <https://flask.palletsprojects.com/> (vid. pág. 5).
- [71] PlantUML contributors. *PlantUML (FAQ)*. Consulta: 13 de enero de 2026. PlantUML. 2026. URL: <https://plantuml.com/faq> (vid. pág. 7).
- [72] Plotly. *Bar Charts in Python*. Guía de gráficos de barras con Plotly en Python. Consultado el 12 de febrero de 2026. 2026. URL: <https://plotly.com/python/bar-charts/> (vid. pág. 48).

-
- [73] Plotly. *Interactive HTML Export with Plotly in Python*. Guía de exportación de gráficos interactivos a HTML con Plotly en Python. Consultado el 22 de febrero de 2026. 2026. URL: <https://plotly.com/python/interactive-html-export/> (vid. pág. 50).
- [74] Plotly. *Line Charts in Python*. Guía de gráficos de líneas con Plotly en Python. Consultado el 12 de febrero de 2026. 2026. URL: <https://plotly.com/python/line-charts/> (vid. pág. 48).
- [75] Plotly. *Pie Charts in Python*. Guía de gráficos de sectores (pie) con Plotly en Python. Consultado el 12 de febrero de 2026. 2026. URL: <https://plotly.com/python/pie-charts/> (vid. pág. 48).
- [76] Plotly. *Static Image Export with Plotly in Python*. Guía de exportación de gráficos como imágenes estáticas con Plotly en Python. Consultado el 22 de febrero de 2026. 2026. URL: <https://plotly.com/python/static-image-export/> (vid. pág. 50).
- [77] Plotly. *Table Subplots in Python*. Guía de subplots con tablas en Plotly para Python. Consultado el 12 de febrero de 2026. 2026. URL: <https://plotly.com/python/table-subplots/> (vid. pág. 48).
- [78] Plotly Technologies Inc. *Plotly About Us*. Consulta: 13 de enero de 2026. Plotly Technologies Inc. 2026. URL: <https://plotly.com/about-us/> (vid. pág. 5).
- [79] Postman, Inc. *Postman Docs*. Consulta: 13 de enero de 2026. Postman, Inc. 2026. URL: <https://learning.postman.com/> (vid. pág. 7).
- [80] Python Software Foundation. *The Python Programming Language*. Consulta: 13 de enero de 2026. Python Software Foundation. 2026. URL: <https://www.python.org/about/> (vid. pág. 4).
- [81] Real Academia Española. *Meteorología*. Versión 23.8.1 en línea, consulta: 5 de enero de 2026. Diccionario de la lengua española. 2026. URL: <https://dle.rae.es/meteorolog%C3%ADa?m=form> (vid. pág. 2).
- [82] Sarah Laoyan. *Qué es la metodología waterfall y cuándo utilizarla*. Consultado el 22 de enero de 2026. 2025. URL: <https://asana.com/es/resources/waterfall-project-management-methodology> (vid. pág. 19).
- [83] Amazon Web Services. *Adding an Apache Spark Step to an EMR Cluster*. Guía oficial de AWS sobre cómo agregar un paso de Spark (spark-submit) en EMR. Consultado el 19 de febrero de 2026. 2026. URL: <https://docs.aws.amazon.com/emr/latest/ReleaseGuide/emr-spark-submit-step.html> (vid. pág. 54).
- [84] Amazon Web Services. *AWS CLI Command Reference — s3*. Referencia de comandos AWS CLI para Amazon S3. Consultado el 19 de febrero de 2026. 2026. URL: <https://docs.aws.amazon.com/cli/latest/reference/s3/> (vid. pág. 53).
- [85] Amazon Web Services. *Boto3 Quickstart — Python SDK for AWS*. Guía rápida de inicio con Boto3 — documentación oficial de AWS. Consultado el 22 de febrero de 2026. 2026. URL: <https://docs.aws.amazon.com/boto3/latest/guide/quickstart.html> (vid. pág. 50).
- [86] Amazon Web Services. *Ejemplos de Amazon S3 usando la AWS SDK for Java*. Documentación oficial de AWS SDK for Java (sección Amazon S3). Consultado el 8 de febrero de 2026. 2025. URL: https://docs.aws.amazon.com/es_es/sdk-for-java/v1/developer-guide/examples-s3.html (vid. pág. 86).

- [87] SoftwareMill. *How sttp client works — sttp 4 documentation*. Documentación oficial de sttp 4. Consultado el 5 de febrero de 2026. 2025. URL: <https://sttp.softwaremill.com/en/v4.0.0/how.html> (vid. pág. 42).
- [88] Vercel. *Next.js Documentation*. Consulta: 13 de enero de 2026. Vercel. 2026. URL: <https://nextjs.org/docs> (vid. págs. 6, 50).
- [89] Wikipedia. *Anexo: Puntos extremos de España*. Lista y descripción de los puntos extremos de España. Consultado el 13 de febrero de 2026. 2026. URL: https://es.wikipedia.org/wiki/Anexo:Puntos_extremos_de_Espa%C3%B1a (vid. pág. 91).
- [90] Wikipedia contributors. *Amazon Web Services*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: https://es.wikipedia.org/wiki/Amazon_Web_Services (vid. pág. 6).
- [91] Wikipedia contributors. *Apache Spark*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: https://es.wikipedia.org/wiki/Apache_Spark (vid. pág. 5).
- [92] Wikipedia contributors. *Big data*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: https://es.wikipedia.org/wiki/Big_data (vid. pág. 4).
- [93] Wikipedia contributors. *ChatGPT*. Consulta: 9 de marzo de 2026. Wikipedia. 2026. URL: <https://es.wikipedia.org/wiki/ChatGPT> (vid. pág. 7).
- [94] Wikipedia contributors. *Ciencia de datos*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: https://es.wikipedia.org/wiki/Ciencia_de_datos (vid. pág. 3).
- [95] Wikipedia contributors. *Climograma*. Consulta: 14 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://es.wikipedia.org/wiki/Climograma> (vid. págs. 13, 16).
- [96] Wikipedia contributors. *Datos abiertos*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: https://es.wikipedia.org/wiki/Datos_abiertos (vid. pág. 4).
- [97] Wikipedia contributors. *Diagrams.net*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://en.wikipedia.org/wiki/Diagrams.net> (vid. pág. 7).
- [98] Wikipedia contributors. *Docker (software)*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: https://es.wikipedia.org/wiki/Docker_%28software%29 (vid. pág. 6).
- [99] Wikipedia contributors. *Era de la información*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: https://es.wikipedia.org/wiki/Era_de_la_informaci%C3%B3n (vid. pág. 3).
- [100] Wikipedia contributors. *Flask*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://es.wikipedia.org/wiki/Flask> (vid. pág. 5).
- [101] Wikipedia contributors. *Git*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://es.wikipedia.org/wiki/Git> (vid. pág. 7).
- [102] Wikipedia contributors. *GitHub*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://es.wikipedia.org/wiki/GitHub> (vid. pág. 7).

- [103] Wikipedia contributors. *JetBrains*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://es.wikipedia.org/wiki/JetBrains> (vid. pág. 7).
- [104] Wikipedia contributors. *LaTeX*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://es.wikipedia.org/wiki/LaTeX> (vid. pág. 8).
- [105] Wikipedia contributors. *Next.js*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://es.wikipedia.org/wiki/Next.js> (vid. pág. 6).
- [106] Wikipedia contributors. *Overleaf*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2025. URL: <https://es.wikipedia.org/wiki/Overleaf> (vid. pág. 8).
- [107] Wikipedia contributors. *PlantUML*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://en.wikipedia.org/wiki/PlantUML> (vid. pág. 7).
- [108] Wikipedia contributors. *Plotly*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://en.wikipedia.org/wiki/Plotly> (vid. pág. 5).
- [109] Wikipedia contributors. *Postman (software)*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: [https://es.wikipedia.org/wiki/Postman_\(software\)](https://es.wikipedia.org/wiki/Postman_(software)) (vid. pág. 7).
- [110] Wikipedia contributors. *React*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://es.wikipedia.org/wiki/React> (vid. pág. 6).
- [111] Wikipedia contributors. *Regresión lineal*. Consulta: 2 de febrero de 2026. Wikimedia Foundation. 2026. URL: https://es.wikipedia.org/wiki/Regresi%C3%B3n_lineal (vid. pág. 83).
- [112] Wikipedia contributors. *Trello*. Consulta: 13 de enero de 2026. Wikimedia Foundation. 2026. URL: <https://es.wikipedia.org/wiki/Trello> (vid. pág. 7).